

RTL Design Sherpa

RTL Clock Domain Crossing Module Reference — Rev 0.92

October 2026

Table of Contents

1 Clock Domain Crossing (CDC) 23
1.1 Choosing a technique 24
1.1.1 Quick reference 24
1.2 Reset Considerations 26
1.2.1 Why encoding decides reset behavior 26
1.2.2 Async FIFO reset behavior 30
1.2.3 Silicon evidence 31
1.2.4 Safe usage checklist for 2-phase 31
1.2.5 Recovering an existing design 31
1.3 cdc_synchronizer 33
1.4 cdc_open_loop 35
1.5 cdc_2_phase_handshake 37
1.5.1 Shared handshake interface 38
1.5.2 Theory of operation 38
1.5.3 Swapping between variants 39
1.6 cdc_4_phase_handshake 40
1.6.1 Theory of operation 41
1.7 Async FIFO Pointers (Gray and Johnson) 42
1.7.1 Gray vs Johnson encoding 42
1.7.2 Sizing the depth 44
1.7.3 Walkthrough: is a non-power-of-2 FIFO worth it at 512 bits? 45
1.8 SDC constraints 50

1.9 Common mistakes....52
1.10 Verification status....53
1.11 Related Modules....54
1.12 Testing....56
1.13 References....57
1.14 Navigation....59
2 Binary-to-Gray converter (bin2gray.sv)....60
2.1 Overview....60
2.2 Parameters....61
2.3 Ports....62
2.4 Functional Description....63
2.4.1 Block diagram....63
2.4.2 Mathematical definition....63
2.4.3 Why Gray code?....63
2.4.4 Gray code sequences....64
2.4.5 Core logic....65
2.4.6 Companion converter: gray2bin....65
2.5 Timing Characteristics....67
2.6 Usage Examples....68
2.7 Design Notes....69
2.8 Related Modules....70
2.9 Testing....71
2.9.1 Comprehensive Test Bench....71
2.9.2 Property-Based Verification....72

2.9.3 Coverage Model 74
2.9.4 Test files 74
2.10 References 75
2.11 Navigation 76
3 Gray-to-binary converter (gray2bin.sv) 77
3.1 Overview 77
3.2 Parameters 78
3.3 Ports 79
3.4 Functional Description 80
3.4.1 Block diagram 80
3.4.2 Conversion algorithm 81
3.4.3 Worked conversions 81
3.4.4 Implementation 82
3.5 Timing Characteristics 84
3.6 Usage Examples 85
3.6.1 Asynchronous FIFO pointers 85
3.6.2 Clock domain crossing counter 85
3.6.3 Position encoding 85
3.7 Design Notes 86
3.7.1 Common mistakes 86
3.8 Related Modules 88
3.9 Testing 89
3.9.1 Exhaustive Testing 89
3.9.2 Property-Based Verification 89

3.9.3 Random Testing....90
3.9.4 Test files....90
3.10 References....91
3.11 Navigation....92
4 Johnson-to-binary converter (johnson2bin.sv)....93
4.1 Overview....93
4.2 Parameters....94
4.3 Ports....95
4.4 Functional Description....96
4.4.1 Block diagram....96
4.4.2 The Johnson sequence....96
4.4.3 The algorithm....97
4.4.4 Three-part binary construction....98
4.4.5 Width calculations....98
4.4.6 Why this algorithm works....98
4.4.7 Worked conversions....98
4.4.8 Comparison with standard gray2bin....99
4.5 Timing Characteristics....100
4.6 Usage Examples....101
4.7 Design Notes....102
4.7.1 The clock and reset ports are unused....102
4.7.2 When to use Johnson vs. Gray....102
4.7.3 Resource planning....102
4.7.4 Verification challenges....102

4.7.5 Invalid Johnson sequences....102
4.7.6 Debug visibility....103
4.7.7 Common issues....103
4.7.8 Advanced topics....103
4.8 Related Modules....104
4.9 Testing....105
4.10 References....106
4.11 Navigation....107
5 Binary-Gray counter (counter_bingray.sv)....108
5.1 Overview....108
5.2 Parameters....109
5.3 Ports....110
5.4 Functional Description....111
5.4.1 Block diagram....111
5.4.2 Gray code sequence (4-bit)....112
5.4.3 Internal logic....112
5.5 Timing Characteristics....114
5.6 Waveforms....115
5.6.1 Scenario 1: Binary vs Gray Code Comparison....115
5.6.2 Scenario 2: Single-Bit Transitions (CDC Safety) KEY FEATURE....115
5.6.3 Scenario 3: Lookahead Signal (counter_bin_next)....115
5.6.4 Scenario 4: Enable and Reset Control....116
5.7 Usage Examples....118
5.8 Design Notes....119

5.8.1 Common mistakes....119
5.9 Related Modules....120
5.10 Testing....121
5.10.1 Test scenarios....121
5.10.2 Coverage....121
5.10.3 Assertions....121
5.10.4 Test files....122
5.11 References....123
5.12 Navigation....124
6 Johnson counter (counter_johnson.sv)....125
6.1 Overview....125
6.2 Parameters....126
6.3 Ports....127
6.4 Functional Description....128
6.4.1 Block diagram....128
6.4.2 State sequences....129
6.4.3 NOT self-starting – reset is mandatory....130
6.4.4 Symmetry....130
6.4.5 Single-bit transitions – but Johnson is not Gray....130
6.5 Timing Characteristics....132
6.6 Waveforms....133
6.6.1 Scenario 1: Walking Ones and Walking Zeros Pattern....133
6.6.2 Scenario 2: Single-Bit Transitions (CDC Safety) KEY FEATURE....133
6.6.3 Scenario 3: Enable Control....133

6.6.4 Scenario 4: Reset Behavior....134
6.7 Usage Examples....135
6.7.1 Example 1: 4-phase clock generator....135
6.7.2 Example 2: LED chaser....135
6.7.3 Example 3: state machine controller....135
6.7.4 Example 4: quadrature signal generation....136
6.7.5 Multi-phase clock for pipeline stages....136
6.7.6 Sequential control (memory refresh)....136
6.7.7 Pattern generation for BIST....137
6.8 Design Notes....138
6.9 Related Modules....139
6.10 Testing....140
6.10.1 Functional tests....140
6.10.2 Coverage....140
6.10.3 Assertions....140
6.10.4 Test files....141
6.11 References....142
6.12 Navigation....143
7 Asynchronous FIFO (fifo_async.sv)....144
7.1 Overview....144
7.2 Parameters....145
7.3 Ports....146
7.4 Functional Description....147
7.4.1 Block diagram....147

7.4.2 Clock domain crossing strategy....147
7.4.3 Core components....147
7.4.4 Gray code pointer system....147
7.4.5 Clock domain crossing....148
7.4.6 Memory organization....148
7.4.7 Full/empty detection....149
7.4.8 The power-of-2 requirement....149
7.5 Timing Characteristics....151
7.6 Waveforms....152
7.7 Usage Examples....153
7.8 Design Notes....154
7.8.1 Depth sizing....154
7.8.2 Clock relationship....154
7.8.3 Almost full/empty settings....154
7.8.4 Error checking....154
7.9 Related Modules....155
7.10 Testing....156
7.11 References....157
7.12 Navigation....158
8 GAXI asynchronous FIFO (gaxi_fifo_async.sv)....159
8.1 Overview....159
8.2 Parameters....160
8.3 Ports....161
8.4 Functional Description....162

8.4.1 Why Gray code?....162
8.4.2 Dependencies....162
8.4.3 Block diagram....162
8.5 Timing Characteristics....164
8.6 Usage Examples....165
8.6.1 Example 1: basic CDC FIFO....165
8.6.2 Example 2: high-speed to low-speed CDC....165
8.7 Design Notes....166
8.7.1 Depth sizing for clock ratio....166
8.7.2 Reset synchronization....166
8.7.3 Metastability protection....166
8.7.4 Error checking....167
8.7.5 Common mistakes....167
8.8 Related Modules....168
8.9 Testing....169
8.10 References....170
8.11 Navigation....171
9 gaxi_skid_buffer_async....172
9.1 Overview....173
9.1.1 Key Features....173
9.2 Parameters....174
9.3 Ports....175
9.4 Functional Description....176
9.4.1 Block diagram....176

9.4.2 Internal Components....176
9.5 Timing Characteristics....178
9.5.1 Latency Components....178
9.5.2 Throughput....178
9.6 Usage Examples....179
9.6.1 Example 1: Basic Async Skid Buffer....179
9.6.2 Example 2: Pipeline Stage with CDC....179
9.7 Design Notes....181
9.7.1 When to Use This Module....181
9.7.2 Depth Selection....181
9.7.3 Reset Synchronization....181
9.7.4 Resource Utilization....181
9.7.5 Common Issues....182
9.8 Related Modules....183
9.9 Testing....184
9.10 References....185
9.11 Navigation....186
10 glitch_free_n_dff_arn....187
10.1 Overview....187
10.2 Parameters....188
10.3 Ports....189
10.4 Functional Description....190
10.4.1 The Metastability Problem....190
10.4.2 Multi-Stage Synchronizer Solution....190

10.4.3 Packed Array Structure....191
10.4.4 Shift Register Behavior....191
10.4.5 Reset Behavior....191
10.4.6 MTBF (Mean Time Between Failures) Analysis....192
10.5 Timing Characteristics....194
10.5.1 Synchronizer Latency....194
10.5.2 Performance Impact....194
10.6 Usage Examples....195
10.6.1 Reset Synchronizer Pattern....195
10.6.2 Enable Signal Synchronization....195
10.6.3 Status Flag Collection....195
10.7 Design Notes....196
10.7.1 Appropriate Use Cases....196
10.7.2 Inappropriate Use Cases....196
10.7.3 Multi-Bit Considerations....196
10.7.4 Synthesis Considerations....196
10.7.5 Debug Features....197
10.8 Related Modules....198
10.9 Testing....199
10.9.1 Verification Strategies....199
10.9.2 Test Suite....199
10.10 References....200
10.11 Navigation....201
11 reset_sync....202

11.1 Overview....202
11.2 Parameters....203
11.3 Ports....205
11.3.1 Inputs....205
11.3.2 Outputs....205
11.4 Functional Description....206
11.4.1 Block diagram....206
11.4.2 The Reset Problem....206
11.4.3 Asynchronous Assert, Synchronous Deassert....206
11.4.4 Synchronization Register Chain....206
11.4.5 Reset Logic....207
11.4.6 Output Assignment....207
11.5 Timing Characteristics....208
11.5.1 Reset Assertion (Asynchronous)....208
11.5.2 Reset Deassertion (Synchronous, N=3)....208
11.5.3 State Sequence for N=3....208
11.6 Usage Examples....210
11.6.1 Repository Convention (read first)....210
11.6.2 Choosing N Value....210
11.6.3 Global Reset Distribution....211
11.6.4 Clock Domain Crossing....211
11.6.5 Power-On Reset....212
11.6.6 Reset Controller Integration....212
11.7 Design Notes....213

11.7.1 1. Why the chain looks inverted....213
11.7.2 2. Metastability Resolution....213
11.7.3 3. Initialization....213
11.7.4 4. Vendor attributes....213
11.7.5 5. Parameterization Benefits....213
11.7.6 Reset Tree Planning....214
11.7.7 Clock Quality Requirements....214
11.7.8 Reset Source Management....214
11.7.9 Incorrect Sensitivity List....214
11.7.10 Wrong Reset Polarity....214
11.7.11 Clock Domain Issues....214
11.8 Related Modules....215
11.9 Testing....216
11.9.1 Test Coverage....216
11.9.2 Test Scenarios....216
11.9.3 Testbench Scenarios....216
11.9.4 Assertions....216
11.9.5 Coverage Points....216
11.9.6 Running Tests....216
11.9.7 Test Results....217
11.10 References....218
11.11 Navigation....219
12 clock_pulse....220
12.1 Overview....220

12.2 Parameters..... 221
12.3 Ports..... 222
12.4 Functional Description..... 223
12.4.1 Block diagram..... 223
12.4.2 Internal Counter..... 223
12.4.3 Core Logic..... 224
12.4.4 Operation Principles..... 224
12.5 Related Modules..... 225
12.6 Timing Characteristics..... 226
12.6.1 Timing Characteristics..... 226
12.6.2 Basic Operation (WIDTH=4)..... 226
12.6.3 Reset Behavior..... 226
12.6.4 Different WIDTH Values..... 226
12.7 Usage Examples..... 228
12.8 Design Notes..... 229
12.8.1 Resource Utilization..... 229
12.8.2 Synthesis Considerations..... 229
12.8.3 Design Considerations..... 230
12.8.4 Common Applications..... 230
12.9 Testing..... 231
12.9.1 Comprehensive Test Bench..... 231
12.9.2 Coverage Model..... 234
12.9.3 Formal Properties..... 235
12.10 References..... 237

12.11 Navigation....238
13 apb4_slave_cdc....239
13.1 Overview....240
13.1.1 Key Features....240
13.2 Parameters....241
13.3 Ports....242
13.4 Functional Description....244
13.4.1 Block diagram....244
13.4.2 Clock Domains....244
13.4.3 CDC Structure....245
13.4.4 Reset Behavior....245
13.4.5 Why Not the Previous 2-Phase Handshake....246
13.5 Design Notes....247
13.6 Timing Characteristics....248
13.6.1 Maximum Clock Ratio....248
13.6.2 Timing Constraints....248
13.7 Waveforms....249
13.7.1 Scenario 1: Write Transaction with CDC....249
13.7.2 Scenario 2: Read Transaction with CDC....249
13.7.3 Scenario 3: Back-to-Back Writes with CDC....250
13.7.4 Scenario 4: Back-to-Back Reads with CDC....250
13.7.5 Scenario 5: Write-to-Read Transition with CDC....250
13.7.6 Scenario 6: Read-to-Write Transition with CDC....251
13.7.7 Scenario 7: Error Response with CDC....251

13.8 Usage Examples....252
13.9 Related Modules....253
13.10 Testing....254
13.11 References....255
13.12 Navigation....256
14 apb4_slave_cdc_cg....257
14.1 Overview....258
14.2 Parameters....259
14.2.1 Derived Parameters (do not override)....259
14.3 Ports....260
14.3.1 Clock-gating status outputs....261
14.4 Functional Description....263
14.4.1 Block diagram....263
14.5 Timing Characteristics....265
14.6 Usage Examples....266
14.7 Design Notes....267
14.8 Related Modules....268
14.9 Testing....269
14.10 References....270
14.11 Navigation....271
15 APB5 Slave CDC....272
15.1 Overview....273
15.1.1 Key Features....273
15.2 Parameters....274

15.2.1 Derived Parameters (do not override)....275
15.3 Ports....276
15.3.1 Clock and Reset....276
15.3.2 APB5 Slave Interface....276
15.3.3 Backend Interface....276
15.4 Functional Description....277
15.4.1 Architecture....277
15.4.2 CDC FIFO Pointer Encoding....278
15.4.3 Reset Synchronization....279
15.5 Timing Characteristics....281
15.5.1 Latency....281
15.5.2 Timing Considerations....281
15.6 Usage Examples....282
15.7 Design Notes....283
15.7.1 FIFO Depth Sizing....283
15.8 Related Modules....284
15.9 Testing....285
15.10 References....286
15.11 Navigation....287
16 APB5 Slave CDC (Clock-Gated)....288
16.1 Overview....289
16.1.1 Key Features....289
16.2 Parameters....290
16.2.1 Derived Parameters (do not override)....291

16.3 Ports....292
16.3.1 Clock and Reset....292
16.3.2 Clock Gating Configuration....292
16.3.3 APB5 Slave Interface....292
16.3.4 Backend Interface....292
16.3.5 Status Outputs....292
16.4 Functional Description....294
16.4.1 Architecture....294
16.4.2 Wake-up Logic....294
16.5 Timing Characteristics....295
16.6 Usage Examples....296
16.7 Design Notes....297
16.7.1 Power and Latency Trade-offs....297
16.7.2 Reset Considerations....298
16.8 Related Modules....299
16.9 Testing....300
16.10 References....301
16.11 Navigation....302

List of Tables

Table 1: CDC building blocks at a glance....23
Table 2: CDC technique quick reference by scenario....24
Table 3: How pointer encoding determines reset behavior....26
Table 4: 2-phase handshake reset domains per flop....27
Table 5: Options for recovering a design using 2-phase across independent resets 31
Table 6: cdc_synchronizer parameters....34
Table 7: cdc_open_loop parameters....35
Table 8: cdc_open_loop ports....36
Table 9: cdc_2_phase_handshake parameters....37
Table 10: Shared handshake interface ports....38
Table 11: Consequences of swapping handshake variants....39
Table 12: cdc_4_phase_handshake parameters....40
Table 13: Async FIFO pointer encodings....42
Table 14: 3-bit Gray vs the equivalent 4-bit Johnson counter, full cycle. “flips bit#” is a bit POSITION; “Ham. dist” is the number of bits that changed, which is 1 everywhere including the wrap....43
Table 15: Pointer width by encoding – Gray is logarithmic, Johnson is linear....43
Table 16: Worked depth-sizing example....45
Table 17: Nominal storage overhead of Gray depth rounding by data width....45
Table 18: 512-bit walkthrough: depth after applying the encoding constraint....46
Table 19: 512-bit walkthrough: nominal storage delta....46
Table 20: 512-bit walkthrough: pointer flop cost....46
Table 21: 512-bit walkthrough: whether the saving is real, by memory style....47
Table 22: Depth-36 case: storage delta when Gray must round 36 up to 64....48
Table 23: Depth-36 case: pointer flop cost....48
Table 24: Depth-36 case: ASIC memory styles, all of which take the word count as an argument....49
Table 25: CDC module reference....54
Table 26: bin2gray parameters....61
Table 27: bin2gray ports....62
Table 28: bin2gray resource usage by width (typical FPGA)....67
Table 29: gray2bin parameters....78
Table 30: gray2bin ports....79
Table 31: gray2bin delay by width....84
Table 32: johnson2bin parameters....94

Table 33: johnson2bin ports....95
Table 34: counter_bingray parameters....109
Table 35: counter_bingray ports....110
Table 36: counter_johnson parameters....126
Table 37: counter_johnson ports....127
Table 38: fifo_async parameters....145
Table 39: fifo_async ports....146
Table 40: gaxi_fifo_async parameters....160
Table 41: gaxi_fifo_async ports....161
Table 42: gaxi_fifo_async latency by clock ratio....164
Table 43: Synchronizer stage guidance....166
Table 44: reset_sync parameters....202
Table 45: reset_sync state sequence for N=3 at default polarities....207

List of Waveforms

Waveform 1.1: 2-Phase Normal Transfer....27
Waveform 1.2: 2-Phase Hazard - Destination Reset Alone....28
Waveform 1.3: 4-Phase Normal Transfer....29
Waveform 1.4: 4-Phase One-Sided Reset Is Safe....29

1 Clock Domain Crossing (CDC)

RTL: `rtl/cdc/` **Filelists:** `rtl/amba/filelists/cdc_*.f` **Status:** Production Ready **for common-clock, common-reset verification.** Reset behavior across independent domains is argued from the RTL and, for the 2-phase hazard, confirmed on silicon – but it is **not** covered by the formal proofs, which drive both domains from one clock and one reset. See Verification status before relying on that badge.

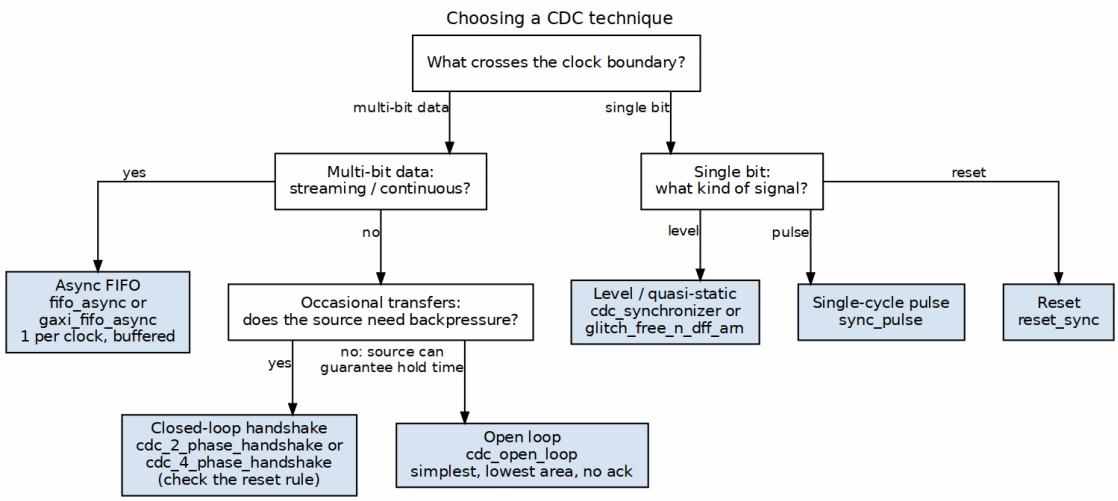
This page is the single reference for clock domain crossing in this repository: every CDC building block, how to choose between them, and the reset behavior that distinguishes them. That last part is the property most likely to bite you, and the least likely to show up in simulation.

CDC building blocks at a glance

Module	Category	One-line
`cdc_synchronizer`	Multi-flop sync	N-stage synchronizer for quasi-static signals
`cdc_open_loop`	Open loop	Source holds data + valid, no acknowledge
`cdc_2_phase_handshake`	Closed loop	Toggle (NRZ) valid/ready handshake, faster
`cdc_4_phase_handshake`	Closed loop	Level (RZ) valid/ready handshake, classic
`fifo_async` / `gaxi_fifo_async`	Pointer sync	Streaming data, buffered

1.1 Choosing a technique

Start with the shape of the traffic, not the protocol:



Choosing a CDC technique

Then check the reset rule before committing to a handshake. It eliminates one of the two handshake variants outright in a common situation, and it is not visible from the port list.

1.1.1 Quick reference

CDC technique quick reference by scenario

Scenario	Module	Why
Config register update (CPU -> peripheral)	`cdc_open_loop`	Infrequent, no backpressure needed
Interrupt signal (1 bit, pulse)	`sync_pulse`	Single-cycle event
Status register read (slow -> fast)	`cdc_synchronizer`	Quasi-static level
APB slave in different clock domain	`apb4_slave_cdc`	Full protocol CDC
Monitor packet crossing	`cdc_4_phase_handshake`	Occasional, needs flow control
DMA data stream	`fifo_async`	High throughput,

Scenario	Module	Why
		continuous
AXI channel crossing	`gaxi_skid_buffer_async`	Pipelined, GAXI protocol
Reset distribution	`reset_sync`	Async assert, sync deassert

1.2 Reset Considerations

> **DESIGN RULE:** If the two clock domains can be reset **independently**, do not use `cdc_2_phase_handshake`. Use `cdc_4_phase_handshake` or an async FIFO. A toggle handshake stores transfer state as **parity**, and parity cannot survive a one-sided reset.

"Independently" covers more cases than people expect: a soft reset that clears a datapath but not a register block, a per-block reset, separate power domains, or a reset synchronizer whose deassertion is gated differently on each side. If any of those can reach one domain alone, the rule applies.

1.2.1 Why encoding decides reset behavior

How pointer encoding determines reset behavior

	2-phase	4-phase	Async FIFO
Transfer state	Parity (toggle) – **relative**	Level – **absolute**	Pointer position – **absolute**
Idle has a value?	No	Yes (`req=0`)	Yes (pointers equal)
One-sided reset from **idle**	**Fabricates a transfer**	Nothing happens	Read-side reset: **replays consumed entries** (benign only if pointers never moved). **Write-side reset: fabricates occupancy**
One-sided reset **mid-transfer**	Duplicate or lost transfer	Duplicate or dropped transfer	Entry may be re-read or dropped

The 2-phase protocol signals "a transfer happened" with a *transition*. The receiver has no absolute reference – it only knows whether the synchronized toggle differs from its previous sample:

```
assign w_req_event = w_req_sync ^ r_req_sync_d;   // cdc_2_phase_handshake.sv:261
```

That XOR is the entire protocol state. Each side resets its own flops:

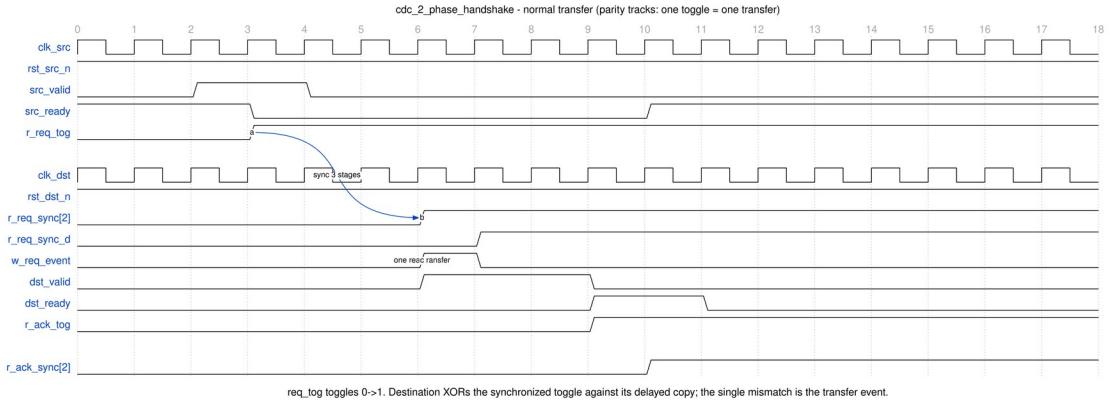
2-phase handshake reset domains per flop

Flop	Domain	Reset by	After reset
`r_req_tog`	source	`rst_src_n`	`1'b0` (:190)
`r_ack_sync`, `r_ack_sync_d`	source	`rst_src_n`	`1'b0` (:173-174)
`r_ack_tog`	destination	`rst_dst_n`	`1'b0`
`r_req_sync`, `r_req_sync_d`	destination	`rst_dst_n`	`1'b0` (:252-253)

Reset **both** domains together and everything lands at 0 – parity agrees, no event. Reset **one** and the two ends land at opposite parity, and the XOR reports an edge no source transfer produced.

Because parity is relative, there is no toggle value that means “idle”. Whatever `r_req_tog` happens to hold, a freshly cleared destination disagrees with it half the time. In 4-phase, `req = 0` *is* idle, absolutely: a reset destination observing `req = 0` correctly concludes nothing is pending.

1.2.1.1 Waveform 1.1: 2-Phase Normal Transfer

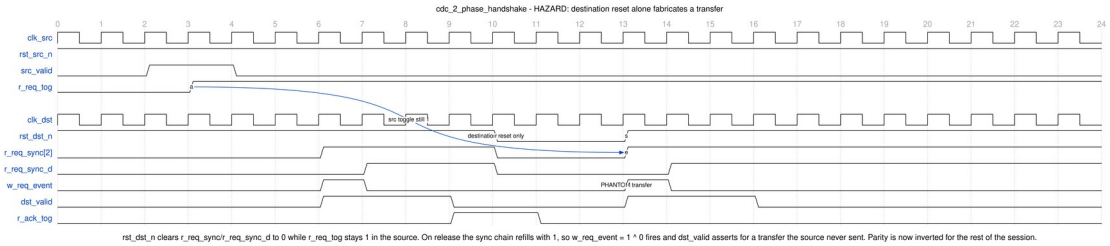


2-phase normal transfer

Source: cdc2_normal_transfer.json

One `src_valid` toggles `r_req_tog` 0 -> 1. Three synchronizer stages later the destination sees the new value, `w_req_event` pulses for one cycle, `dst_valid` asserts. Parity now agrees at 1 on both ends.

1.2.1.2 Waveform 1.2: 2-Phase Hazard - Destination Reset Alone



2-phase asymmetric reset hazard

Source: cdc2_asymmetric_reset_hazard.json

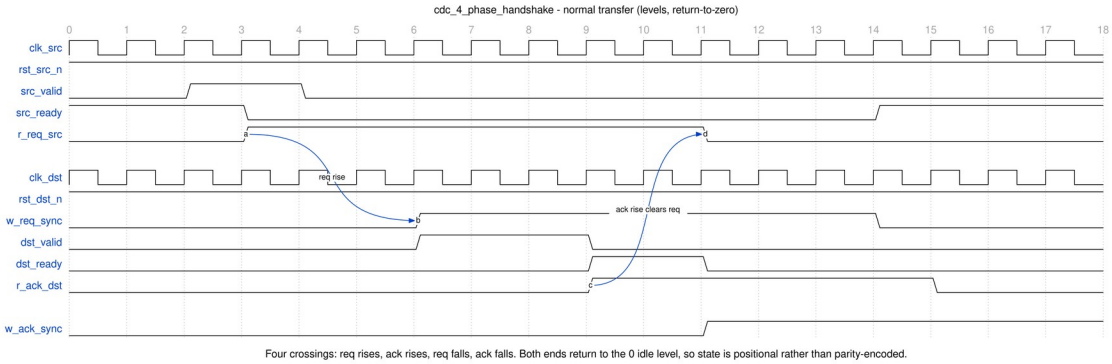
`rst_dst_n` asserts while `rst_src_n` stays high:

1. The destination clears `r_req_sync` and `r_req_sync_d` to 0.
2. The source is untouched – `r_req_tog` is still 1.
3. On release the sync chain refills with 1 while `r_req_sync_d` is the freshly cleared 0.
4. `w_req_event = 1 ^ 0 = 1` – a **phantom transfer**. `dst_valid` asserts with stale `r_src_data_hold`, for a transfer the source never sent.

Resetting the source alone abandons an in-flight transfer too: the source FSM returns to `S_IDLE` from `S_WAIT_ACK` (`:188-190`). It does **not** clear `r_ack_tog` – that flop lives in the destination domain and is reset only by `rst_dst_n`, as the flop table above records. The spurious-ack mechanism belongs to the destination-reset case, where `r_ack_tog` clears while the source is still waiting. Either way a one-sided reset can **lose** a transfer as well as fabricate one.

Scope of the damage. The handshake itself re-synchronizes: once `r_req_sync_d` catches up to `r_req_tog`, parity agrees again. What is permanent is the **extra item already delivered downstream**. A consumer that samples the latest value self-heals on the next real transfer; a consumer that counts transfers, pushes into a FIFO, or pairs responses to requests positionally is corrupted for the rest of the session.

1.2.1.3 Waveform 1.3: 4-Phase Normal Transfer

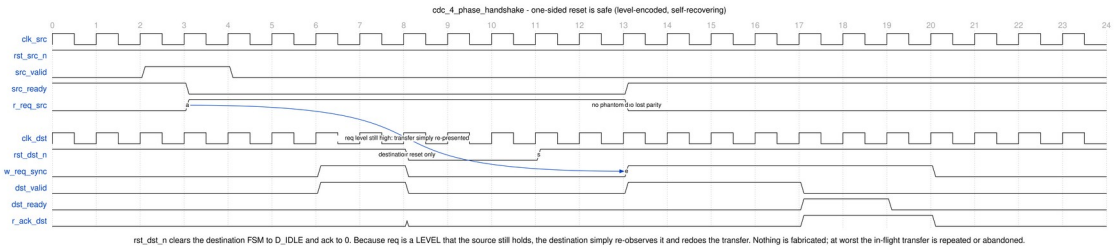


4-phase normal transfer

Source: cdc4_normal_transfer.json

Four crossings: `r_req_src` rises, `r_ack_dst` rises, `r_req_src` falls, `r_ack_dst` falls. Both ends finish at the 0 idle level.

1.2.1.4 Waveform 1.4: 4-Phase One-Sided Reset Is Safe



4-phase one-sided reset is safe

Source: cdc4_asymmetric_reset_safe.json

`rst_dst_n` asserts mid-transfer while the source runs:

1. The destination FSM returns to D_IDLE and `r_ack_dst` clears to 0.
2. The source is unaffected, still holding `r_req_src = 1` with data stable.
3. On release the destination re-observes `w_req_sync = 1`, treats it as a fresh request, and redoes the transfer.
4. The source sees the ack and completes normally.

Worst case is one transfer **repeated or dropped**, and only for a transfer genuinely in flight. If the link is idle when the reset lands, the destination sees `req = 0`, reads it as “nothing pending”, and does nothing.

Caveat – **duplicate delivery.** Step 3 re-delivers a transfer the destination may already have accepted. If the consumer is not idempotent (a FIFO push, a counter increment, a command queue), hold it in reset alongside the destination FSM or drain it afterwards. Level encoding guarantees you never get a transfer the source did not send; it does not guarantee exactly-once delivery.

1.2.2 Async FIFO reset behavior

Gray/Johnson pointers are absolute positions, and each domain resets its own pointer **and its crossed copy of the remote pointer** from the local reset:

```
rd_ptr_gray_cross_inst -> .rst_n (axi_wr_aresetn)   // synchronizer,
local reset
wr_ptr_gray_cross_inst -> .rst_n (axi_rd_aresetn)   // synchronizer,
local reset
```

Only the synchronizers take a reset. The decode stage beside each one has none to take: with the default `USE_JOHNSON=0` it is a `gray2bin`, whose entire port list is `.gray` and `.binary`. (With `USE_JOHNSON=1` it is a `johnson2bin`, which does carry `clk`/`rst_n` ports – but leaves them unused.) The local-reset property below rests on the synchronizers alone; the decode is combinational and simply follows whatever the synchronizer holds.

There is no parity state to invert, so the FIFO cannot fabricate the way a 2-phase handshake does – but "one-sided reset is safe" is too strong, and the asymmetry matters before you rely on it.

A read-side reset is benign only when the pointers have never moved. `rd_ptr` clears to 0, and the read domain's copy of the write pointer (`wr_ptr_gray_cross_inst`) is cleared with it – but that copy is a LIVE synchronizer, and within `N_FLOP_CROSS` rd clocks it re-samples the write side's untouched pointer. If the link has seen traffic – even fully consumed traffic, both pointers resting at some K – the settled state is `rd_ptr = 0` against `wr_ptr_sync = K`: the read side believes the FIFO holds K entries, and the K words it already consumed are **replayed**. Empty-flag "idle" is not enough; the pointers must be at zero.

A write-side reset from idle is not. The read domain's copy of the write pointer (`wr_ptr_gray_cross_inst` in `fifo_async.sv`) is reset by **`rd_rst_n`, not `wr_rst_n`** – a write-side reset does not clear it. So with both pointers idle at some value K:

```
wr_rst_n pulsed:   wr_ptr      K -> 0
                   rd_ptr      K     (untouched)
                   wr_ptr_sync K -> 0 over N_FLOP_CROSS rd clocks

read side sees:    rd_ptr (K) != wr_ptr_sync (0)  =>  NOT EMPTY
                   count = 0 - K  mod 2^(AW+1)    =>  27 at K=5, AW=4
```

The read side now believes the FIFO holds more entries than it has slots, and pops words that were never written. That is fabrication, and it is worse than losing the writes.

The rule is the same either way – **quiesce the link before resetting one side** – but the reason is not symmetric, and resetting only the write domain is the dangerous direction.

This covers reset ASSERTION, not reset RELEASE. Each domain's reset must still be released synchronously *within that domain* – which is what `reset_sync` (async assert, sync deassert) is for. A raw asynchronous release can violate recovery/removal timing on the pointer flops, and if the two domains release at sufficiently different times the FIFO can begin operating while one side's synchronized copy of the remote pointer is still catching up. That window is benign for an idle FIFO (it reads empty) but not if traffic starts immediately. Hold traffic off until both domains are out of reset, or gate the producer on a released-and-settled indication.

1.2.3 Silicon evidence

This is not theoretical. On the Nexys A7 DDR2 characterization board, an APB CDC using the 2-phase handshake had its core-side reset pulsed by `CTRL.soft_reset` while the APB side stayed up. One phantom transfer entered a slave FSM that pairs responses to requests positionally (`apb4_slave.sv` returns whatever response sits at the head of its skid buffer), and **every register read from then on returned the previous register's value** – lagged by ~3 transactions, permanently, until reprogramming. Writes were unaffected, so it presented as a readback bug rather than a CDC bug. I've debugged enough of these to know that disguise is what makes them expensive.

Replacing that handshake with `gaxi_fifo_async` fixed it: reads became stable across `soft_reset`, verified on silicon.

1.2.4 Safe usage checklist for 2-phase

This is a hard stop, not a scorecard. Every condition must hold. Failing any one of them disqualifies the module – there is no partial credit and no mitigation short of changing the crossing or the reset topology.

Use `cdc_2_phase_handshake` only when **all** hold:

- ☐ `rst_src_n` and `rst_dst_n` assert and release together, **or** both derive from one reset no subsystem can pulse independently
- ☐ No soft reset, per-block reset, or power-domain reset touches one side alone
- ☐ The consumer correlates responses to requests by tag, not by position

If any box is unchecked, choose a level-encoded or pointer-encoded crossing.

1.2.5 Recovering an existing design

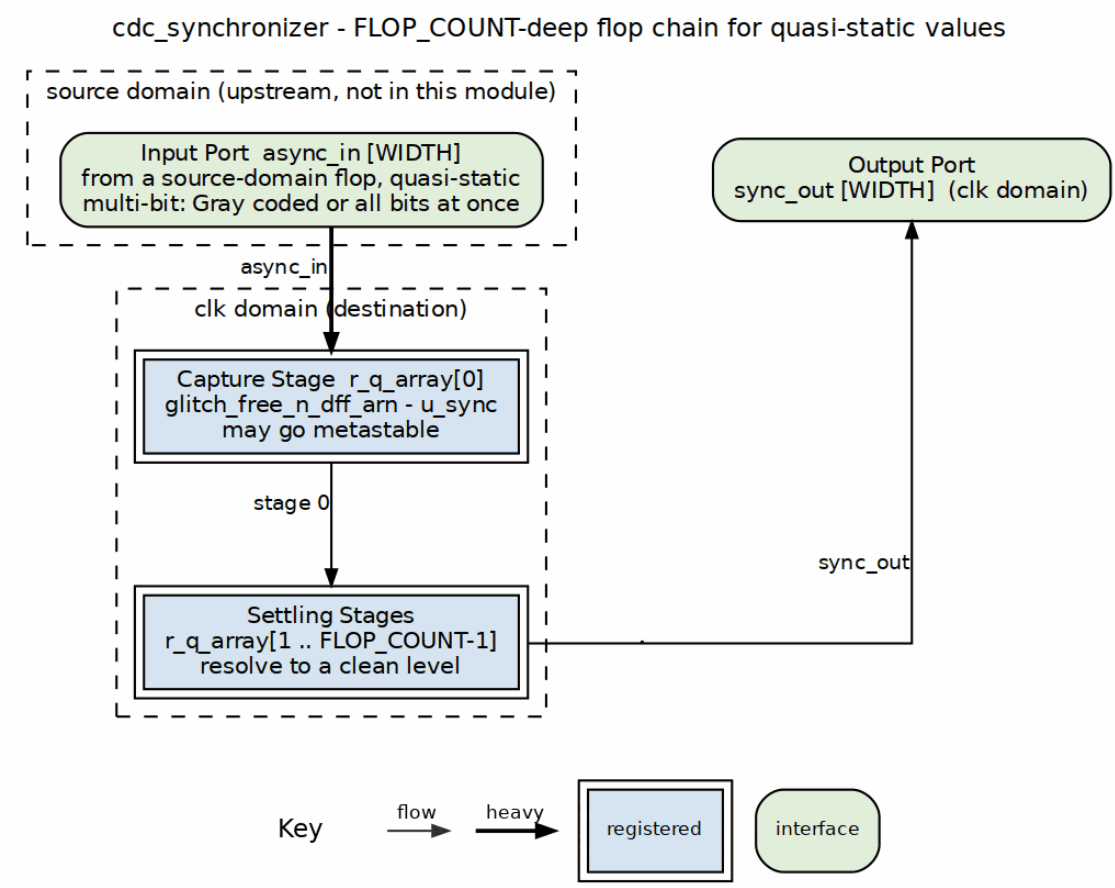
Options for recovering a design using 2-phase across independent resets

Option	Effort	Notes
Move both domains onto a common reset	Low	Best when the split was accidental
Swap to	Low	Pin-compatible; level

Option	Effort	Notes
`cdc_4_phase_handshake`		encoding self-recovers
Swap to `gaxi_fifo_async`	Medium	Absolute pointers fail less catastrophically than a stuck toggle, adds buffering – the quiesce-before-reset rule above still applies

1.3 cdc_synchronizer

Multi-flop synchronizer for **quasi-static** signals. Not a handshake – there is no flow control and no guarantee the destination sees every value.



cdc_synchronizer block diagram

The module owns only the destination clock. `async_in` arrives from a register upstream in the source domain and makes the one crossing into the first flop of a `glitch_free_n_dff_arn` chain, which may go metastable. The remaining `FLOP_COUNT - 1` stages let it resolve, and `sync_out` leaves the last stage as a clean level in the `clk` domain.

```
module cdc_synchronizer #(
    parameter int WIDTH      = 1,   // Bus width to synchronize
    parameter int FLOP_COUNT = 3    // Number of synchronizer stages
(2-5)
) (
    input  logic             clk,       // Destination clock
    input  logic             rst_n,     // Asynchronous active-low
reset
    input  logic [WIDTH-1:0] async_in,  // Asynchronous input from
```

```
source domain
    output logic [WIDTH-1:0] sync_out   // Synchronized output in
destination domain
);
```

cdc_synchronizer parameters

Parameter	Default	Description
`WIDTH`	1	Number of bits to synchronize (1+)
`FLOP_COUNT`	3	Synchronizer chain depth (2-5)

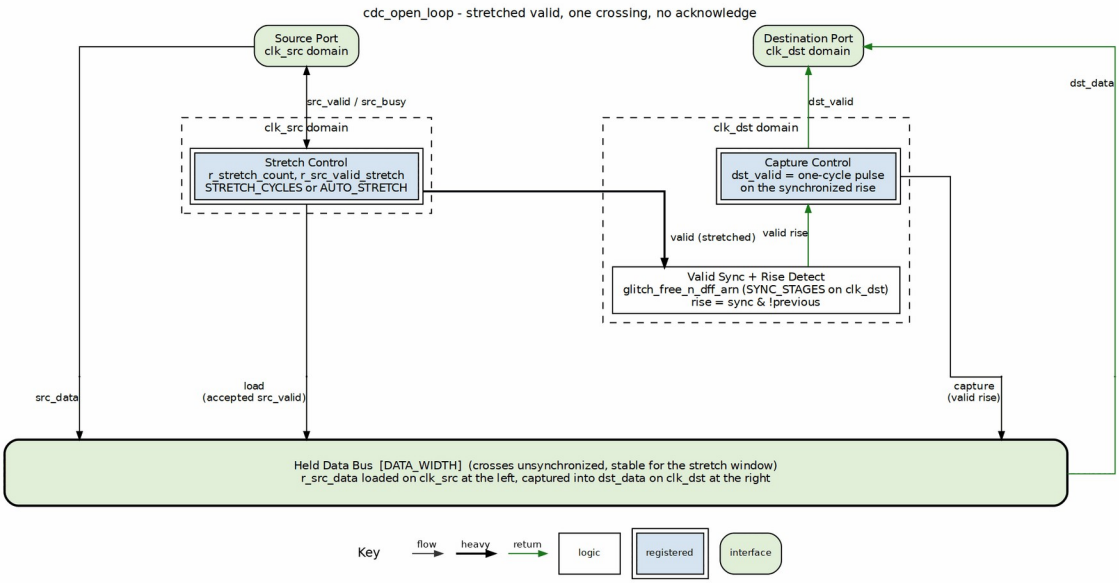
Usage rules

- Input must be **quasi-static**: it changes rarely enough, relative to the destination clock, that missing a transition is acceptable for the application. This is a statistical property, not a hard timing check the synchronizer enforces – it will sample whatever is present. As a sizing rule of thumb, allow at least `FLOP_COUNT + 2` destination clocks between transitions; change faster than that and the destination can miss transitions entirely or sample mid-flight. (`FLOP_COUNT + 2` is the figure `glitch_free_n_dff_arn`'s own header states, and `cdc_synchronizer` is a thin wrapper over it. This page previously said `+ 1`; neither is enforced by logic, but the two should not disagree, and the safer bound wins.)
- For multi-bit buses, all bits must change simultaneously **or** be Gray coded.
- `FLOP_COUNT=2` for relaxed MTBF, `3` for production (default)
- Do **not** use for pulses (`sync_pulse`) or streaming data (`fifo_async`)

Gray-coded input must be REGISTERED in the source domain before it crosses. The single-bit-change guarantee is a property of a registered counter sequence, not of the encoding in the abstract. Feeding this module a Gray value straight out of combinational logic re-introduces exactly the hazard Gray coding exists to remove: the bits arrive with different delays, and the synchronizer can latch a transient that was never a real state. Register the Gray value, then cross it. The same applies to a Gray value that can *jump* rather than increment by one – multiple bits change and no encoding saves you.

1.4 cdc_open_loop

Source holds `valid` + data stable for a stretch window long enough that the destination cannot miss it. No acknowledge, so the lowest area and latency of the data-carrying crossings – at the cost of no backpressure.



cdc_open_loop block diagram

`src_data` drops down the left into the held data register, which loads on an accepted `src_valid`. The stretch control then holds the valid high for the stretch window, and that stretched valid is the only synchronized crossing. In the destination domain a rise detector turns it into a one-cycle `dst_valid` and captures the held bus, whose value climbs the right edge as `dst_data`. Nothing crosses back: `src_busy` is the source's only protection.

cdc_open_loop parameters

Parameter	Default	Description
`DATA_WIDTH`	8	Width of the data bus
`STRETCH_CYCLES`	8	Source clocks to hold valid+data stable (`AUTO_STRETCH=0`)
`SYNC_STAGES`	2	Destination synchronizer depth (2-4)
`AUTO_STRETCH`	0	`0` = use `STRETCH_CYCLES`; `1` = compute from clock frequencies

Parameter	Default	Description
SRC_CLK_HZ	25_000_000	Source clock frequency (AUTO_STRETCH=1)
DST_CLK_HZ	100_000_000	Destination clock frequency (AUTO_STRETCH=1)

cdc_open_loop ports

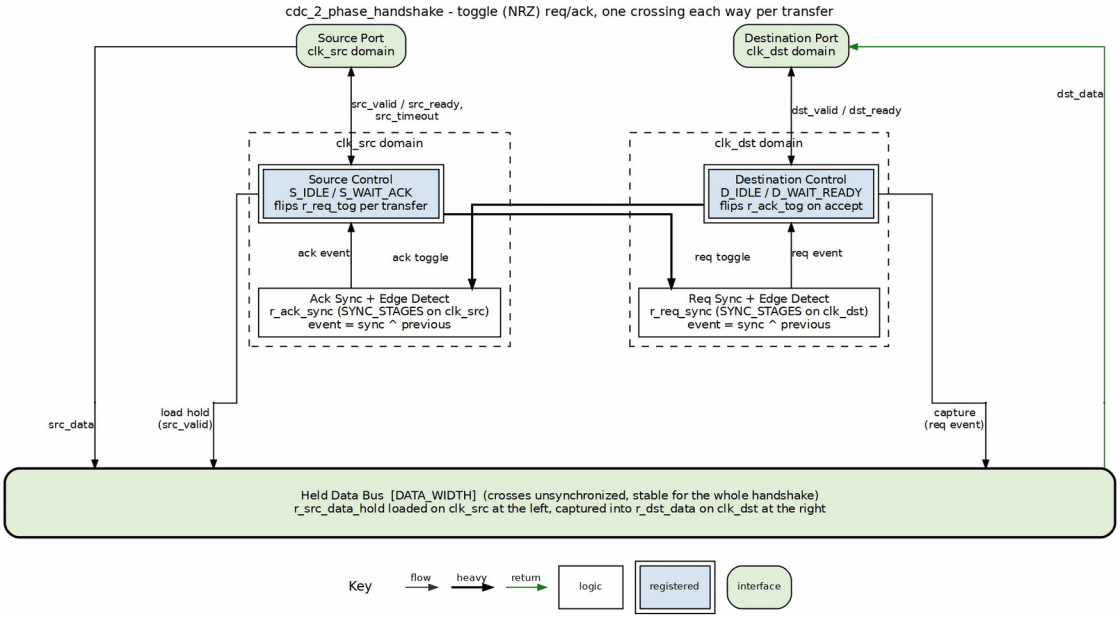
Port	Direction	Width	Description
clk_src / rst_src_n	input	1	Source domain clock / async reset
src_valid	input	1	Single-cycle pulse: data is valid
src_data	input	DATA_WIDTH	Data to transfer
src_busy	output	1	High during stretch countdown
clk_dst / rst_dst_n	input	1	Destination domain clock / async reset
dst_valid	output	1	Single-cycle pulse: data latched
dst_data	output	DATA_WIDTH	Latched data, stable until next dst_valid

The failure mode to watch: sending a new transfer before the previous one is sampled. Without an acknowledge the source cannot know the destination latched the data. Respect src_busy, or space transfers by at least STRETCH_CYCLES source clocks (the busy window; the sizing rule makes it >= SYNC_STAGES + 1 destination clocks).

1.5 cdc_2_phase_handshake

Toggle (NRZ) closed-loop handshake. Two synchronizer crossings per transfer instead of four.

> **Read Reset Considerations before choosing this module.** It is pin-compatible with the 4-phase variant but not behaviorally equivalent under independent resets.



cdc_2_phase_handshake block diagram

Data drops down the left into the held data bus and climbs the right edge as `dst_data`; the bus crosses unsynchronized because the handshake keeps it stable. The only synchronized crossings are the two toggles, which form an X in the center. The source flips `r_req_tog` to start a transfer. The destination's synchronizer and edge detector turn that flip into a request event, which captures the bus and raises `dst_valid`. On `dst_ready` the destination flips `r_ack_tog` back across, and the resulting ack event returns the source to idle.

cdc_2_phase_handshake parameters

Parameter	Default	Description
`DATA_WIDTH`	8	Width of the data bus (1+)
`SYNC_STAGES`	3	Synchronizer depth for req/ack (2 or 3)
`TIMEOUT_CYCLES`	0	0 = disabled; >0 asserts `src_timeout` after stall

1.5.1 Shared handshake interface

Both handshake modules expose an identical port list, which is why swapping them is a one-word edit:

Shared handshake interface ports

Port	Direction	Width	Description
`clk_src` / `rst_src_n`	input	1	Source domain clock / async reset
`src_valid`	input	1	Source indicates data valid
`src_ready`	output	1	Handshake ready (asserted when idle)
`src_data`	input	DATA_WIDTH	Data from source domain
`src_timeout`	output	1	Stall timeout (when `TIMEOUT_CYCLES > 0`)
`clk_dst` / `rst_dst_n`	input	1	Destination domain clock / async reset
`dst_valid`	output	1	Data valid to receiver
`dst_ready`	input	1	Receiver ready
`dst_data`	output	DATA_WIDTH	Data transferred to destination domain

1.5.2 Theory of operation

```
Source FSM
  S_IDLE      src_ready=1. On src_valid: latch data, toggle r_req_tog,
              drop src_ready, go to S_WAIT_ACK.
  S_WAIT_ACK  Wait for ack edge (w_ack_event). On edge: src_ready=1,
S_IDLE.

Destination FSM
  D_IDLE        On req edge (w_req_event): copy data into r_dst_data,
                raise dst_valid, go to D_WAIT_READY.
  D_WAIT_READY  Hold dst_valid. On dst_ready: drop dst_valid, toggle
                r_ack_tog, return to D_IDLE.

Edge detection
  event = current_sync_output ^ previous_sync_output
```

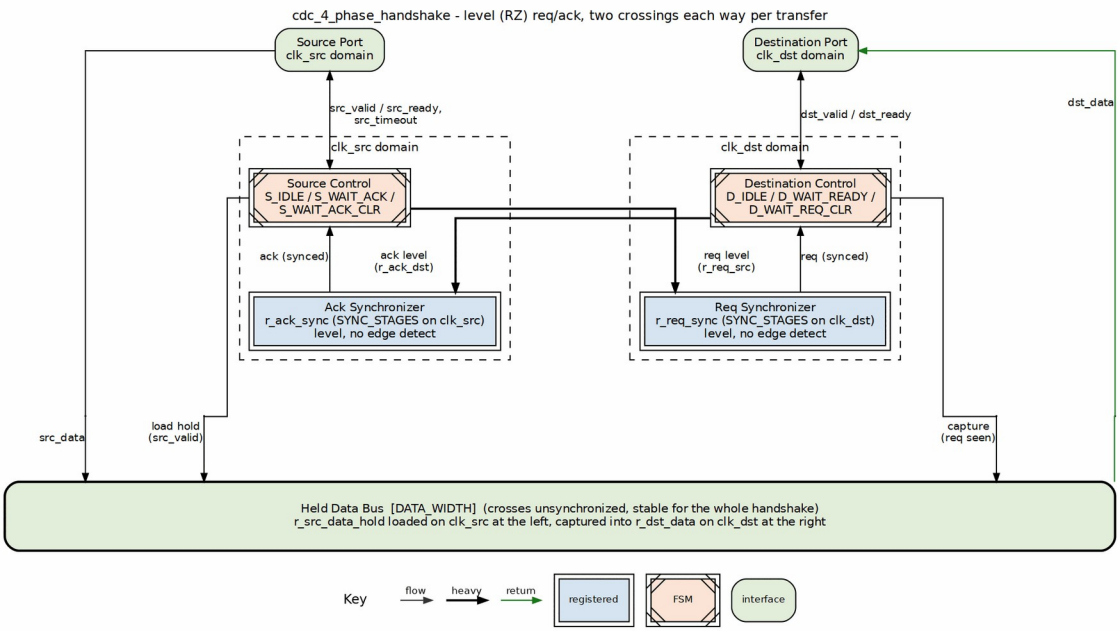
1.5.3 Swapping between variants

Consequences of swapping handshake variants

Direction	Consequence
4-phase -> 2-phase	Gains throughput. **Silently forfeits independent-reset tolerance.** Only safe when both domains share one reset.
2-phase -> 4-phase	Costs roughly two extra synchronizer crossings (~2 x `SYNC_STAGES` clocks, so ~6 at the default of 3), split between the two domains. Always safe.

1.6 cdc_4_phase_handshake

Level (RZ) closed-loop handshake: `req` rises, `ack` rises, `req` falls, `ack` falls. Slower than 2-phase, and tolerant of independent domain resets.



cdc_4_phase_handshake block diagram

Same geometry as the 2-phase diagram: data down the left into the held bus, `dst_data` up the right, and the control crossings in the center. The crossings carry levels, not toggles, so each one crosses twice per transfer: `r_req_src` rises and later falls, and `r_ack_dst` follows it. The plain level synchronizers need no edge detector, and the third state in each FSM waits for the return to zero.

cdc_4_phase_handshake parameters

Parameter	Default	Description
`DATA_WIDTH`	8	Width of the data bus (1 to 1024+)
`SYNC_STAGES`	3	Synchronizer depth for req/ack (2 or 3)
`TIMEOUT_CYCLES`	0	0 = disabled; >0 asserts `src_timeout` after stall

Ports are identical to the shared handshake interface.

1.6.1 Theory of operation

```
Source FSM
  S_IDLE          On src_valid: capture data, r_req_src=1, S_WAIT_ACK.
  S_WAIT_ACK      On synchronized ack: r_req_src=0, S_WAIT_ACK_CLR.
  S_WAIT_ACK_CLR  On ack cleared: src_ready=1, S_IDLE.

Destination FSM
  D_IDLE          On synchronized req: latch data, dst_valid=1,
D_WAIT_READY.
  D_WAIT_READY    On dst_ready: r_ack_dst=1, dst_valid=0,
D_WAIT_REQ_CLR.
  D_WAIT_REQ_CLR  On req cleared: r_ack_dst=0, D_IDLE.
```

Data stability guarantee: data is captured at REQ assertion and held until ACK completion, so it is stable for the entire crossing regardless of clock ratio.

Latency: four synchronizer crossings per transfer against two for 2-phase. At the default `SYNC_STAGES=3` that is roughly 12 clocks of round trip against 6 – counted in the clock domain each crossing lands in, so the wall-clock cost depends on the src:dst frequency ratio. Treat these as order-of-magnitude, not budgetable: measure your own configuration.

1.7 Async FIFO Pointers (Gray and Johnson)

For streaming data, synchronize the read and write **pointers** of a dual-port memory rather than the data. Pointers use an encoding where only one bit changes per increment, so any sample during a transition yields either the old or the new value – never a corrupted intermediate. The memory itself needs no CDC.

```
Write Domain:                                   Read Domain:
  wr_ptr (binary) ---> bin2gray --->             gray2bin ---> compare with
rd_ptr
                         [sync N stages]                       for empty
detection

  gray2bin <--- [sync N stages] <--- bin2gray <--- rd_ptr (binary)
  compare with wr_ptr for full detection
```

1.7.1 Gray vs Johnson encoding

Both `fifo_async` (rtl/cdc) and `gaxi_fifo_async` (rtl/cdc) select the encoding with a `USE_JOHNSON` parameter:

Async FIFO pointer encodings

USE_JOHNSON	Encoding	Pointer width	Converter	Legal DEPTH
0 (default)	Gray	`log2(DEPTH)+1`	`gray2bin` (combinational)	power of 2 only
1	Johnson	`DEPTH`	`johnson2bin` (combinational)	any depth, odd included

An illegal combination (Gray with a non-power-of-2 depth) fails at **elaboration** with an explicit `$error`, not at runtime.

1.7.1.1 State walk: why the pointers are different widths

Both encodings guarantee the same CDC-critical property – exactly one bit changes per increment – but they buy it very differently. Walking a 3-bit Gray counter through its full cycle and putting the equivalent Johnson counter beside it shows why.

A 3-bit Gray counter has `2**3 = 8` states. To reach the same 8 states a Johnson counter needs `2N = 8`, i.e. **4 bits** – it only ever produces `2 x WIDTH` states, not `2**WIDTH`. The Johnson sequence fills with ones from the LSB, then flushes them back out with zeros; that is the whole counter (`counter_johnson.sv:32`: `{counter_gray[WIDTH-2:0], ~counter_gray[WIDTH-1]}`).

The two “which bit” columns give the **bit POSITION that flipped** (0 = LSB). They are not counts. The Hamming-distance columns state the count explicitly, and it is 1 on every row – that is the property being demonstrated.

3-bit Gray vs the equivalent 4-bit Johnson counter, full cycle. “flips bit#” is a bit POSITION; “Ham. dist” is the number of bits that changed, which is 1 everywhere including the wrap.

Step	Gray (3-bit)	flips bit#	Ham. dist	Johnson (4-bit)	flips bit#	Ham. dist	Johnson phase
0	`000`	0	1	`0000`	0	1	ones in
1	`001`	1	1	`0001`	1	1	ones in
2	`011`	0	1	`0011`	2	1	ones in
3	`010`	2	1	`0111`	3	1	ones in
4	`110`	0	1	`1111`	0	1	zeros in
5	`111`	1	1	`1110`	1	1	zeros in
6	`101`	0	1	`1100`	2	1	zeros in
7	`100`	2	1	`1000`	3	1	zeros in
wrap	`-> 000`	2	1	`-> 0000`	3	1	–

Every step – **including the wrap back to state 0** – flips exactly one bit in both encodings. Hamming distance 1 is the only property the synchronizer needs, which is why both are safe to cross.

> **The Johnson wrap is single-bit, and this is easy to get wrong.** A reviewer of this document read the earlier “bit changed” column as a bit *count* and concluded the Johnson wrap `1000 -> 0000` changed four bits. It changes one (bit 3). A twisted-ring counter is specifically constructed so that the wrap is a single-bit transition like every other step – that is what makes the `2N` cycle usable as a CDC pointer at all.

The cost difference is in the width, and it compounds:

Pointer width by encoding – Gray is logarithmic, Johnson is linear

States needed (2 x DEPTH)	DEPTH	Gray pointer bits	Johnson pointer bits
8	4	3	4
16	8	4	8

States needed (2 x DEPTH)	DEPTH	Gray pointer bits	Johnson pointer bits
32	16	5	16
64	32	6	32
128	64	7	64

Gray is `log2` in the state count; Johnson is linear. At DEPTH=4 the difference is one flop; at DEPTH=64 it is 57 flops per pointer, and each pointer is duplicated per domain and again per synchronizer stage. That is the entire argument for Gray being the default – and equally, why Johnson stays viable only at the modest depths where non-power-of-2 sizing actually matters.

What Johnson buys for that width is the freedom to stop at any count at all: its `2N` progression lands on 6, 10, 14 – and on 21 or 36 – just as naturally as on 8 or 16, whereas Gray's `2**N` can only land on powers of two. Odd depths are legal too; the "even only" restriction is stale language from the retired `fifo_async_div2`, and the only elaboration check in the RTL fires on Gray.

1.7.2 Sizing the depth

```
depth >= burst_length * (1 - read_rate / write_rate)
```

where `rate = freq * duty_cycle`. Round up, then apply the encoding's constraint: Gray rounds to the next power of two; Johnson takes the depth you ask for. The only elaboration check is `(USE_JOHNSON == 0) && ((DEPTH & (DEPTH-1)) != 0)`, so Johnson imposes no restriction at all.

> **This formula assumes the reader keeps draining throughout the burst.** It sizes for a *sustained rate mismatch*, not for burst isolation. If the reader can stall completely – arbitration loss, a blocked downstream, a paused consumer – then nothing drains during the stall and you need `depth >= burst_length` to avoid dropping writes. Between those extremes, size for the worst-case drain rate the reader can guarantee, not its average.

One term the formula above leaves out: the full flag does not track the true fill level instantly. It is derived from a pointer that has been synchronised into the write domain AND then registered, so it lags real reads by `N_FLOP_CROSS + 1` write clocks – the synchroniser stages plus the flop on `wr_full` itself in `fifo_control`. Production sizing therefore adds `N_FLOP_CROSS + 1` slots on top of the raw depth – three for the default two-stage synchroniser, four if you have configured a third stage. The worked example below shows the raw depth only, so that it lines up with the storage-overhead tables that follow; add the margin before committing to a number.

Spreadsheet. `docs/fifo_depth_calculator_v2.xlsx` does this whole calculation – rate-from-duty, raw depth, synchroniser margin, then the Gray power-of-two rounding and Johnson's absence of rounding side by side, with the slot and percentage saving between them. It carries worked example cases and a one-page cheat sheet. Note that it deliberately does *not* model clock

drift: crystal tolerance is ppm-level and moves the answer by well under one slot over any bounded burst, and a divergence large enough to matter is a steady-state violation whose fix is back-pressure, not a deeper FIFO. To size a worst case, enter the worst-case frequencies directly – fastest writer, slowest reader.

Worked example. Writer at 100 MHz bursts 100 back-to-back writes; reader pulls 1 word/clk at 80 MHz:

Worked depth-sizing example

Step	Value
Burst duration	100 / 100 MHz = 1 µs
Reads during burst	1 µs × 80 MHz = 80
Net build-up	100 − 80 = **20 words**
Gray FIFO (power-of-2)	round up to **32**
Johnson FIFO (any depth)	**20** as-is

Gray adds 12 unused slots, and the nominal overhead scales with data width:

Nominal storage overhead of Gray depth rounding by data width

`DATA_WIDTH`	Gray depth 32	Johnson depth 20	Nominal saving
32 b	1024 b	640 b	384 b (38%)
64 b	2048 b	1280 b	768 b (38%)
256 b	8192 b	5120 b	3072 b (38%)
1024 b	32 768 b	20 480 b	**12 288 b** (38%)

Those are *nominal* bit counts. Whether they translate into real area depends entirely on how the array is implemented – see the walkthrough below, which is the case most people actually hit.

Add margin for synchronizer latency if you are near the safe depth – the backpressure to the writer lags real fill level by `N_FLOP_CROSS + 1` write clocks (the synchroniser stages, plus the registered flag).

1.7.3 Walkthrough: is a non-power-of-2 FIFO worth it at 512 bits?

This is the question that motivates `USE_JOHNSON`, and the honest answer is “sometimes” – the naive 38% figure above is frequently zero in practice. Work it through.

Scenario. A 512-bit streaming path. Writer at 250 MHz bursts 96 words back-to-back; reader drains 1 word/clk at 200 MHz.

Step 1 – required depth.

```
depth >= 96 * (1 - 200/250) = 96 * 0.2 = 19.2  ->  20 words
```

Step 2 – apply the encoding constraint.

512-bit walkthrough: depth after applying the encoding constraint

Encoding	Legal depth	Chosen	Entries wasted
Gray (`USE_JOHNSON=0`)	power of 2	32	12
Johnson (`USE_JOHNSON=1`)	any depth	20	0

Step 3 – nominal storage delta.

512-bit walkthrough: nominal storage delta

	Depth	Bits
Gray	32 x 512	16 384 b (2 KiB)
Johnson	20 x 512	10 240 b (1.25 KiB)
Delta	12 entries	**6 144 b (768 B)**

Step 4 – what Johnson costs. The pointer is `DEPTH` bits wide instead of `log2(DEPTH)+1`, and it is duplicated per domain and again per synchronizer stage. With `N_FLOP_CROSS=2`:

512-bit walkthrough: pointer flop cost

	Gray (depth 32)	Johnson (depth 20)
Pointer width	6 b	20 b
Per domain: own binary + encoded pointer	6 + 6 = 12	6 + 20 = 26
Per domain: synchronized remote pointer	2 x 6 = 12	2 x 20 = 40
Per domain: gray->bin converter	0 (combinational)	0 (combinational)
Both domains	**~48 flops**	**~132 flops**

So Johnson costs roughly +**84 flops** to save **6 144 memory bits**. If the memory is flops, that is a ~73:1 win. If it is not, the trade can invert completely.

Step 5 – the part that decides it: memory granularity. `MEM_STYLE` (`FIFO_AUTO` / `FIFO_SRL` / `FIFO_BRAM`) determines whether 12 fewer entries costs 12 fewer entries of area:

512-bit walkthrough: whether the saving is real, by memory style

MEM_STYLE	Granularity	Depth 32 vs 20	Verdict
Flop array	1 entry	16 384 vs 10 240 flops	**Johnson wins big** – saving is exactly proportional, and dwarfs the +84 pointer flops
`FIFO_SRL`	32 deep per LUT (SRL32)	Both <= 32, so 512 LUTs either way	**No saving.** Johnson costs +84 flops for nothing
`FIFO_BRAM`	512 deep x 72 b per BRAM	512-bit width needs ceil(512/72) = 8 BRAMs, and both depths are far under 512	**No saving.** Same 8 BRAMs either way

Conclusion for this example. At 512 bits and depth ~20, Johnson is a clear win *only* if the array is implemented as flops. Under SRL or BRAM the depth rounding is absorbed by the primitive's own granularity, and choosing Johnson is a pure loss of +84 flops plus a wider pointer comparator on the critical path.

When a non-power-of-2 depth genuinely pays.

1. **Flop-based arrays** – no granularity, saving is proportional. The most common real win, and it grows with `DATA_WIDTH`.
2. **The depth is externally constrained, not free to round up.** If you need exactly 20 entries to cover a 20-beat burst, a credit limit, or a descriptor count, rounding to 32 is not "spare capacity", it is 12 entries of dead silicon you provisioned to satisfy an encoding.
3. **Crossing a primitive boundary.** If the rate analysis lands at 33 and Gray forces 64, an SRL32-based FIFO goes from 2 primitives per bit to 2 – no help – but a flop array pays the full 31 entries. Check where your required depth

sits relative to the primitive's step size, not relative to the next power of two.

1.7.3.1 On an ASIC the granularity argument disappears

Everything in Step 5 above is an FPGA argument. SRL32 and BRAM have a fixed step size, and that step size is what absorbs the rounding and erases Johnson's advantage. On an ASIC there is no such step: both **register files** and **compiled SRAM** are generated to a requested word count, and every memory compiler in common use takes the word count as an argument. So the depth you compute is the depth you instantiate, and an odd rounding to the next power of two is pure wasted silicon rather than something a primitive was going to charge you for anyway.

Worked case: a required depth of 36. Take the same 512-bit path, but with a rate analysis that lands on 36 entries – above 32, so Gray must round to 64.

Depth-36 case: storage delta when Gray must round 36 up to 64

	Depth	Storage at 512 b
Gray (power of 2)	64	32 768 b (4 KiB)
Johnson (any depth)	36	18 432 b (2.25 KiB)
Delta	**28 entries**	**14 336 b (1.75 KiB)**

Gray pays for 28 entries it will never use – 44 percent of the array – because 36 is not a power of two. The pointer cost runs the other way, by the same accounting as Step 4:

Depth-36 case: pointer flop cost

	Gray (depth 64)	Johnson (depth 36)
Pointer width	7 b	36 b
Per domain: own binary + encoded pointer	7 + 7 = 14	7 + 36 = 43
Per domain: synchronized remote pointer	2 x 7 = 14	2 x 36 = 72
Per domain: bin converter	0 (combinational)	0 (combinational)
Both domains	**~56 flops**	**~230 flops**

Johnson costs about **+174 flops** to save **14 336 memory bits** – roughly an 82:1 return, better than the depth-20 case because the rounding gap is wider.

Which ASIC memory style. At depth 36 both are available, and the choice is about aspect ratio rather than about the encoding:

Depth-36 case: ASIC memory styles, all of which take the word count as an argument

Style	Arbitrary depth?	Fit at 36 x 512
Flop array	Yes, no granularity at all	Works, but 18 432 flops is a lot of area and a synthesis burden
Register file	Yes, word count is a compiler argument	Good fit. Shallow and wide is what register files are for
Compiled SRAM	Yes, word count is a compiler argument	Works, but 36 words is shallow for SRAM; check the compiler's minimum depth and its area per bit at this aspect ratio

The practical answer at this shape is usually a register file: it takes depth 36 directly, it is denser than a flop array, and it avoids the periphery overhead that makes a 36-word SRAM inefficient. Confirm against your own compiler – minimum depth, word-count granularity (some compilers step in multiples of 4 or 8 words, which would reintroduce a smaller version of the FPGA problem), and area per bit at a shallow, wide aspect ratio.

Rule of thumb. Compute the required depth first. Then ask what the memory maps to and what its step size is. Only if the rounding actually crosses a step boundary – or there is no step, as with flops, register files, and compiled SRAM – does `USE_JOHNSON=1` buy you anything. On an ASIC that condition is usually met, so Johnson is worth pricing whenever the required depth is not already a power of two. On an FPGA it usually is not met, and Gray is the default: narrower pointers, and one less thing to reason about. (Both converters are combinational – `gray2bin` and `johnson2bin` alike – so the difference is pointer width, not a pipeline stage.)

1.8 SDC constraints

Both handshakes need the control toggles constrained as bounded crossings, and the quasi-static data bus constrained by `set_max_delay`, **not** `set_false_path`.

The source flop names differ between the two modules, so the constraints are not copy-paste interchangeable. The synchronizer registers (`r_req_sync`, `r_ack_sync`) and the data registers (`r_src_data_hold`, `r_dst_data`) are named the same in both; only the req/ack source flops differ:

	2-phase	4-phase
request source flop	`r_req_tog`	`r_req_src`
acknowledge source flop	`r_ack_tog`	`r_ack_dst`

The example below is written for **`cdc_2_phase_handshake`**. For `cdc_4_phase_handshake`, substitute per the table (its own RTL header prints the constraints with the right names).

```
# 1. Req / Ack single-bit crossings   [2-phase names -- see table above]
set_max_delay -datapath_only \
    -from [get_pins u_cdc/r_req_tog_reg/C] \
    -to   [get_pins u_cdc/r_req_sync_reg[0]/D] \
    <dst_period_ns>

set_max_delay -datapath_only \
    -from [get_pins u_cdc/r_ack_tog_reg/C] \
    -to   [get_pins u_cdc/r_ack_sync_reg[0]/D] \
    <src_period_ns>

# 2. Data bus (quasi-static, protected by the handshake)
set_max_delay -datapath_only \
    -from [get_pins u_cdc/r_src_data_hold_reg[*]/C] \
    -to   [get_pins u_cdc/r_dst_data_reg[*]/D] \
    <dst_period_ns>
```

Why `-datapath_only`, and why not `set_false_path`:

- `set_false_path` tells the tool to ignore the path entirely. The data bus is then free to take arbitrarily long, but the handshake only holds it stable for a bounded window – so the destination can latch a value that has partially updated. The crossing is unconstrained, not safe.
- `set_max_delay` **without** `-datapath_only` still includes clock skew and clock-uncertainty terms in the calculation. Across asynchronous domains those terms are meaningless – the tool has no defined phase relationship to

work from – so you get pessimistic, unfixable violations that tempt people back to `set_false_path`.

- `set_max_delay -datapath_only` constrains **only the combinational datapath delay** from source register to destination register, which is exactly the quantity that must fit inside the handshake's stable window. This is the one that expresses the real requirement.

Set the value to the destination clock period (or the source period, for the ack/return direction) unless you have a specific reason to tighten it.

1.9 Common mistakes

1. Multi-flop synchronizer on a multi-bit bus that changes simultaneously. Safe only for single-bit or Gray-coded values. If multiple bits change at once, different bits may be sampled from different time points, producing a value that never existed. Fix: handshake or async FIFO.

2. `set_false_path` on CDC data buses. It tells the tool to ignore timing entirely, but the data bus must arrive within a bounded window. Use `set_max_delay -datapath_only`.

3. Assuming async FIFO depth 2 is enough. Both entries are usable – `fifo_control` uses a wrap-bit full test, not a reserved slot – but the synchronized pointers lag by `N_FLOP_CROSS` cycles, so a depth-2 FIFO spends most of its time reporting stale fullness and stalls a writer that could have proceeded. Use depth >= 4.

4. Open-loop transfer before the previous one is sampled. Without an ack the NEW transfer is silently dropped – the capture guard (`src_valid && !src_busy`) ignores a pulse that arrives while the previous one is still crossing, and the first transfer completes. Minimum spacing `STRETCH_CYCLES` source clocks (the busy window; sized >= `SYNC_STAGES + 1` destination clocks).

5. Choosing 2-phase for speed without checking the reset domains. See Reset Considerations. This one has cost real silicon debug time.

1.10 Verification status

The formal harness at `formal/cdc/cdc_handshake/formal_cdc_handshake.sv` wires the DUT as:

```
.clk_src (clk),  .rst_src_n (rst_n),
.clk_dst (clk),  .rst_dst_n (rst_n),
```

Single clock, single reset for both domains. The proof therefore **cannot express asymmetric reset or asynchronous clocks**. Treat "formally verified" for the handshakes as scoped to protocol correctness under a common clock and reset.

And the DUT it proves is stale. The harness compiles `cdc_handshake_formal.sv`, a Yosys-compatible copy taken before the module was renamed to `cdc_4_phase_handshake` and grew parameters. The copy carries `DATA_WIDTH` alone; the live `rtl/cdc/cdc_4_phase_handshake.sv` carries `DATA_WIDTH`, `SYNC_STAGES`, `TIMEOUT_CYCLES` and `FAST_PATH`.

Updated 2026-09-16 (CDC-FORMAL-STALE closed). The fork is gone: the proof now flattens and reads the shipped module, and drives each parameter from its own sby task. The timeout path is proven – it never fires before a transfer is sent and always fires once one stalls past the programmed count.

`FAST_PATH` no longer exists (cdc BUG-001 (was CDC-002), 2026-09-16). Formal proved it unsound: `D_IDLE` sampled `dst_ready` and then raised `dst_valid` and the ack together on the NEXT cycle, so a receiver that dropped ready in between was acknowledged for a beat it never took – silent data loss. Made correct it saved nothing, so the parameter was removed rather than left as a knob that did nothing. The destination now always waits for an observed `dst_valid && dst_ready`.

The reset behavior described in this document is argued from the RTL encoding and – for the 2-phase hazard – confirmed on silicon. It is **not** currently covered by the formal proof or by a directed test. Extending the harness to independent clocks and resets would cover it, and would be expected to fail on the 2-phase variant.

1.11 Related Modules

CDC module reference

Module	RTL	Filelist	Test
bin2gray	rtl/cdc/bin2gray.sv	bin2gray.f	val/cdc/test_bin2gray.py
cdc_2_phase_handshake	rtl/cdc/cdc_2_phase_handshake.sv	cdc_2_phase_handshake.f	val/cdc/test_cdc_2_phase_handshake.py
cdc_4_phase_handshake	rtl/cdc/cdc_4_phase_handshake.sv	cdc_4_phase_handshake.f	val/cdc/test_cdc_4_phase_handshake.py
cdc_open_loop	rtl/cdc/cdc_open_loop.sv	cdc_open_loop.f	val/cdc/test_cdc_open_loop.py
cdc_synchronizer	rtl/cdc/cdc_synchronizer.sv	cdc_synchronizer.f	_
counter_bingray	rtl/cdc/counter_bingray.sv	counter_bingray.f	val/cdc/test_counter_bingray.py, val/cdc/test_counter_bingray_wavedrom.py
counter_johnson	rtl/cdc/counter_johnson.sv	counter_johnson.f	val/cdc/test_counter_johnson.py, val/cdc/test_counter_johnson_wavedrom.py
fifo_async	rtl/cdc/fifo_async.sv	fifo_async.f	val/cdc/test_fifo_buffer_async.py, val/cdc/test_fifo_async_wavedrom.py
gaxi_fifo_async	rtl/cdc/gaxi_fifo_	gaxi_fifo_async.f	val/cdc/test_gaxi_bu

Module	RTL	Filelist	Test
	async.sv		ffer_async.py
gaxi_skid_buffer_async	rtl/cdc/gaxi_skid_buffer_async.sv	gaxi_skid_buffer_async.f	val/cdc/test_gaxi_buffer_async.py
gray2bin	rtl/cdc/gray2bin.sv	gray2bin.f	val/cdc/test_gray2bin.py
johnson2bin	rtl/cdc/johnson2bin.sv	johnson2bin.f	val/cdc/test_johnson2bin.py

Protocol-level CDC built on these blocks: apb4_slave_cdc, apb4_slave_cdc_cg, apb5_slave_cdc, apb5_slave_cdc_cg, gaxi_skid_buffer_async, axi4_to_apb4_shim.

Supporting primitives (rtl/cdc): bin2gray, gray2bin, johnson2bin, counter_bingray, counter_johnson, glitch_free_n_dff_arn, sync_pulse, reset_sync.

1.12 Testing

Each mechanism documented here has its own module page with a Testing section and suite. The whole area runs with `make -C val/cdc clean-all && make -C val/cdc run-all-{gate,func,full}-parallel`, and every rtl/cdc module carries a passing formal harness under `formal/cdc/`.

1.13 References

- Chaney, T.J., Molnar, C.E. “Anomalous Behavior of Synchronizer and Arbiter Circuits.” *IEEE Transactions on Computers*, C-22(4), Apr. 1973. DOI: 10.1109/T-C.1973.223730 – synchronizer metastability, the reason every crossing here exists.
- Veendrick, H.J.M. “The Behaviour of Flip-Flops Used as Synchronizers and Prediction of Their Failure Rate.” *IEEE Journal of Solid-State Circuits*, 15(2), Apr. 1980. DOI: 10.1109/JSSC.1980.1051359 – the MTBF model behind every `SYNC_STAGES` / `FLOP_COUNT` choice.
- Ginosar, R. “Metastability and Synchronizers: A Tutorial.” *IEEE Design & Test of Computers*, 28(5), Sept./Oct. 2011. DOI: 10.1109/MDT.2011.113 – peer-reviewed tutorial: N-flop synchronizer, req/ack handshake, two-clock FIFO.
- Ginosar, R. “Fourteen Ways to Fool Your Synchronizer.” *Proc. 9th IEEE Int. Symp. on Asynchronous Circuits and Systems (ASYNC)*, pp. 89-96, May 2003. DOI: 10.1109/ASYNC.2003.1199169 – four-phase handshake, pulse synchronizer and the common ways synchronizers are broken.
- Sutherland, I.E. “Micropipelines.” *Communications of the ACM*, 32(6), June 1989. DOI: 10.1145/63526.63532 – two-phase (transition) signalling, the basis of `cdc_2_phase_handshake`.
- Cummings, C.E. “Clock Domain Crossing (CDC) Design & Verification Techniques Using SystemVerilog.” *SNUG Boston*, 2008 – open-loop sampling (`cdc_open_loop`, sec. 4.4) and toggle-pulse / acknowledged MCP formulations (sec. 5.6).
- Cummings, C.E. “Synthesis and Scripting Techniques for Designing Multi-Asynchronous Clock Designs.” *SNUG San Jose*, 2001 – synchronizer module partitioning behind `cdc_synchronizer`.
- Cummings, C.E. “Simulation and Synthesis Techniques for Asynchronous FIFO Design.” *SNUG San Jose*, 2002 – Gray-pointer async FIFO.
- Harris, S.L., Harris, D. *Digital Design and Computer Architecture, RISC-V Edition*, Morgan Kaufmann, 2022, ch. 3 “Sequential Logic Design”, sec. 3.5.4-3.5.6 (metastability, synchronizers, resolution time) – textbook treatment of synchronizers (secondary source).
- Deschamps, J.-P., Sutter, G.D., Cantó, E. *Guide to FPGA Implementation of Arithmetic Functions*, Springer (LNEE 149), 2012, ch. 5 “Electronic Aspects of Digital Design” – sec. 5.3.5 “Interfacing Different Clock Domains”, 5.3.5.2

handshake signaling, 5.3.5.3 asynchronous FIFO – textbook treatment (secondary source).

- Indexed in `rtl/cdc/references/README.md` (local PDF: `MetastabilitySynchronizers__Ginosar2011.pdf`, `SynchronizerPitfalls__Ginosar2003.pdf`, `ClockDomainCrossing__Cummings2008.pdf`, `MultiAsyncClockDesign__Cummings2001.pdf`, `AsyncFifoGrayPointers__Cummings2002.pdf`)

1.14 Navigation

- ← Back to CDC Index
- ← Back to Main Documentation Index

2 Binary-to-Gray converter (`bin2gray.sv`)

2.1 Overview

`bin2gray` is a purely combinational binary-to-Gray converter. Gray code – reflected binary code, unit-distance code, same thing – is a binary numeral system where two successive values differ in exactly one bit. That one-bit property is what reduces glitches and metastability exposure when a value moves between asynchronous domains, which is why Gray code turns up everywhere in clock domain crossing work.

Where it earns its keep:

- **Asynchronous FIFOs** – pointer comparisons that can't afford metastability
- **Clock domain crossing** – safe multi-bit signal transfer
- **Position encoders** – mechanical/optical encoder interfaces
- **Memory address generation** – reduces EMI and power spikes
- **Test pattern generation** – controlled single-bit transitions
- **Error detection** – single-bit error detection schemes
- **ADC/DAC interfaces** – fewer glitches in conversion systems

The whole module is one parameter and two ports:

```
module bin2gray #(
    parameter int WIDTH = 4
) (
    input  wire [WIDTH-1:0] binary,
    output wire [WIDTH-1:0] gray
);
```

2.2 Parameters

bin2gray parameters

Parameter	Default	Description
`WIDTH`	4	Bit width of both input and output (`int`, any positive integer ≥ 1; common values 4, 8, 16, 32 for address counters and data buses). Determines the number of XOR gates required.

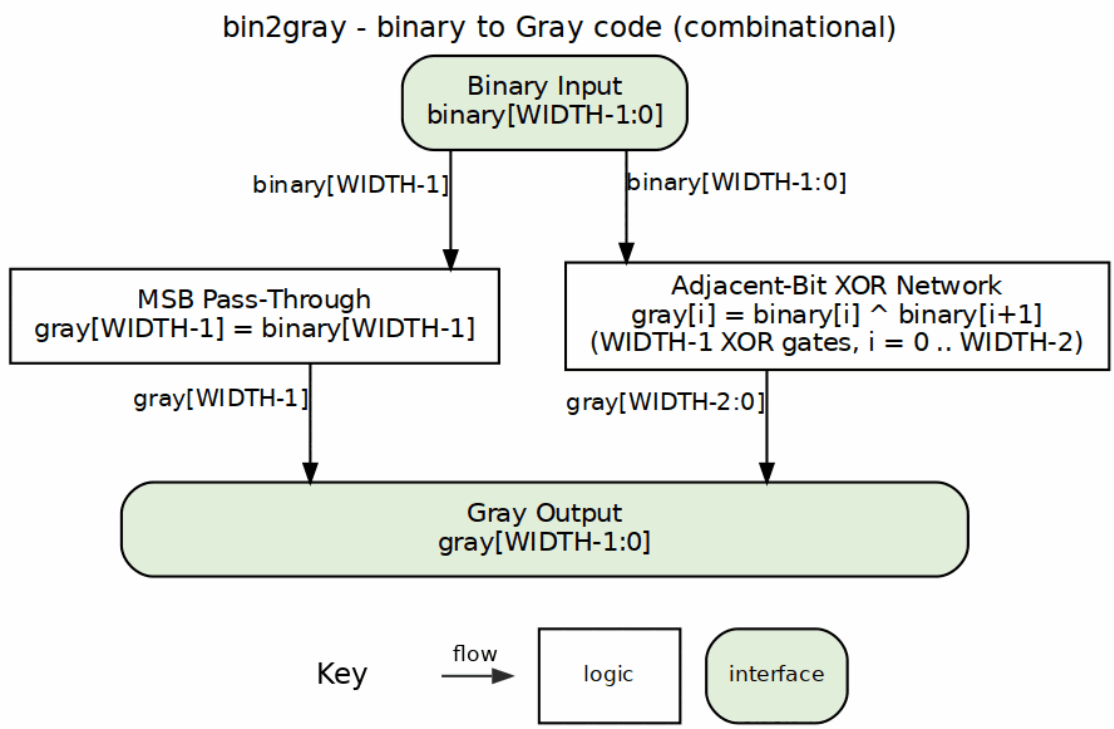
2.3 Ports

bin2gray ports

Port	Direction	Width	Description
`binary`	input	WIDTH	Input binary value
`gray`	output	WIDTH	Output Gray code value

2.4 Functional Description

2.4.1 Block diagram



bin2gray block diagram

The binary input fans out two ways. The MSB passes straight through to `gray[WIDTH-1]`. The full input also feeds a bank of WIDTH-1 XOR gates, where each lower Gray bit is its binary bit XORed with the bit above it. Both results merge into the Gray output. The module is purely combinational: one level of XOR logic, no clock.

2.4.2 Mathematical definition

The conversion follows one simple relationship:

- **MSB**: `gray[WIDTH-1] = binary[WIDTH-1]` (MSB unchanged)
- **Other bits**: `gray[i] = binary[i] ⊕ binary[i+1]` for i = 0 to WIDTH-2

2.4.3 Why Gray code?

That one-bit property buys you four things:

1. **Single Bit Transitions**: Adjacent values differ by exactly one bit
2. **Glitch Reduction**: Eliminates intermediate states during transitions

3. **Metastability Prevention**: Safer for asynchronous clock domain crossings
4. **Mechanical Encoders**: Natural for optical/magnetic position encoders

2.4.4 Gray code sequences

Walk the tables and watch the “changed bit” column – one bit, every time.

2.4.4.1 2-bit Gray Code

Decimal	Binary	Gray	Transition
0	00	00	-
1	01	01	bit 0
2	10	11	bit 1
3	11	10	bit 0

2.4.4.2 3-bit Gray Code

Decimal	Binary	Gray	Changed Bit
0	000	000	-
1	001	001	0
2	010	011	1
3	011	010	0
4	100	110	2
5	101	111	0
6	110	101	1
7	111	100	0

2.4.4.3 4-bit Gray Code (Complete Table)

Dec	Binary	Gray	Dec	Binary	Gray
0	0000	0000	8	1000	1100
1	0001	0001	9	1001	1101
2	0010	0011	10	1010	1111
3	0011	0010	11	1011	1110
4	0100	0110	12	1100	1010
5	0101	0111	13	1101	1011
6	0110	0101	14	1110	1001
7	0111	0100	15	1111	1000

2.4.5 Core logic

```
genvar i;
generate
    for (i = 0; i < WIDTH - 1; i++) begin : gen_gray
        assign gray[i] = binary[i] ^ binary[i+1];
    end
endgenerate

assign gray[WIDTH-1] = binary[WIDTH-1];
```

What the RTL builds is a bank of XOR gates: each lower Gray bit is the XOR of the current and next binary bits, the MSB comes straight across, and everything evaluates in parallel – a single level of logic.

The block diagram shows the same structure for any WIDTH.

2.4.6 Companion converter: gray2bin

You'll almost always need the trip back. The inverse conversion:

```
module gray2bin #(
    parameter int WIDTH = 4
) (
    input  wire [WIDTH-1:0] gray,
    output wire [WIDTH-1:0] binary
);

    genvar i;

    // MSB is unchanged
    assign binary[WIDTH-1] = gray[WIDTH-1];

    // Other bits: XOR accumulation from MSB down
    generate
        for (i = WIDTH-2; i >= 0; i--) begin : gen_binary
            assign binary[i] = binary[i+1] ^ gray[i];
        end
    endgenerate

endmodule
```

2.4.6.1 Gray-to-Binary Truth Table (4-bit)

Gray	Binary	Conversion Process
0000	0000	bin[3]=0, bin[2]=0⊕0=0,

Gray	Binary	Conversion Process
		bin[1]=0⊕0=0, bin[0]=0⊕0=0
0001	0001	bin[3]=0, bin[2]=0⊕0=0, bin[1]=0⊕0=0, bin[0]=0⊕1=1
0011	0010	bin[3]=0, bin[2]=0⊕0=0, bin[1]=0⊕1=1, bin[0]=1⊕1=0
0010	0011	bin[3]=0, bin[2]=0⊕0=0, bin[1]=0⊕1=1, bin[0]=1⊕0=1

2.5 Timing Characteristics

One XOR gate of delay – typically 0.1-0.3ns in modern processes – and that's the whole critical path. Each binary bit drives at most 2 XOR gates, and the delay is constant regardless of WIDTH. This module will never be your timing problem.

Typical FPGA resource usage, by width:

bin2gray resource usage by width (typical FPGA)

WIDTH	LUTs	Delay (ns)	Max Freq (MHz)
4	3	0.2	800+
8	7	0.2	800+
16	15	0.3	600+
32	31	0.4	500+

2.6 Usage Examples

Every parameter and port below is read from the module declaration.

```
bin2gray #(
    .WIDTH                  (4)
) u_bin2gray (
    .binary                 (binary),
    .gray                   (gray)
);
```

2.7 Design Notes

- **Always use Gray for async boundaries** – it's the baseline tool for safe asynchronous transfer of a multi-bit count.
- **Timing is a non-issue**: purely combinational, no setup/hold concerns of its own.
- **Verify the single-bit property**: adjacent values must differ by exactly one bit. Check it in simulation, not by eye.
- **Plan for the trip back**: you usually need `gray2bin` on the other side.
- **Check the width**: enough bits for the application range, no more.

The binary-to-Gray converter is one of those fundamental building blocks – simple, combinational, dependable. For asynchronous interfaces and glitch-free operation it's close to indispensable. Respect the two rules from the multi-bit synchronizer example above – register the Gray value in the source domain, and make sure it only ever increments by one – and this module will never be the source of your bug.

2.8 Related Modules

- **gray2bin**: performs the inverse conversion (Gray to binary)
- **counter_bingray**: combined binary/Gray counter; what the FIFOs instantiate
- **glitch_free_n_dff_arn** / **cdc_synchronizer**: multi-flop synchronizers that carry the registered Gray value across
- **fifo_async** / **gaxi_fifo_async**: the primary users of this conversion

2.9 Testing

2.9.1 Comprehensive Test Bench

```
module tb_bin2gray;

    parameter int WIDTH = 4;
    parameter int MAX_VAL = (1 << WIDTH) - 1;

    logic [WIDTH-1:0] binary, gray;
    logic [WIDTH-1:0] expected_gray;
    logic [WIDTH-1:0] binary_check;

    // DUT
    bin2gray #(.WIDTH(WIDTH)) dut (
        .binary(binary),
        .gray(gray)
    );

    // Reference converter for checking
    gray2bin #(.WIDTH(WIDTH)) check_conv (
        .gray(gray),
        .binary(binary_check)
    );

    // Test sequence
    initial begin
        $display("Testing %d-bit Binary to Gray converter", WIDTH);

        // Test all possible values
        for (int i = 0; i <= MAX_VAL; i++) begin
            binary = i;
            expected_gray = compute_gray_reference(i);

            #1; // Allow propagation

            // Check conversion correctness
            if (gray !== expected_gray) begin
                $error("Mismatch: binary=%b, expected_gray=%b,
actual_gray=%b",
                       binary, expected_gray, gray);
            end

            // Check round-trip conversion
            if (binary_check !== binary) begin
                $error("Round-trip failed: original=%b, recovered=%b",
```

```
                    binary, binary_check);
            end

            // Check single-bit change property
            if (i > 0) begin
                logic [WIDTH-1:0] prev_gray =
compute_gray_reference(i-1);
                int bit_changes = $countones(gray ^ prev_gray);
                if (bit_changes != 1) begin
                    $error("Multiple bit change: %d→%d, gray %b→%b,
changes=%d",
                           i-1, i, prev_gray, gray, bit_changes);
                end
            end

            $display("PASS: %d → binary:%b → gray:%b", i, binary,
gray);
        end

        $display("All tests passed!");
        $finish;
    end

    // Reference Gray code computation
    function [WIDTH-1:0] compute_gray_reference(input [WIDTH-1:0]
bin);
        compute_gray_reference[WIDTH-1] = bin[WIDTH-1];
        for (int i = WIDTH-2; i >= 0; i--) begin
            compute_gray_reference[i] = bin[i] ^ bin[i+1];
        end
    endfunction

endmodule
```

2.9.2 Property-Based Verification

```
// bin2gray is purely combinational -- it has no clock, so there is no
clocking
// event to write concurrent assertions against. Use IMMEDIATE
assertions in an
// always_comb block: they re-evaluate whenever an input settles, need
no clock,
// and work under both simulation and formal.
//
// The checker declares PORTS. A checker with no ports bound with
`(.*)` would
// connect nothing, and every assertion would silently evaluate on X.
```

```
module bin2gray_properties #(
    parameter int WIDTH = 4
) (
    input logic [WIDTH-1:0] binary,
    input logic [WIDTH-1:0] gray
);

    // Reference encode/decode as functions -- a property body cannot
contain
    // variable declarations, continuous assigns, or genvar loops.
    function automatic logic [WIDTH-1:0] gray_encode(input logic
[WIDTH-1:0] bin);
        gray_encode = bin ^ (bin >> 1);
    endfunction

    function automatic logic [WIDTH-1:0] gray_decode(input logic
[WIDTH-1:0] gry);
        gray_decode[WIDTH-1] = gry[WIDTH-1];
        for (int i = WIDTH-2; i >= 0; i--) begin
            gray_decode[i] = gray_decode[i+1] ^ gry[i];
        end
    endfunction

    always_comb begin
        // MSB is passed through unchanged
        a_msb_unchanged: assert (gray[WIDTH-1] == binary[WIDTH-1]);

        // Output matches the reference encoding
        a_matches_reference: assert (gray == gray_encode(binary));

        // Adjacent codes differ in exactly one bit (skip the wrap at
the top,
        // which is also single-bit but is worth asserting separately)
        if (binary < WIDTH'((2**WIDTH) - 1)) begin
            a_single_bit_change: assert (
                $countones(gray ^ gray_encode(binary + WIDTH'(1))) ==
1
            );
        end

        // Decoding the output recovers the input
        a_round_trip: assert (gray_decode(gray) == binary);
    end

endmodule
```

```
// Bind at the point of use, OUTSIDE the checker -- a module cannot
bind itself
// into another module from within its own body. Ports connect by name
via (.*)
// because the checker's port names match the DUT's.
bind bin2gray bin2gray_properties #(.WIDTH(WIDTH)) props_inst (.*);
```

The `bind` line belongs in a file compiled alongside the DUT (typically the testbench), not inside the checker module. Under Verilator these compile with `--assert`; under SymbiYosys the same immediate assertions are proven exhaustively rather than sampled.

2.9.3 Coverage Model

```
covergroup bin2gray_cg;

    cp_binary_values: coverpoint binary {
        bins zero = {0};
        bins powers_of_two[] = {1, 2, 4, 8, 16}; // For appropriate
WIDTH
        bins max_value = {2**WIDTH - 1};
        bins mid_range[] = {[1:2**(WIDTH-1)-1]};
        bins upper_range[] = {[2**(WIDTH-1):2**WIDTH-2]};
    }

    cp_gray_values: coverpoint gray {
        bins all_values[] = {[0:2**WIDTH-1]};
    }

    cp_bit_patterns: coverpoint binary {
        bins alternating_01 = {8'b01010101}; // For WIDTH=8
        bins alternating_10 = {8'b10101010};
        bins all_ones = {'1};
        bins all_zeros = {'0};
    }

    // Cross coverage between input patterns and outputs
    cross_binary_gray: cross cp_binary_values, cp_gray_values;

endcovergroup
```

2.9.4 Test files

- `val/cdc/test_bin2gray.py` – functional verification

```
pytest val/cdc/test_bin2gray.py -v
```

2.10 References

- Gray, F. “Pulse Code Communication.” U.S. Patent 2,632,058, filed Nov. 1947, issued Mar. 1953. Google Patents – the reflected binary code and the XOR conversion this module implements.
- Cummings, C.E. “Simulation and Synthesis Techniques for Asynchronous FIFO Design.” *SNUG San Jose*, 2002 – why a Gray-coded value is the safe way to synchronize a multi-bit counter.
- Roy, S. *Advanced Digital System Design*, Springer, 2024, ch. 3 “Basic Combinational Circuits”, sec. 3.11 (binary/Gray conversion) – textbook treatment (secondary source).
- Indexed in `rtl/cdc/references/README.md` (local PDF: `GrayCode__Gray1953.pdf`, `AsyncFifoGrayPointers__Cummings2002.pdf`)

2.11 Navigation

- ← Back to CDC Index
- ← Back to Main Documentation Index

3 Gray-to-binary converter (`gray2bin.sv`)

3.1 Overview

`gray2bin` converts Gray code (reflected binary code) back to standard binary using XOR reduction. When a Gray pointer crosses into a domain that needs to do arithmetic on it, this is the module that turns it back into a number – asynchronous FIFO pointer comparison is the classic case, and it shows up in plenty of other CDC paths too.

3.2 Parameters

gray2bin parameters

Parameter	Default	Description
`WIDTH`	4	Data width in bits

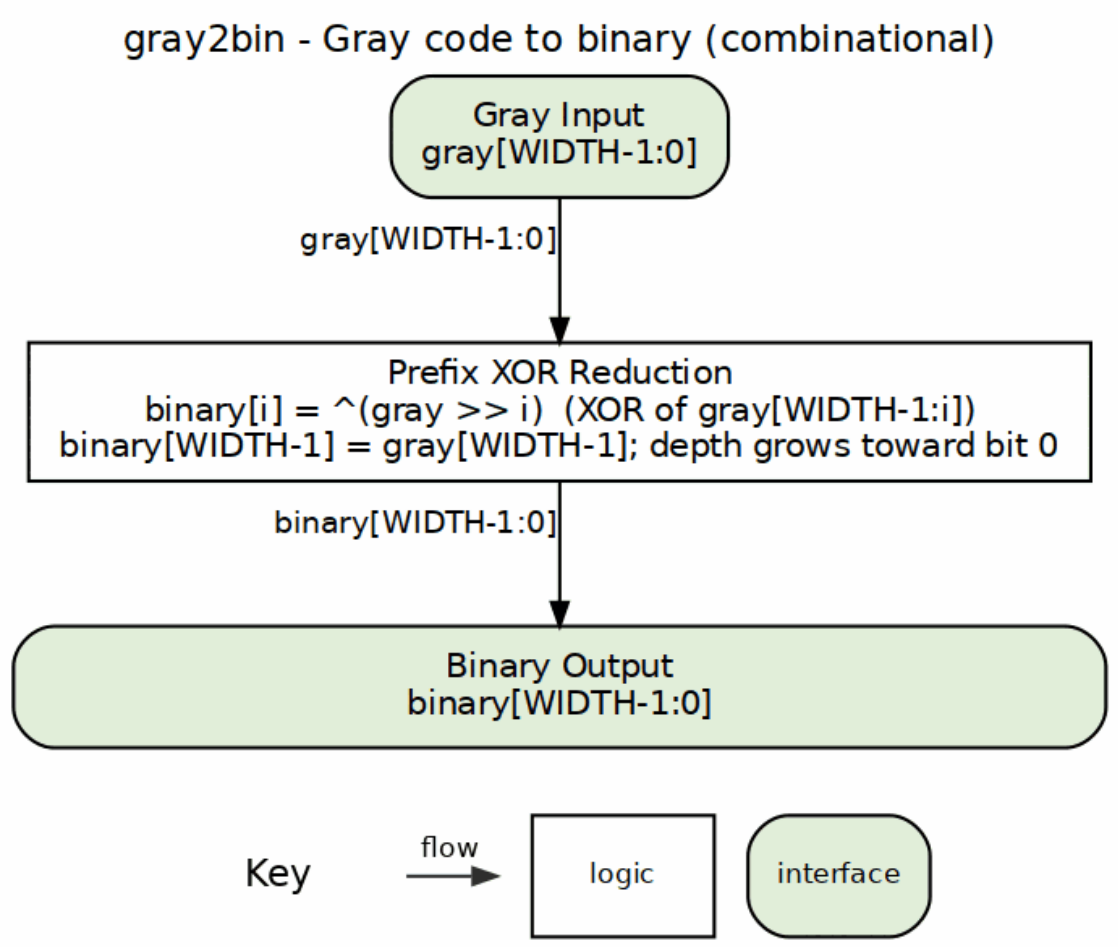
3.3 Ports

gray2bin ports

Port	Direction	Width	Description
`gray`	input	WIDTH	Gray code input
`binary`	output	WIDTH	Binary code output

3.4 Functional Description

3.4.1 Block diagram



gray2bin block diagram

The Gray input feeds a prefix XOR reduction. Each binary bit is the XOR of every Gray bit at and above it, so `binary[WIDTH-1]` equals `gray[WIDTH-1]` and each lower bit adds one more term. The result drives the binary output. The module is purely combinational, and its depth grows toward bit 0.

Gray code is a binary numeral system where **only one bit changes** between consecutive values:

```
Binary  Gray   Transitions
000  →  000
001  →  001    1 bit change
010  →  011    1 bit change
011  →  010    1 bit change
100  →  110    1 bit change
```

```
101  →  111    1 bit change
110  →  101    1 bit change
111  →  100    1 bit change
```

That property is exactly what CDC needs:

- **Metastability protection**: only one bit in transition means no multi-bit race conditions
- **Safe sampling**: intermediate values during a transition are still valid Gray codes
- **FIFO pointers**: essential for async FIFO full/empty detection

3.4.2 Conversion algorithm

Each binary bit is the XOR of all Gray bits from that position up to the MSB:

```
binary[i] = gray[MSB] ⊕ gray[MSB-1] ⊕ ... ⊕ gray[i]
```

The RTL says the same thing with a shift and a reduction:

```
genvar i;
generate
    for (i = 0; i < WIDTH; i++) begin : gen_gray_to_bin
        assign binary[i] = ^(gray >> i);
    end
endgenerate
```

Bit by bit, for a 4-bit converter:

```
// For 4-bit example:
assign binary[3] = gray[3];                          // MSB unchanged
assign binary[2] = gray[3] ^ gray[2];                // XOR from MSB
down
assign binary[1] = gray[3] ^ gray[2] ^ gray[1];      // XOR from MSB
down
assign binary[0] = gray[3] ^ gray[2] ^ gray[1] ^ gray[0]; // XOR all
bits
```

3.4.3 Worked conversions

3.4.3.1 4-Bit Conversion Table

Gray[3:0]	Binary[3:0]	Calculation
0000	0000	0⊕0⊕0⊕0=0, 0⊕0⊕0=0, 0⊕0=0, 0=0
0001	0001	0⊕0⊕0⊕1=1,

Gray[3:0]	Binary[3:0]	Calculation
		0⊕0⊕0=0, 0⊕0=0, 0=0
0011	0010	0⊕0⊕1⊕1=0, 0⊕0⊕1=1, 0⊕0=0, 0=0
0010	0011	0⊕0⊕1⊕0=1, 0⊕0⊕1=1, 0⊕0=0, 0=0
0110	0100	0⊕1⊕1⊕0=0, 0⊕1⊕1=0, 0⊕1=1, 0=0
0111	0101	0⊕1⊕1⊕1=1, 0⊕1⊕1=0, 0⊕1=1, 0=0
0101	0110	0⊕1⊕0⊕1=0, 0⊕1⊕0=1, 0⊕1=1, 0=0
0100	0111	0⊕1⊕0⊕0=1, 0⊕1⊕0=1, 0⊕1=1, 0=0

3.4.3.2 Step-by-Step Example (Gray 0110 → Binary)

```
Gray input: 0110

binary[3] = gray[3] = 0
binary[2] = gray[3] ^ gray[2] = 0 ^ 1 = 1
binary[1] = gray[3] ^ gray[2] ^ gray[1] = 0 ^ 1 ^ 1 = 0
binary[0] = gray[3] ^ gray[2] ^ gray[1] ^ gray[0] = 0 ^ 1 ^ 1 ^ 0 = 0

Result: binary = 0100 (decimal 4)
```

3.4.4 Implementation

The whole module is one line per bit:

```
assign binary[i] = ^(gray >> i);
```

Read it as: shift `gray` right by `i`, then XOR-reduce whatever is left – which XORs together every bit from position `i` to the MSB. The generate loop makes it work for any WIDTH, synthesis tools

recognize the pattern and optimize it, all bits compute in parallel, and the result maps onto XOR tree structures.

3.5 Timing Characteristics

The delay is an XOR tree of depth `log2(WIDTH)` – typically 1-2 LUT delays for most widths, with the critical path running from the MSB input to the LSB output.

gray2bin delay by width

WIDTH	XOR Levels	Typical Delay
4	2	1 LUT delay
8	3	2 LUT delays
16	4	2 LUT delays
32	5	3 LUT delays

Modern synthesis tools need no help here: they recognize the XOR-reduction pattern, build balanced trees, and share XOR gates where possible.

3.6 Usage Examples

3.6.1 Asynchronous FIFO pointers

Convert the synchronized Gray pointers back to binary so you can do arithmetic on them:

```
// Convert synchronized Gray pointers back to binary for comparison
gray2bin #(.WIDTH(5)) wr_ptr_conv (
    .gray(sync_wr_ptr_gray),
    .binary(sync_wr_ptr_bin)
);

gray2bin #(.WIDTH(5)) rd_ptr_conv (
    .gray(sync_rd_ptr_gray),
    .binary(sync_rd_ptr_bin)
);

// Now can perform binary arithmetic for occupancy calculation
assign occupancy = sync_wr_ptr_bin - sync_rd_ptr_bin;
```

3.6.2 Clock domain crossing counter

```
// Convert Gray counter back to binary for address generation
gray2bin #(.WIDTH(AW)) addr_converter (
    .gray(gray_counter),
    .binary(memory_address)
);
```

3.6.3 Position encoding

```
// Convert Gray-encoded position to binary for processing
gray2bin #(.WIDTH(8)) position_decoder (
    .gray(gray_position),
    .binary(bin_position)
);
```

3.7 Design Notes

Width scaling. `binary[i]` is the XOR of Gray bits `i..WIDTH-1`, so the unshared cost is `WIDTH*(WIDTH-1)/2` XOR gates – quadratic in WIDTH, as the module header notes ("WIDTH * (WIDTH/2 average)"). Synthesis shares the common prefixes, so realised area lands between linear and quadratic; do not budget it as linear for wide converters. Delay grows with log(WIDTH), and the module works well up to 64+ bits.

Input validation. Gray codes have no invalid states – every input decodes to something. If you want to gate on the *source* of the Gray code instead:

```
// Gray codes have no invalid states - all inputs are valid
// However, may want to validate source of Gray code
always_comb begin
    if (gray_code_valid) begin
        binary_out = converted_binary;
    end else begin
        binary_out = '0;  // Default when invalid
    end
end
```

Pipelining. If you're closing timing at very high speed, a pipeline stage is cheap insurance:

```
// Optional pipeline stage for timing closure
always_ff @(posedge clk) begin
    if (!rst_n) begin
        binary_reg <= '0;
    end else begin
        binary_reg <= binary_comb;
    end
end
```

3.7.1 Common mistakes

Bit order confusion. The shift direction matters:

```
// WRONG: Bit order matters in Gray codes
assign binary[i] = ^(gray << i);  // Left shift instead of right

// CORRECT:
assign binary[i] = ^(gray >> i);  // Right shift
```

Width mismatches. Match the parameter to the actual bus width:

```
// WRONG: Input/output width mismatch
gray2bin #(.WIDTH(4)) converter (
    .gray(gray_5bit),     // 5-bit input
    .binary(binary_4bit)  // 4-bit output
);
```

```
// CORRECT: Match widths
gray2bin #(.WIDTH(5)) converter (
    .gray(gray_5bit),
    .binary(binary_5bit)
);
```

Timing assumptions. The output is combinational, not instantaneous:

```
// WRONG: Assuming zero delay
gray_in <= new_value;
binary_out <= converted_value;  // May not be updated yet

// CORRECT: Account for combinational delay
gray_in <= new_value;
#1;  // Wait for conversion
binary_out <= converted_value;
```

3.8 Related Modules

- **bin2gray**: Performs inverse conversion (binary to Gray)
- **counter_bingray**: Combined binary/Gray counter
- **fifo_async**: Uses Gray codes for CDC
- **johnson2bin**: Johnson counter to binary converter (different algorithm)

3.9 Testing

3.9.1 Exhaustive Testing

```
// For reasonable widths, test all possible inputs
for (int i = 0; i < (1 << WIDTH); i++) begin
    gray_input = int_to_gray(i);      // Convert integer to Gray
    expected_binary = i;              // Expected result
    #1;
    assert(binary_output == expected_binary);
end
```

3.9.2 Property-Based Verification

```
// gray2bin is purely combinational -- no clock port, so there is no
clocking
// event for a concurrent assertion to sample. Use IMMEDIATE
assertions in an
// always_comb block, the same shape bin2gray.md uses: they re-
evaluate whenever
// an input settles, need no clock, and hold under simulation and
formal alike.
//
// The checker declares PORTS. Binding one with no ports via `(.*)`
connects
// nothing, and every assertion then evaluates silently on X.

module gray2bin_properties #(
    parameter int WIDTH = 4
) (
    input logic [WIDTH-1:0] gray,
    input logic [WIDTH-1:0] binary
);

    function automatic logic [WIDTH-1:0] gray_encode(input logic
[WIDTH-1:0] bin);
        gray_encode = bin ^ (bin >> 1);
    endfunction

    always_comb begin
        // MSB passes through unchanged
        a_msb_unchanged: assert (binary[WIDTH-1] == gray[WIDTH-1]);

        // Round trip: re-encoding the output reproduces the input
        a_round_trip: assert (gray_encode(binary) == gray);
    end

endmodule
```

```
// Bind at the point of use, OUTSIDE the checker.
bind gray2bin gray2bin_properties #(.WIDTH(WIDTH)) u_props (.*);
```

3.9.3 Random Testing

```
repeat (10000) begin
    random_gray = $random();
    expected_result = reference_gray2bin(random_gray);
    #1;
    assert(binary_output == expected_result);
end
```

3.9.4 Test files

- `val/cdc/test_gray2bin.py` – functional verification

```
pytest val/cdc/test_gray2bin.py -v
```

3.10 References

- Gray, F. “Pulse Code Communication.” U.S. Patent 2,632,058, filed Nov. 1947, issued Mar. 1953. Google Patents – the reflected binary code this module decodes.
- Cummings, C.E. “Simulation and Synthesis Techniques for Asynchronous FIFO Design.” *SNUG San Jose*, 2002 – Gray-to-binary conversion of a synchronized pointer before arithmetic on it.
- Roy, S. *Advanced Digital System Design*, Springer, 2024, ch. 3 “Basic Combinational Circuits”, sec. 3.11 (binary/Gray conversion) – textbook treatment (secondary source).
- Indexed in `rtl/cdc/references/README.md` (local PDF: `GrayCode__Gray1953.pdf`, `AsyncFifoGrayPointers__Cummings2002.pdf`)

3.11 Navigation

- ← Back to CDC Index
- ← Back to Main Documentation Index

4 Johnson-to-binary converter (`johnson2bin.sv`)

4.1 Overview

`johnson2bin` converts Johnson counter codes to binary – the piece that makes asynchronous FIFOs with non-power-of-2 depths possible. Standard Gray-to-binary conversion won't do the job here; the Johnson sequence has its own structure, and decoding it takes position detection rather than XOR reduction.

4.2 Parameters

johnson2bin parameters

Parameter	Default	Description
`JCW`	10	Johnson Counter Width (equals FIFO DEPTH)
`WIDTH`	4	Binary output width (typically `$clog2(DEPTH) + 1`)

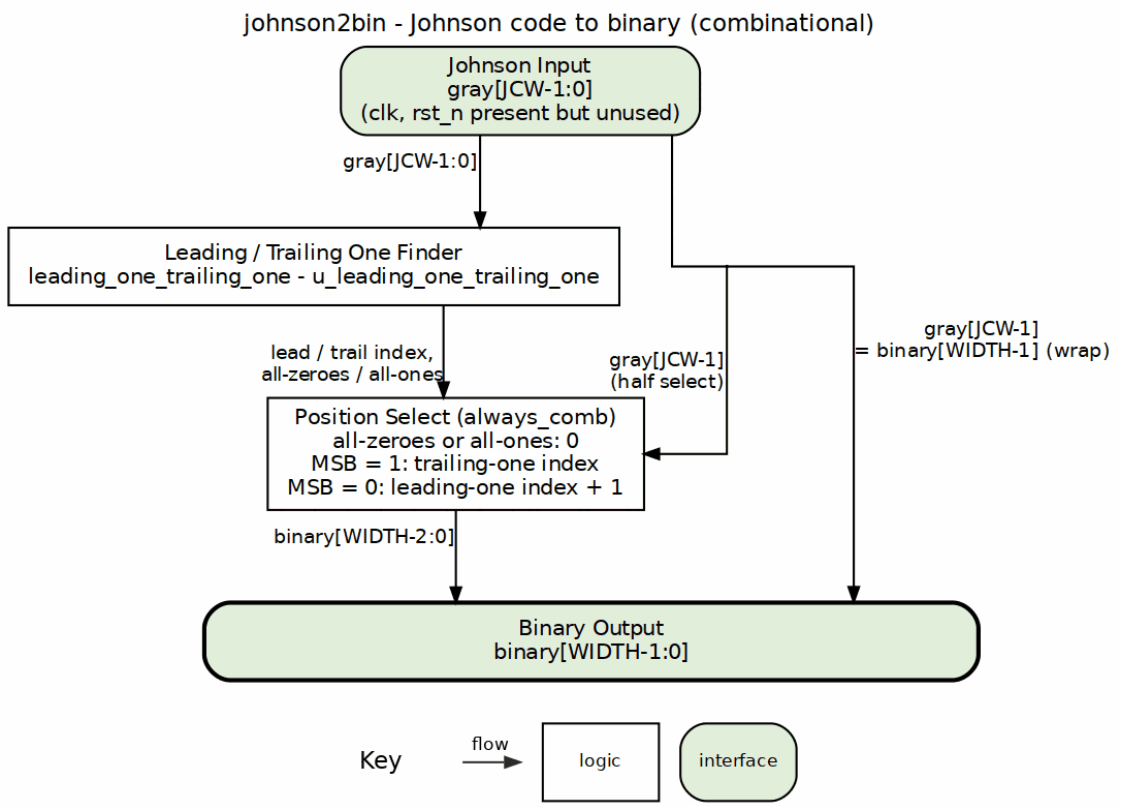
4.3 Ports

johnson2bin ports

Port	Direction	Width	Description
`clk`	input	1	Clock input. Declared but unused; the module is combinational.
`rst_n`	input	1	Active-low reset. Declared but unused; the module is combinational.
`gray`	input	JCW	Johnson counter input (misleadingly named “gray”)
`binary`	output	WIDTH	Binary output representing position in sequence

4.4 Functional Description

4.4.1 Block diagram



johnson2bin block diagram

The Johnson input drives `leading_one_trailing_one`, which finds the boundary between the run of ones and the run of zeros. The position select picks the answer: zero for the all-zeros and all-ones codes, the trailing-one index when the MSB is 1, and the leading-one index plus one when the MSB is 0. Separately, the MSB passes straight through as `binary[WIDTH-1]`, the wrap bit. The module is combinational; its `clk` and `rst_n` ports are unused.

4.4.2 The Johnson sequence

Johnson counters are **shift registers with inverted feedback**:

```
For JCW=6 (DEPTH=6):
State 0:  000000  ← Initial state
State 1:  000001  ← Shift left, insert ~MSB (1)
State 2:  000011
State 3:  000111
State 4:  001111
State 5:  011111
```

```
State 6:  111111  ← All 1s (fill complete)
State 7:  111110  ← Shift left, insert ~MSB (0)
State 8:  111100
State 9:  111000
State 10: 110000
State 11: 100000
State 0:  000000  ← Cycle complete (12 states total)
```

> **Fill direction.** This library shifts in at the LSB. `counter_johnson.sv` implements `counter_gray <= {counter_gray[WIDTH-2:0], ~counter_gray[WIDTH-1]}`, so ones enter at bit 0 and march upward. An earlier revision of this page showed the mirror image (ones entering at the MSB), which is also a valid Johnson counter but is *not* the one this library builds – and both worked conversion examples inherited the error. Note that external references frequently use the MSB-fill convention, so check against `counter_johnson.sv` rather than against a textbook.

The properties that matter for decoding:

- **Single bit transitions**: only one bit changes per state
- **Two phases**:
  - **First half** (0 to DEPTH-1): filling with 1s from the **right** (ones enter at bit 0 and march upward: 000000 → 000001 → 000011 → ...)
  - **Second half** (DEPTH to 2×DEPTH-1): emptying 1s from the **right** (111111 → 111110 → 111100 → ...)
- **Wrap indicator**: the MSB says which half of the cycle you're in

4.4.3 The algorithm

The conversion works by **position detection** – finding the transition between the 1s and the 0s:

```
if (w_all_zeroes || w_all_ones) begin
    w_binary = {WIDTH{1'b0}};
end else if (gray[JCW-1]) begin
    // Second half: use trailing one position directly
    w_binary = {{(WIDTH-N){1'b0}}, w_trailing_one};
end else begin
    // First half: use leading one + 1
    w_binary = {{(WIDTH-N){1'b0}}, (w_leading_one + 1'b1)};
end
```

Position detection itself is farmed out to a helper:

```
leading_one_trailing_one #(
    .WIDTH(JCW)
) u_leading_one_trailing_one (
    .data(gray),
    .leadingone(w_leading_one),
    .trailingone(w_trailing_one),
    .all_zeroes(w_all_zeroes),
    .all_ones(w_all_ones),
    .valid(w_valid)
);
```

4.4.4 Three-part binary construction

```
assign binary[WIDTH-1]   = gray[JCW-1];                 // MSB = wrap
indicator
assign binary[WIDTH-2:0] = w_binary[WIDTH-2:0];         // Lower bits
= position
```

4.4.5 Width calculations

```
localparam int N = $clog2(JCW);                         // Address
bits needed
localparam int PAD_WIDTH = (WIDTH > N+1) ? WIDTH-N-1 : 0; // Padding
if needed
```

4.4.6 Why this algorithm works

First half (MSB = 0). The pattern is `000...0111...1` – zeros followed by ones. The leading-one position tells you how far the fill has progressed: position = leading_one + 1, because we've filled (leading_one + 1) positions with 1s.

Second half (MSB = 1). The pattern flips to `111...1000...0` – ones followed by zeros. We're emptying from the right, and the trailing-one position shows how far: position = trailing_one.

4.4.7 Worked conversions

4.4.7.1 First Half Conversion (MSB = 0)

```
JCW=6, WIDTH=$clog2(6)+1=4 -- the legal pairing, and what both FIFOs
instantiate.

Johnson: 001111 (set bits {0,1,2,3}: w_leading_one = 3, w_trailing_one
= 0)
Logic: gray[5]=0 → first half, position = w_leading_one + 1 = 3 + 1 =
4
Binary: 4'b0100 = {wrap=0, addr=4} -- Johnson state 4
```

4.4.7.2 Second Half Conversion (MSB = 1)

```
Johnson: 111000 (set bits {3,4,5}: w_leading_one = 5, w_trailing_one =
3)
```

```
Logic: gray[5]=1 → second half, position = w_trailing_one = 3
Binary: 4'b1011 = {wrap=1, addr=3} -- Johnson state 9 of 12

The output is not the number 9. It is the FIFO pointer {wrap, addr}:
the ninth
state of the sequence is address 3 on the second lap. Reading
`4'b1011` as the
decimal 11 and expecting 9 is the usual first confusion with this
module.
```

4.4.7.3 Special Cases

```
Johnson: 000000 → Binary: 4'b0000 (all zeros case)
Johnson: 111111 → Binary: 4'b1000 (all ones: lower bits forced to 0,
but the RTL
         unconditionally sets binary[WIDTH-1] = gray[JCW-1] = 1, so
the wrap
         bit is SET -- this is {wrap=1, addr=0}, NOT the same as all-
zeros.
         FIFO full detection depends on this.)
```

4.4.8 Comparison with standard gray2bin

Aspect	Standard Gray2Bin	Johnson2Bin (johnson2bin)
Input type	Traditional Gray code	Johnson counter sequence
Algorithm	XOR reduction	Position detection
Complexity	Simple combinational	Complex position logic
Clock requirement	None	None (clk/rst_n ports exist but are unused)
Width scaling	Logarithmic	Linear with JCW
Use case	Power-of-2 sequences	Any sequence length

4.5 Timing Characteristics

The critical path runs through the position detection logic: leading/trailing one detection, a binary addition, and the output mux. Synthesis complexity is higher than a standard Gray conversion, and resources scale with JCW – the Johnson width, not its log:

```
Resources scale with JCW (Johnson Counter Width):
- Small FIFOs (JCW ≤ 16): Reasonable overhead
- Medium FIFOs (JCW = 32-64): Significant resources
- Large FIFOs (JCW > 64): May become limiting factor
```

4.6 Usage Examples

This is where the module actually lives – decoding the crossed pointer in `fifo_async.sv` (`USE_JOHNSON=1`):

```
// fifo_async.sv (USE_JOHNSON=1) usage:
johnson2bin #(
    .JCW(JCW),                      // = DEPTH
    .WIDTH(AW + 1)                  // Address width + wrap bit
) rd_ptr_gray2bin_inst(
    .binary(w_wdom_rd_ptr_bin),     // Binary for arithmetic
    .gray(r_wdom_rd_ptr_gray),      // Johnson counter from CDC
    .clk(wr_clk),
    .rst_n(wr_rst_n)
);
```

4.7 Design Notes

4.7.1 The clock and reset ports are unused

`johnson2bin` declares `clk` and `rst_n` but **never uses them**. The conversion is entirely combinational: a single `always_comb` block driven by `leading_one_trailing_one`, which is itself combinational (one `always_comb` plus continuous assignments, no `always_ff` anywhere).

The ports are retained so the module can be dropped into a clocked context without an interface change, and so a future revision could register the output without breaking callers. Tie them off to whatever the surrounding domain uses; nothing in this module samples them. Do not expect a cycle of latency – the output follows `gray` combinationally, and long conversion paths must be registered by the caller.

4.7.2 When to use Johnson vs. Gray

```
// Use Johnson counter approach when:
parameter int DEPTH = 10;  // Non-power-of-2 required
parameter int DEPTH = 6;   // Specific size needed
parameter int DEPTH = 14;  // Standard Gray won't work

// Use standard Gray approach when:
parameter int DEPTH = 8;   // Power-of-2 is acceptable
parameter int DEPTH = 16;  // Efficiency more important
parameter int DEPTH = 64;  // Large depth, resource conscious
```

4.7.3 Resource planning

```
// Resource overhead estimation:
// Standard Gray: ~log2(DEPTH) XOR gates
// Johnson: ~DEPTH comparators + position logic

// Break-even point typically around DEPTH = 16-32
```

4.7.4 Verification challenges

```
// More complex verification due to:
// 1. Two-phase sequence behavior
// 2. Position detection correctness
// 3. Special case handling (all 0s, all 1s)
// 4. Wraparound boundary conditions
```

4.7.5 Invalid Johnson sequences

Nothing checks them. It is worth being blunt, because the `.valid(w_valid)` port on the helper reads like it does:

```
.valid(w_valid)  // NOT a Johnson-pattern check -- see below
```

`leading_one_trailing_one` computes `assign valid = |data`, so `valid` means "input is nonzero" and nothing more. A pattern such as `6'b010101`, which is not a Johnson sequence at all, drives it high. And in `johnson2bin.sv` `w_valid` is declared, connected, and then never read – no output carries it, no assertion consumes it. A malformed sequence is decoded into a wrong pointer silently.

A valid Johnson pattern is one with a single 1-to-0 transition (or its mirror). If you need that enforced, it has to come from somewhere else: the upstream `counter_johnson` produces only legal patterns by construction, which is why the decoder can assume them.

4.7.6 Debug visibility

```
// Monitor internal state for debugging:
// - w_leading_one: Position of leftmost 1
// - w_trailing_one: Position of rightmost 1
// - w_all_zeroes: Special case flag
// - w_all_ones: Special case flag
// - w_valid: nonzero-input flag ONLY -- not a Johnson-pattern check
```

4.7.7 Common issues

1. **Invalid sequences**: non-Johnson input decodes silently to a wrong pointer
2. **Width mismatches**: JCW vs. actual Johnson counter width
3. **Phase confusion**: misunderstanding first vs. second half logic
4. **Boundary conditions**: all-zeros and all-ones handling

4.7.8 Advanced topics

Hierarchical Johnson counters. For very large depths, a hierarchical approach is worth considering:

```
// Break large Johnson counter into smaller segments
// Use multiple johnson2bin instances with higher-level arbitration
```

Alternative position detection. Other implementations detect position differently:

- **Priority encoders**: find first/last set bit
- **Thermometer decoders**: convert 1-hot positions
- **LUT-based**: for small, fixed widths

4.8 Related Modules

- **counter_johnson**: Generates Johnson counter sequences
- **leading_one_trailing_one**: Position detection helper
- **fifo_async / gaxi_fifo_async (USE_JOHNSON=1)**: Primary users of this conversion
- **gray2bin**: Standard Gray-to-binary for comparison
- **fifo_control**: Uses converted binary values for status generation

4.9 Testing

From the test suite (`val/cdc/test_johnson2bin.py`):

- **Key test scenarios**:
  - Gray Johnson Counter to Binary Converter Test
  - This test verifies the Gray Johnson counter to binary conversion functionality:
  - CONFIGURATION:
  - JCW: Johnson Counter Width (10, 12, 16, 20)
  - WIDTH: Binary output width (4, 5, 6, 8)

Run levels come from the standard grid: `REG_LEVEL=GATE|FUNC|FULL` selects the parameter set, `TEST_LEVEL` the per-test depth. Run the whole area with `make -C val/cdc run-all-func-parallel`, never bare pytest for suites.

4.10 References

- Johnson, R.R. “Electronic Counter.” U.S. Patent 3,030,581, issued Apr. 1962 (continuation of a 1953 application). Google Patents – provenance of the Johnson-counter name only; leading/trailing-one decode is repo construction on a standard code.
- Roy, S. *Advanced Digital System Design*, Springer, 2024, ch. 4 “Basic Sequential Circuits”, sec. 4.9 “Shift Register Counters” – textbook treatment of the Johnson counter sequence (secondary source).
- Indexed in `rtl/cdc/references/README.md` (no paper archived for this technique; marked standard technique)

4.11 Navigation

- ← Back to CDC Index
- ← Back to Main Documentation Index

5 Binary-Gray counter (`counter_bingray.sv`)

5.1 Overview

`counter_bingray` is a dual-output counter: you get the binary and the Gray representation of the same count, registered in parallel. Asynchronous FIFOs are the reason it exists – the Gray value is what crosses the clock boundary safely (one bit changes per increment, so metastability never gets a foothold), while the binary value stays home for the arithmetic.

You'll also see it used for clock-domain-crossing counters, position encoder interfaces, state machine counters that need glitch-free outputs, and memory address generation in dual-port systems.

```
module counter_bingray #(
    parameter int WIDTH = 4
) (
    input  logic             clk,
    input  logic             rst_n,
    input  logic             enable,
    output logic [WIDTH-1:0] counter_bin,
    output logic [WIDTH-1:0] counter_bin_next,
    output logic [WIDTH-1:0] counter_gray
);
```

5.2 Parameters

counter_bingray parameters

Parameter	Default	Description
`WIDTH`	4	Bit width of both binary and Gray code outputs (`int`, any positive integer ≥ 1). Determines maximum count value (2^WIDTH - 1).

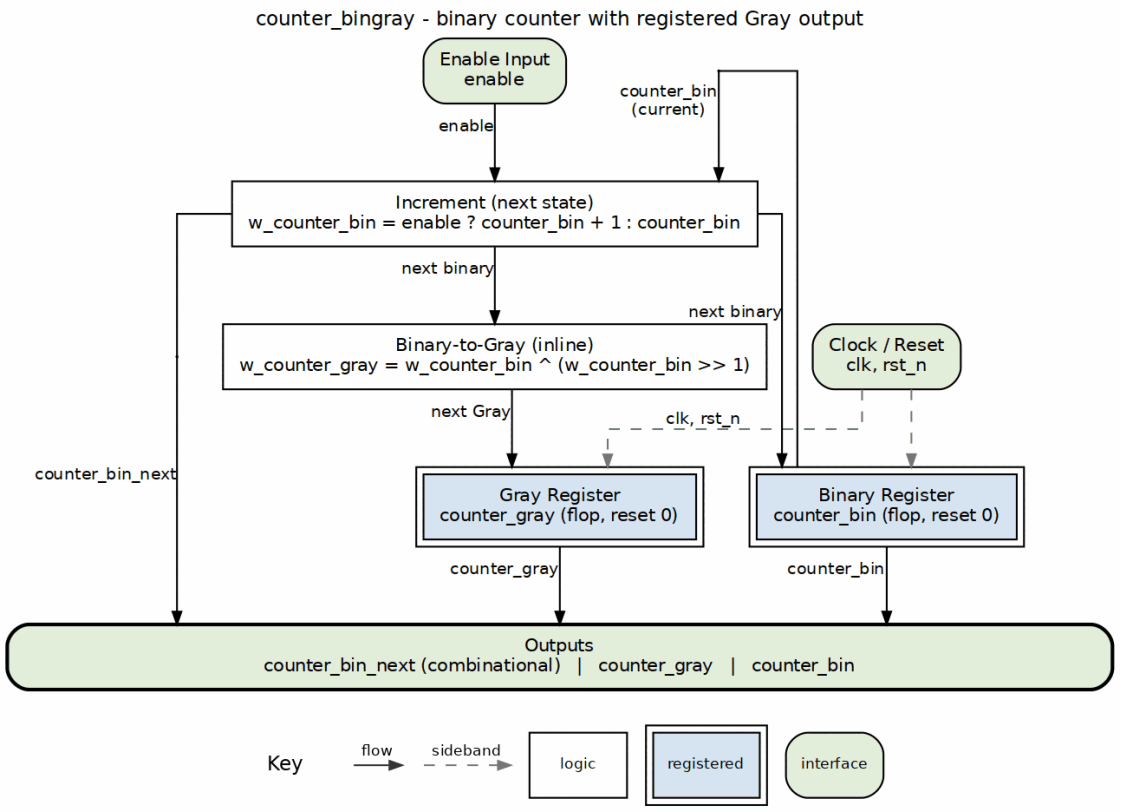
5.3 Ports

counter_bingray ports

Port	Direction	Width	Description
`clk`	input	1	System clock input
`rst_n`	input	1	Active-low asynchronous reset
`enable`	input	1	Counter enable control
`counter_bin`	output	WIDTH	Binary counter output (registered)
`counter_bin_next`	output	WIDTH	Next binary value (combinational)
`counter_gray`	output	WIDTH	Gray code output (registered)

5.4 Functional Description

5.4.1 Block diagram



counter_bingray block diagram

`enable` drives the increment, which computes the next binary value. That value goes three ways: straight out as `counter_bin_next` (combinational), into the binary register, and through an inline XOR (`w ^ (w >> 1)`) into the Gray register. Both registers drive their outputs, so `counter_gray` is a registered value that changes one bit per step. The binary register feeds back up the right edge into the increment. `clk` and `rst_n` are sideband to both registers.

Gray code (reflected binary code) guarantees adjacent values differ by exactly one bit. That buys you:

1. **Metastability Prevention**: when crossing clock domains, only one bit changes at a time
2. **Glitch Elimination**: reduces intermediate states during transitions
3. **Asynchronous Safety**: safe for use in asynchronous circuits

The conversion is the standard one:

- **MSB**: `gray[WIDTH-1] = binary[WIDTH-1]`
- **Other bits**: `gray[i] = binary[i] ^ binary[i+1]` for i = 0 to WIDTH-2

5.4.2 Gray code sequence (4-bit)

Decimal	Binary	Gray	Changes
0	0000	0000	-
1	0001	0001	bit 0
2	0010	0011	bit 1
3	0011	0010	bit 0
4	0100	0110	bit 2
5	0101	0111	bit 0
6	0110	0101	bit 1
7	0111	0100	bit 0
8	1000	1100	bit 3
9	1001	1101	bit 0
10	1010	1111	bit 1
11	1011	1110	bit 0
12	1100	1010	bit 2
13	1101	1011	bit 0
14	1110	1001	bit 1
15	1111	1000	bit 0

5.4.3 Internal logic

```
logic [WIDTH-1:0] w_counter_bin, w_counter_gray;

assign w_counter_bin = enable ? (counter_bin + 1) : counter_bin;
assign w_counter_gray = w_counter_bin ^ (w_counter_bin >> 1);
assign counter_bin_next = w_counter_bin;
```

The next binary value is a conditional increment; the Gray value is that next binary value XORed with itself shifted right by one. Both registers update from the same `always_ff`:

```
always_ff @(posedge clk, negedge rst_n) begin
    if (!rst_n) begin
        counter_bin  <= 'b0;
        counter_gray <= 'b0;
    end else begin
        counter_bin  <= w_counter_bin;
```

```
        counter_gray <= w_counter_gray;
    end
end
```

Note what this buys you over a `bin2gray` plus a separate flop: the Gray output is registered from the *next-state* value in the same process, so the encoding transient never leaves the module.

5.5 Timing Characteristics

Three paths matter:

1. **Binary increment** – standard binary addition timing
2. **Gray conversion** – a single XOR level: `gray = bin ^ (bin >> 1)` gives every output bit one 2-input XOR, so the delay is constant in WIDTH. (The log(WIDTH) XOR depth belongs to the *decode* direction, `gray2bin`.)
3. **Combined path** – increment + conversion in the same cycle

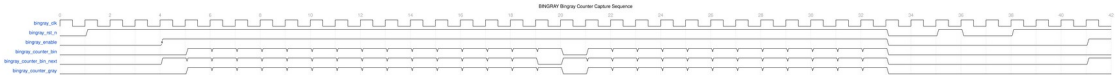
Clock-to-Q is the standard flip-flop delay, the combinational piece depends on the adder and XOR depth, and your setup check has to absorb the longest of them. Expect 200-400 MHz in a modern FPGA as-is; pipeline the Gray conversion if you need more.

5.6 Waveforms

WaveDrom timing diagrams for the Binary-Gray counter are available.

Four scenarios walk through the dual-output counter design:

5.6.1 Scenario 1: Binary vs Gray Code Comparison

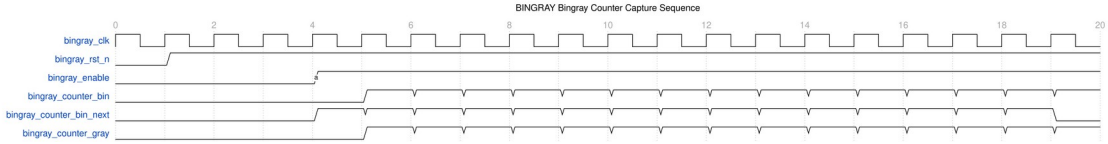


BinGray Comparison

WaveJSON: bingray_counter_binary_vs_gray.json

Side-by-side comparison of both outputs: - Binary: Normal sequential counting (0→1→2→3→...) - Gray: Single-bit transitions between values - Shows full cycle demonstrating encoding differences - Illustrates why Gray code is CDC-safe

5.6.2 Scenario 2: Single-Bit Transitions (CDC Safety) KEY FEATURE

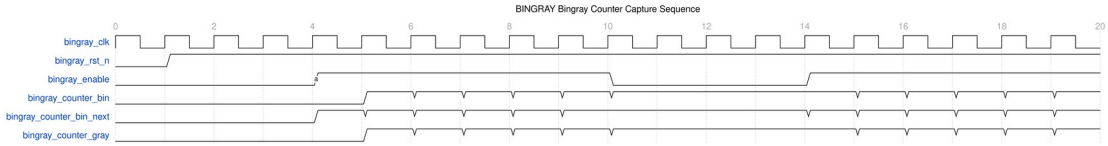


BinGray Single-Bit Transitions

WaveJSON: bingray_counter_single_bit_transitions.json

Gray code CDC safety property: - Each Gray transition changes EXACTLY one bit - Hamming distance = 1 between adjacent values - **Critical for fifo_async CDC mechanism** - Prevents metastability in clock domain crossing

5.6.3 Scenario 3: Lookahead Signal (counter_bin_next)

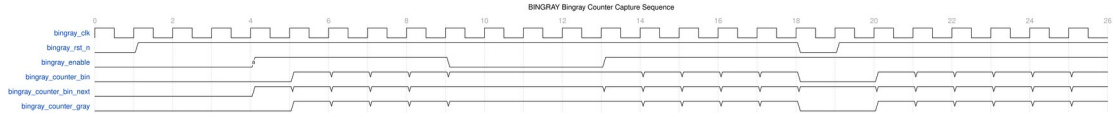


BinGray Lookahead

WaveJSON: bingray_counter_lookahead.json

Combinational lookahead feature: - Predicts next binary value one cycle ahead - Useful for FIFO full/empty prediction - Shows enable gating behavior - Enables early decision-making

5.6.4 Scenario 4: Enable and Reset Control



BinGray Enable and Reset

WaveJSON: bingray_counter_enable_reset.json

Control signal behavior: - Both outputs hold when enable=0 - Asynchronous reset to zero - Clean state transitions - Reset during counting demonstration

---

To regenerate these waveforms:

```
pytest val/cdc/test_counter_bingray_wavedrom.py -v
# Then convert JSON to PNG:
cd docs/markdown/assets/rtl-cdc/waves/counter_bingray
for f in *.json; do wavedrom-cli -i "$f" -p "${f%.json}.png"; done
```

What Makes Binary-Gray Counters Special:

The waveforms put the unique dual-output design on display: - **Dual Encoding**: Binary for arithmetic, Gray for CDC safety - **Single-Bit Transitions**: Gray code changes one bit at a time - **Lookahead**: counter_bin_next provides one-cycle prediction - **CDC-Safe**: Safe for asynchronous clock domain crossing

Relationship to fifo_async:

Binary-Gray counters are the foundation of the standard `fifo_async` module: - **fifo_async** uses this counter for read/write pointers - Gray outputs cross clock domains safely - Binary outputs used for arithmetic (occupancy, address generation) - Logarithmic width (`$clog2(DEPTH) + 1`) for resource efficiency

Comparison Table:

Feature	BinGray Counter	Johnson Counter
Output Width	`$clog2(DEPTH) + 1` bits	`DEPTH` bits
CDC Method	Standard Gray code	Johnson code
Depth Support	Power-of-2 only	Any depth, odd included
Resource Efficiency	Logarithmic (better for large depths)	Linear
Conversion	XOR tree (simple)	Position detection (complex)
Used In	`fifo_async` USE_JOHNSON=0	`fifo_async` USE_JOHNSON=1

Comparison with Other Modules:

- `test_counter_johnson_wavedrom.py` - Johnson counter (any depth, linear width)
- `test_fifo_async_wavedrom.py` - BinGray counter in action (async FIFO, power-of-2)

5.7 Usage Examples

Every parameter and port below is read from the module declaration.

```
counter_bingray #(
    .WIDTH                  (4)
) u_counter_bingray (
    .clk                    (clk),
    .rst_n                  (rst_n),
    .enable                 (enable),
    .counter_bin            (counter_bin),
    .counter_bin_next       (counter_bin_next),
    .counter_gray           (counter_gray)
);
```

5.8 Design Notes

Resource cost: 2×WIDTH flip-flops (binary and Gray registers are separate), plus the increment logic and the single-level XOR for the Gray conversion. The critical path runs through the binary increment and the Gray conversion in series.

```
// Optional: Pipeline Gray conversion for high speed
logic [WIDTH-1:0] counter_gray_pipe;
always_ff @(posedge clk) begin
    counter_gray_pipe <= w_counter_gray;
end
```

Mark the Gray register as an async-crossing register so the placer keeps the sync chain tight:

```
(* ASYNC_REG = "TRUE" *) logic [WIDTH-1:0] counter_gray; // Xilinx
// synthesis attribute ASYNC_REG of counter_gray is "TRUE"  // Quartus
```

Dynamic power scales with switching activity; static is minimal. Gate the clock with `enable` if the counter sits idle for long stretches.

5.8.1 Common mistakes

1. **Metastability** – the Gray value must go through a proper synchronizer in the destination domain; the encoding alone doesn't sample for you.
2. **Timing violations** – pipeline the Gray conversion if the increment+XOR path doesn't close.
3. **Incorrect FIFO status** – check the Gray-to-binary decode on the synchronized pointer before blaming the flags.
4. **Reset skew** – use proper reset synchronization in each domain.

For debug: assert the Gray properties in simulation, capture real transitions on a logic analyzer, run timing analysis on the crossing paths, and verify the synchronizer stage count against your MTBF target.

5.9 Related Modules

- **counter_johnson**: the non-power-of-2 alternative – any depth, linear width
- **bin2gray** / **gray2bin**: the standalone converters, if you ever need the encode or decode alone
- **glitch_free_n_dff_arn**: the multi-flop synchronizer the Gray output crosses through
- **fifo_async** / **gaxi_fifo_async**: the modules this counter exists to serve

5.10 Testing

5.10.1 Test scenarios

1. **Sequential counting**: both outputs increment correctly
2. **Gray code properties**: single-bit changes between adjacent values
3. **Reset behavior**: both outputs reset to zero
4. **Enable control**: hold behavior when disabled
5. **Rollover**: clean wrap-around from maximum value

5.10.2 Coverage

```
covergroup counter_bingray_cg @(posedge clk);
    cp_binary: coverpoint counter_bin {
        bins all_values[] = {[0:2**WIDTH-1]};
    }

    cp_gray: coverpoint counter_gray {
        bins all_values[] = {[0:2**WIDTH-1]};
    }

    cp_enable: coverpoint enable {
        bins enabled = {1};
        bins disabled = {0};
    }

    cp_transitions: coverpoint counter_bin {
        bins transitions[] = ([0:2**WIDTH-2] => [1:2**WIDTH-1]);
        bins rollover = (2**WIDTH-1 => 0);
    }
endcovergroup
```

5.10.3 Assertions

```
// Verify Gray code has single bit changes
property gray_single_bit_change;
    @(posedge clk) disable iff (!rst_n)
    enable && !$isunknown(counter_gray) |->
    $countones(counter_gray ^ $past(counter_gray)) <= 1;
endproperty

assert property (gray_single_bit_change);

// Verify binary and Gray relationship
property bin_gray_relationship;
    @(posedge clk) disable iff (!rst_n)
```

```
    counter_gray == (counter_bin ^ (counter_bin >> 1));
endproperty

assert property (bin_gray_relationship);
```

5.10.4 Test files

- `val/cdc/test_counter_bingray.py` – full functional verification
- `val/cdc/test_counter_bingray_wavedrom.py` – WaveDrom timing diagrams

```
# Full functional test (basic/medium/full levels)
pytest val/cdc/test_counter_bingray.py -v

# WaveDrom waveform generation
pytest val/cdc/test_counter_bingray_wavedrom.py -v
```

5.11 References

- Cummings, C.E. “Simulation and Synthesis Techniques for Asynchronous FIFO Design.” *SNUG San Jose*, 2002 – “Gray code counter - Style #2”: binary and Gray registers updated together, binary addresses memory, Gray crosses the boundary.
- Gray, F. “Pulse Code Communication.” U.S. Patent 2,632,058, filed Nov. 1947, issued Mar. 1953. Google Patents – the reflected binary code.
- Roy, S. *Advanced Digital System Design*, Springer, 2024, ch. 3 “Basic Combinational Circuits”, sec. 3.11 (binary/Gray conversion) – textbook treatment of the conversion (secondary source).
- Indexed in `rtl/cdc/references/README.md` (local PDF: `AsyncFifoGrayPointers__Cummings2002.pdf`, `GrayCode__Gray1953.pdf`)

5.12 Navigation

- ← Back to CDC Index
- ← Back to Main Documentation Index

6 Johnson counter (`counter_johnson.sv`)

6.1 Overview

`counter_johnson` implements a Johnson counter – also called a twisted-ring counter or a switch-tail counter. It's a shift register with a twist: the inverted output of the last stage feeds back into the first, which gives you a sequence with 2×WIDTH unique states. You'll see these generating multi-phase clock signals, driving state machine control, and running sequential timing applications – and, in this library, serving as the non-power-of-2 FIFO pointer behind `USE_JOHNSON=1`.

```
module counter_johnson #(
    parameter int WIDTH = 4
) (
    input  logic                clk,
    input  logic                rst_n,
    input  logic                enable,
    output logic [WIDTH - 1:0]  counter_gray
);
```

6.2 Parameters

counter_johnson parameters

Parameter	Default	Description
`WIDTH`	4	Number of stages in the Johnson counter (`int`, `>= 2`). Generates 2×WIDTH unique states; determines sequence length and number of output phases.

The `>= 2` bound is real, not a style preference. The shift expression is `counter_gray <= {counter_gray[WIDTH-2:0], ~counter_gray[WIDTH-1]}`, whose `[WIDTH-2:0]` part-select is reversed at `WIDTH=1` and fails elaboration. A 1-stage Johnson counter has no use anyway.

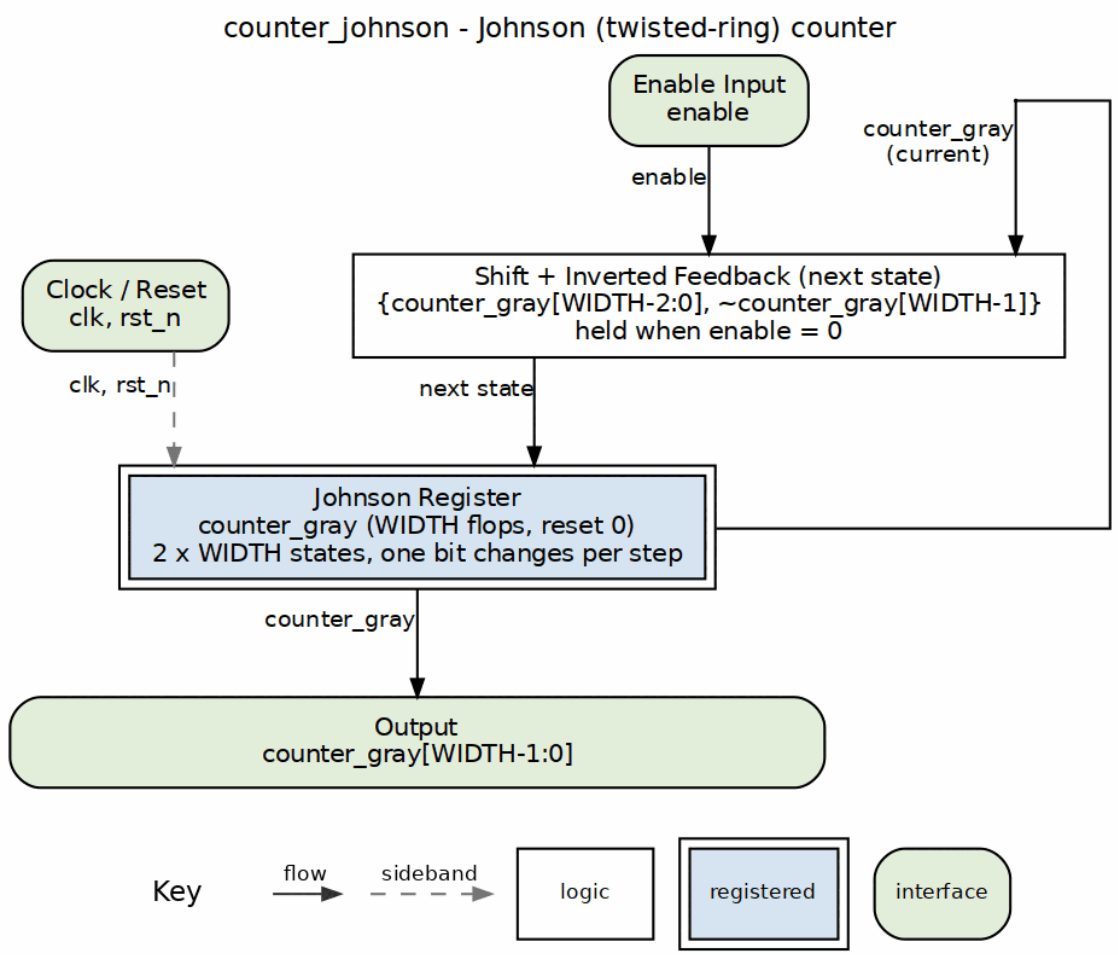
6.3 Ports

counter_johnson ports

Port	Direction	Width	Description
`clk`	input	1	System clock input
`rst_n`	input	1	Active-low asynchronous reset
`enable`	input	1	Counter enable control
`counter_gray`	output	WIDTH	Johnson counter output register

6.4 Functional Description

6.4.1 Block diagram



counter_johnson block diagram

`enable` gates the next-state logic, which shifts the register left and feeds the inverted MSB into bit 0 (it holds the value when `enable` is 0). The Johnson register drives `counter_gray`. Its current value feeds back up the right edge into the shift logic. `clk` and `rst_n` are sideband to the register.

A Johnson counter is a shift register with inverted feedback:

- **Normal operation**: bits shift left (or right) each clock cycle
- **Feedback**: the complement of the MSB feeds back to the LSB
- **Sequence length**: 2×WIDTH states before repeating

Mathematically, for a WIDTH-bit counter:

```
Next_State[i] = Current_State[i-1]  for i = 1 to WIDTH-1
Next_State[0] = ~Current_State[WIDTH-1]
```

And here is the entire implementation:

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        counter_gray <= {WIDTH{1'b0}};  // Reset to all zeros
    else if (enable) begin
        counter_gray <= {counter_gray[WIDTH-2:0], ~counter_gray[WIDTH-
1]};
    end
end
```

Four lines of actual logic: shift the low bits up, feed the inverted MSB into the LSB, hold when `enable` is low, clear on reset.

6.4.2 State sequences

6.4.2.1 4-bit Johnson Counter (WIDTH=4)

Step	counter_gray	Decimal	Description
0	0000	0	Reset state
1	0001	1	First ‘1’ enters
2	0011	3	‘1’ propagates
3	0111	7	‘1’ propagates
4	1111	15	All ’1’s
5	1110	14	First ‘0’ enters
6	1100	12	‘0’ propagates
7	1000	8	‘0’ propagates
8	0000	0	Back to start

6.4.2.2 3-bit Johnson Counter (WIDTH=3)

Step	counter_gray	Binary Pattern
0	000	All zeros
1	001	One bit set
2	011	Two bits set
3	111	All ones
4	110	Two bits set
5	100	One bit set

Step	counter_gray	Binary Pattern
6	000	Cycle repeats

6.4.2.3 2-bit Johnson Counter (WIDTH=2)

Step	counter_gray	State Name
0	00	State A
1	01	State B
2	11	State C
3	10	State D
4	00	Back to A

6.4.3 NOT self-starting – reset is mandatory

This RTL is a bare twisted-ring shift register (`counter_gray <= {counter_gray[WIDTH-2:0], ~counter_gray[WIDTH-1]}`) with **no state-correction logic**, so it is **not** self-starting:

- **Valid states**: 2×WIDTH states in the normal sequence
- **Invalid states**: (2^WIDTH - 2×WIDTH) states not in sequence
- **Recovery**: **none.** The invalid states form their own closed cycle and never converge to the valid sequence. For WIDTH=4 the 8 invalid states cycle `0010→0101→1011→0110→1101→1010→0100→1001→0010` forever; for WIDTH=3 the pair `010↔101` is a parasitic 2-cycle. **You must guarantee a clean reset** – after a glitch or SEU the counter can lock permanently into the parasitic cycle. If self-correction is required, add explicit decode/recovery logic (not present here).

6.4.4 Symmetry

The sequence has inherent symmetry: the first half fills with '1's (0→1→11→111…), the second half flushes them back out with '0's (111→110→100→000…). Each bit effectively represents a different phase of the cycle.

6.4.5 Single-bit transitions – but Johnson is not Gray

Johnson counter outputs share the one-bit-change property that makes Gray code safe to synchronize – consecutive states differ by exactly **one** bit, including the wrap from the last state back to state 0 – with predictable progression and easy state decode. But they are **not** Gray code:

> **Johnson is not Gray code.** Johnson achieves single-bit transitions through sequential shift-register operation; Gray code achieves them through a mathematical encoding (`gray = bin ^ (bin >> 1)`).

Consequences: - Johnson yields `2N` states from N bits; Gray yields `2**N`. - Johnson is safe only for a counter stepping through its own sequence. It cannot encode an arbitrary value, and there is no closed-form binary conversion – decoding needs `johnson2bin`, which detects position. - Do not substitute one for the other in general-purpose CDC. For an arbitrary binary value crossing domains, use Gray (`bin2gray`/`gray2bin`).

6.5 Timing Characteristics

The logic is minimal – a shift register and an inverter – so there are no complex timing requirements. Typical maximum frequency is 400-600 MHz in modern FPGAs, limited by the shift-register timing. For extreme speeds, pipeline:

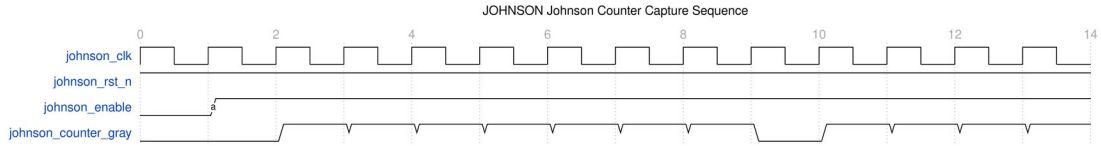
```
// For high-speed applications, consider pipeline stages
logic [WIDTH-1:0] johnson_pipe;
always_ff @(posedge clk) begin
    johnson_pipe <= {counter_gray[WIDTH-2:0], ~counter_gray[WIDTH-1]};
end
```

6.6 Waveforms

WaveDrom timing diagrams for the Johnson counter are available.

Four scenarios demonstrate the unique properties of Johnson counters:

6.6.1 Scenario 1: Walking Ones and Walking Zeros Pattern

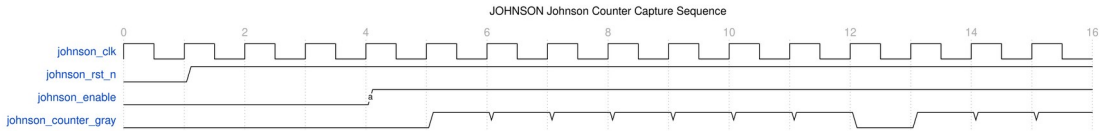


Johnson Walking Pattern

WaveJSON: johnson_counter_walking_pattern.json

Complete 2×WIDTH state cycle (8 states for WIDTH=4): - Walking ones: 0000 → 0001 → 0011 → 0111 → 1111 - Walking zeros: 1111 → 1110 → 1100 → 1000 → 0000 - Demonstrates predictable sequential progression

6.6.2 Scenario 2: Single-Bit Transitions (CDC Safety) KEY FEATURE

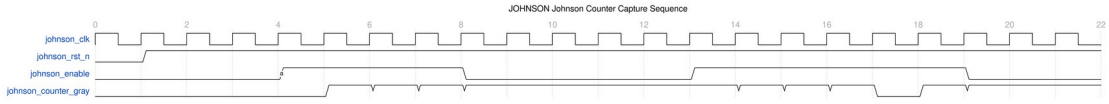


Johnson Single-Bit Transitions

WaveJSON: johnson_counter_single_bit_transitions.json

Each transition changes only ONE bit: - CDC-safe like Gray codes - **Critical for the fifo_async USE_JOHNSON=1 CDC mechanism** - Prevents metastability in clock domain crossing - Hamming distance = 1 between all adjacent states

6.6.3 Scenario 3: Enable Control

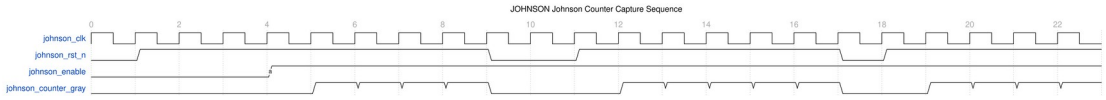


Johnson Enable Control

WaveJSON: johnson_counter_enable_control.json

Enable control and state holding: - Counter advances when enable=1 - Counter holds state when enable=0 - Clean enable/disable transitions - Demonstrates conditional counting

6.6.4 Scenario 4: Reset Behavior



Johnson Reset Behavior

WaveJSON: johnson_counter_reset_behavior.json

Reset and initialization: - Asynchronous reset to all zeros (0000) - Immediate reset effect - Clean restart from reset state - Reset during counting operation

---

To regenerate these waveforms:

```
pytest val/cdc/test_counter_johnson_wavedrom.py -v
# Then convert JSON to PNG:
cd docs/markdown/assets/rtl-cdc/waves/counter_johnson
for f in *.json; do wavedrom-cli -i "$f" -p "${f%.json}.png"; done
```

What Makes Johnson Counters Special:

The waveforms highlight the unique properties that make Johnson counters useful: - **Single-Bit Transitions**: Only one bit changes per state, making them CDC-safe - **2×WIDTH States**: More efficient than one-hot (N states with N/2 flip-flops) - **Walking Pattern**: Natural “fill then empty” sequence useful for visual effects - **Reset-dependent**: NOT self-starting – a valid state is only guaranteed by reset

Relationship to fifo_async (USE_JOHNSON=1):

Johnson counters are the foundation of the `fifo_async` `USE_JOHNSON=1` CDC mechanism: - **fifo_async with USE_JOHNSON=1** uses Johnson counters for pointer synchronization - Single-bit transitions enable safe clock domain crossing - Linear width scaling (DEPTH bits) allows any depth, odd included

Comparison with Other Counters:

- `test_counter_bingray_wavedrom.py` - Binary-Gray counter (power-of-2 depths, logarithmic width)
- `test_fifo_async_wavedrom.py` - Gray code in action (async FIFO)

6.7 Usage Examples

6.7.1 Example 1: 4-phase clock generator

```
counter_johnson #(
    .WIDTH(4)
) phase_gen (
    .clk(master_clk),
    .rst_n(rst_n),
    .enable(1'b1),
    .counter_gray(phases)
);

// Decode individual phases
assign phase_0 = (phases == 4'b0001);  // Step 1
assign phase_1 = (phases == 4'b0011);  // Step 2
assign phase_2 = (phases == 4'b0111);  // Step 3
assign phase_3 = (phases == 4'b1111);  // Step 4
```

6.7.2 Example 2: LED chaser

```
counter_johnson #(
    .WIDTH(8)
) led_chaser (
    .clk(slow_clk),        // ~1Hz clock
    .rst_n(rst_n),
    .enable(chase_enable),
    .counter_gray(led_pattern)
);

// LEDs show walking pattern
assign leds[7:0] = led_pattern;
```

6.7.3 Example 3: state machine controller

```
counter_johnson #(
    .WIDTH(3)
) state_ctrl (
    .clk(clk),
    .rst_n(rst_n),
    .enable(state_advance),
    .counter_gray(state_code)
);

// Decode states for control logic
always_comb begin
    case (state_code)
        3'b001: current_state = INIT;
        3'b011: current_state = LOAD;
```

```
        3'b111: current_state = PROCESS;
        3'b110: current_state = STORE;
        3'b100: current_state = CLEANUP;
        3'b000: current_state = IDLE;
        default: current_state = ERROR;
    endcase
end
```

6.7.4 Example 4: quadrature signal generation

```
// Generate quadrature encoder-like signals
counter_johnson #(
    .WIDTH(4)
) quad_gen (
    .clk(position_clk),
    .rst_n(rst_n),
    .enable(motor_moving),
    .counter_gray(quad_state)
);

// Extract A and B channels with 90° phase shift
assign quad_a = quad_state[0] ^ quad_state[1];
assign quad_b = quad_state[1] ^ quad_state[2];
```

6.7.5 Multi-phase clock for pipeline stages

```
// Multi-phase clock for pipeline stages
counter_johnson #(.WIDTH(4)) pipeline_phases (
    .clk(master_clk),
    .enable(pipeline_enable),
    .counter_gray(phase_vector)
);

// Generate phase-shifted clocks
assign clk_ph0 = master_clk & phase_vector[0];
assign clk_ph1 = master_clk & phase_vector[1];
assign clk_ph2 = master_clk & phase_vector[2];
assign clk_ph3 = master_clk & phase_vector[3];
```

6.7.6 Sequential control (memory refresh)

```
// Memory refresh controller
counter_johnson #(.WIDTH(3)) refresh_ctrl (
    .clk(clk),
    .enable(refresh_request),
    .counter_gray(refresh_phase)
);

// Control refresh operations
```

```
assign precharge = (refresh_phase == 3'b001);
assign activate  = (refresh_phase == 3'b011);
assign refresh   = (refresh_phase == 3'b111);
assign restore   = (refresh_phase == 3'b110);
```

6.7.7 Pattern generation for BIST

```
// Test pattern generator
counter_johnson #(.WIDTH(8)) pattern_gen (
    .clk(test_clk),
    .enable(pattern_enable),
    .counter_gray(test_pattern)
);

// Use for built-in self-test (BIST)
assign test_data = test_pattern;
```

6.8 Design Notes

What it does well. The logic is minimal – a shift register and an inverter – so states can be used directly with no decode logic and no complex timing requirements. It runs fast (400-600 MHz typical in modern FPGAs, limited by the shift-register timing), it's power-friendly because only one bit changes per cycle, and the progression is completely predictable. Any width works, every bit gives you a different phase, and the parameterization is trivial.

What it costs. Count efficiency: 2×WIDTH states versus 2^WIDTH for a binary counter of the same flops. The progression is a fixed sequence – no random access, you step through it. Decoding specific states takes comparators, the output isn't binary-compatible without `johnson2bin`, and you need care to avoid state conflicts when overlapping decoded phases. And the big one, from the theory section: it is **not** self-starting, so a clean reset is a hard requirement and glitch-free operation is only guaranteed inside the valid sequence.

Synthesis. Resources: WIDTH flip-flops, almost no LUTs (shift and invert only), and simple routing between adjacent stages. Keep the tool from swallowing the register into an SRL – the feedback path needs real flops:

```
(* SHREG_EXTRACT = "NO" *) logic [WIDTH-1:0] counter_gray; // Prevent
SRL inference
```

Dynamic power is low – one bit changes per cycle – and `enable` gives you free clock gating.

6.9 Related Modules

- `counter_ring`: different shift register pattern
- `counter_bingray`: binary and Gray code counter
- `counter`: simple binary counter
- Standard shift register implementations

6.10 Testing

6.10.1 Functional tests

1. **Sequence verification**: check the complete 2×WIDTH state cycle
2. **Reset behavior**: verify initialization to all zeros
3. **Enable control**: hold behavior when disabled
4. **Invalid-state containment**: force an invalid state and confirm it stays trapped in the parasitic cycle (this counter does NOT auto-recover), so the design's reset strategy is the only guarantee of a valid state

6.10.2 Coverage

```
covergroup johnson_cg @(posedge clk);
    cp_states: coverpoint counter_gray {
        // Valid Johnson counter states for WIDTH=4
        bins valid_states[] = {4'b0000, 4'b0001, 4'b0011, 4'b0111,
                               4'b1111, 4'b1110, 4'b1100, 4'b1000};
        bins invalid_states[] = default;
    }

    cp_transitions: coverpoint counter_gray {
        bins valid_trans[] = (4'b0000 => 4'b0001),
                             (4'b0001 => 4'b0011),
                             (4'b0011 => 4'b0111),
                             (4'b0111 => 4'b1111),
                             (4'b1111 => 4'b1110),
                             (4'b1110 => 4'b1100),
                             (4'b1100 => 4'b1000),
                             (4'b1000 => 4'b0000);
    }

    cp_enable: coverpoint enable {
        bins enabled = {1};
        bins disabled = {0};
    }
endcovergroup
```

6.10.3 Assertions

```
// Verify valid state progression
property johnson_sequence;
    @(posedge clk) disable iff (!rst_n)
    enable && (counter_gray == 4'b0000) |=>
    counter_gray == 4'b0001;
endproperty
```

```
assert property (johnson_sequence);

// Check for invalid states
property no_invalid_states;
    @(posedge clk) disable iff (!rst_n)
    counter_gray inside {4'b0000, 4'b0001, 4'b0011, 4'b0111,
                         4'b1111, 4'b1110, 4'b1100, 4'b1000};
endproperty

assert property (no_invalid_states);
```

6.10.4 Test files

- `val/cdc/test_counter_johnson.py` – full functional verification
- `val/cdc/test_counter_johnson_wavedrom.py` – WaveDrom timing diagrams

```
# Full functional test (basic/medium/full levels)
pytest val/cdc/test_counter_johnson.py -v

# WaveDrom waveform generation
pytest val/cdc/test_counter_johnson_wavedrom.py -v
```

6.11 References

- Johnson, R.R. “Electronic Counter.” U.S. Patent 3,030,581, issued Apr. 1962 (continuation of a 1953 application). Google Patents – provenance of the Johnson-counter name only; the patent’s worked example is not the 2N twisted ring, which is treated as a standard technique.
- Roy, S. *Advanced Digital System Design*, Springer, 2024, ch. 4 “Basic Sequential Circuits”, sec. 4.9 “Shift Register Counters” – textbook treatment of the Johnson (twisted-ring) counter (secondary source).
- Indexed in `rtl/cdc/references/README.md` (no paper archived for this technique; marked standard technique)

6.12 Navigation

- ← Back to CDC Index
- ← Back to Main Documentation Index

7 Asynchronous FIFO (`fifo_async.sv`)

7.1 Overview

An asynchronous FIFO for crossing data safely between different clock domains. **Restricted to power-of-2 depths only** – that restriction comes from the Gray code pointer implementation, and `USE_JOHNSON=1` is the supported way around it.

7.2 Parameters

fifo_async parameters

Parameter	Default	Description
MEM_STYLE	—	Memory implementation (FIFO_AUTO/SRL/BRAM). The BRAM branch registers the read path (registered read even when REGISTERED=0).
REGISTERED	—	0 = mux mode, 1 = flop mode for read output
DATA_WIDTH	8	Width of data
DEPTH	16	FIFO depth. Power-of-2 with the default binary Gray pointers; set **USE_JOHNSON=1** for **non-power-of-2** depths (Johnson-coded pointers via counter_johnson/johnson2bin).
USE_JOHNSON	0	0 = Gray pointers (power-of-2 depth), 1 = Johnson pointers (arbitrary depth). This is the supported route to non-power-of-2 depth (the old fifo_async_div2 module was retired).
N_FLOP_CROSS	2	Number of synchronizer stages
ALMOST_WR_MARGIN	1	Almost full threshold
ALMOST_RD_MARGIN	1	Almost empty threshold

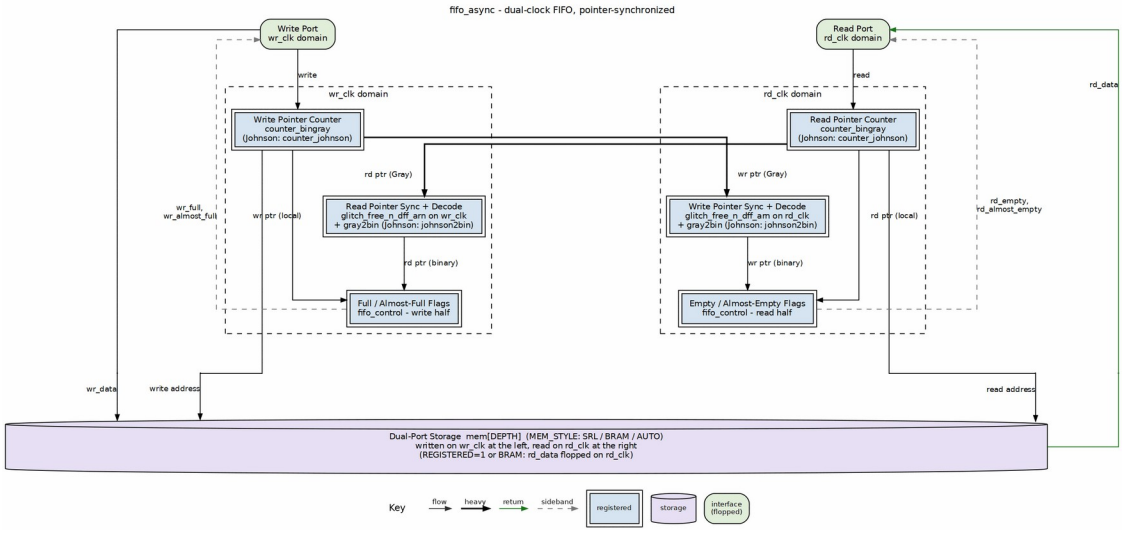
7.3 Ports

fifo_async ports

Port	Direction	Width	Description
wr_clk	input	1	Write domain clock
wr_rst_n	input	1	Write domain active-low reset
write	input	1	Write enable signal
wr_data	input	DATA_WIDTH	Data to write
wr_full	output	1	Write domain full flag
wr_almost_full	output	1	Write domain almost full flag
rd_clk	input	1	Read domain clock
rd_rst_n	input	1	Read domain active-low reset
read	input	1	Read enable signal
rd_data	output	DATA_WIDTH	Data read from FIFO
rd_empty	output	1	Read domain empty flag
rd_almost_empty	output	1	Read domain almost empty flag

7.4 Functional Description

7.4.1 Block diagram



fifo_async block diagram

Write data drops down the left into the dual-port storage on `wr_clk`; read data comes back up the right on `rd_clk`. The storage array is the one structure both clocks touch. The only signals that cross between the two domains are the two pointers, Gray coded (Johnson coded when `USE_JOHNSON=1`). Each one is synchronized in the destination domain by an N-flop `glitch_free_n_dff_arn` and decoded back to binary. Each flag block compares its own domain's local pointer against the synchronized remote one: full on the write side, empty on the read side.

7.4.2 Clock domain crossing strategy

The FIFO uses **Gray code pointers** for safe clock domain crossing.

7.4.3 Core components

1. **Binary-Gray counters** (`counter_bingray`) for pointer generation
2. **Multi-stage synchronizers** (`glitch_free_n_dff_arn`) for CDC
3. **Gray-to-binary converters** (`gray2bin`) for pointer comparison
4. **Shared memory array** accessible from both domains
5. **FIFO control logic** for status flag generation

7.4.4 Gray code pointer system

Gray codes ensure **only one bit changes** per increment:

- **Binary**: 011 → 100 (3 bits change simultaneously)
- **Gray**: 010 → 110 (only 1 bit changes)
- **Benefit**: eliminates metastability from multi-bit transitions

```
// Write domain Gray counter
counter_bingray #(.WIDTH(AW + 1)) wr_ptr_counter_gray (
    .clk(wr_clk),
    .rst_n(wr_rst_n),
    .enable(write && !wr_full),
    .counter_bin(r_wr_ptr_bin),          // Binary for addressing
    .counter_bin_next(w_wr_ptr_bin_next), // Next binary value
    .counter_gray(r_wr_ptr_gray)         // Gray for CDC
);
```

Pointer width: `AW = $clog2(DEPTH)` address bits, plus one extra bit for wrap detection – so DEPTH=16 gives AW=4 and a 5-bit pointer.

7.4.5 Clock domain crossing

```
// Cross read pointer to write domain
glitch_free_n_dff_arn #(
    .FLOP_COUNT(N_FLOP_CROSS),
    .WIDTH(AW + 1)
) rd_ptr_gray_cross_inst (
    .q(r_wdom_rd_ptr_gray),    // Synchronized output
    .d(r_rd_ptr_gray),         // Gray pointer input
    .clk(wr_clk),              // Destination clock
    .rst_n(wr_rst_n)           // Destination reset
);
```

The default is 2 flip-flops per crossing (`N_FLOP_CROSS=2`). Each additional stage improves MTBF at the cost of latency – the classic trade.

7.4.6 Memory organization

```
logic [DW-1:0] mem[DEPTH];  // Memory array -- sized by DEPTH, not by
                            // 1<<AW: with USE_JOHNSON=1 and a
                            // non-power-of-2 DEPTH those differ


// Write port (write domain)
always_ff @(posedge wr_clk) begin
    if (write && !wr_full) begin  // !wr_full guard REQUIRED (prevents
overwrite)
        mem[r_wr_addr] <= wr_data;
    end
end

// Read port (read domain) -- dual mode, like fifo_sync:
if (REGISTERED != 0) begin : g_flop
```

```
    always_ff ... r_rd_data <= mem[r_rd_addr];
    assign rd_data = r_rd_data;
end else begin : g_mux
    assign rd_data = mem[r_rd_addr];
end
```

Addresses come from the binary pointers, truncated to the low bits: `r_wr_addr = r_wr_ptr_bin[AW-1:0]` and `r_rd_addr = r_rd_ptr_bin[AW-1:0]`.

7.4.7 Full/empty detection

```
// In write domain (fifo_control): computed combinationally,
REGISTERED out
assign w_wr_full_d = (w_wdom_ptr_xor &&
                      (wr_ptr_bin[AW-1:0] == wdom_rd_ptr_bin[AW-1:0]));

always_ff @(posedge wr_clk, negedge wr_rst_n) begin
    if (!wr_rst_n) wr_full <= 'b0;
    else           wr_full <= w_wr_full_d;
end

// Where:
assign w_wdom_ptr_xor = wr_ptr_bin[AW] ^ wdom_rd_ptr_bin[AW];

// In read domain: same shape, and rd_empty RESETS TO 1 (empty out of
reset)
assign w_rd_empty_d = (!w_rdom_ptr_xor_for_empty &&
                       (rd_ptr_bin[AW:0] == w_wr_ptr_for_empty[AW:0]));

always_ff @(posedge rd_clk, negedge rd_rst_n) begin
    if (!rd_rst_n) rd_empty <= 'b1;
    else           rd_empty <= w_rd_empty_d;
end
```

Both flags are registered, so each lags its pointer comparison by one cycle of its own clock – conservative in the safe direction (a write that just made the FIFO non-empty shows up at the reader one `rd_clk` later).

The algorithm in words: **full** is MSBs differ AND LSBs equal (the write pointer has wrapped and caught the read pointer); **empty** is all bits equal (pointers at the same location). The MSB is the wrap bit – it records which pointer has lapped.

7.4.8 The power-of-2 requirement

Why power-of-2 only, with Gray pointers:

1. **Gray code properties**: natural binary-Gray relationship
2. **Wraparound behavior**: clean modulo-2^n arithmetic

3. **Address truncation**: simple bit slicing for memory addressing
4. **Pointer comparison**: efficient full/empty detection

```
// Valid depths: 2, 4, 8, 16, 32, 64, 128, 256, ...
// Invalid depths: 3, 5, 6, 7, 9, 10, 12, 15, ...
```

7.5 Timing Characteristics

- **Pointer propagation**: 2-3 clock cycles (depending on `N_FLOP_CROSS`)
- **Status flag delay**: flags reflect state with synchronizer latency
- **Conservative design**: prevents overflow/underflow despite the delay

Metastability protection comes from the usual pair: Gray code transitions (single bit changes only) and multi-stage synchronization, which reduces metastability failure probability exponentially (raises MTBF). Proper timing constraints on the crossing paths are essential – see the SDC section of the CDC reference.

Throughput and latency at a glance:

- **Throughput**: up to 1 operation per clock per domain
- **Latency**: 0-1 cycles (depending on REGISTERED mode)
- **CDC latency**: 2-3 cycles for status propagation
- **Efficiency**: ~100% utilization possible

7.6 Waveforms

WaveDrom timing diagrams of the Gray code CDC mechanism are available.

Run the WaveDrom test to generate detailed timing diagrams:

```
# Generate Gray code waveforms (standard async FIFO implementation)
pytest val/cdc/test_fifo_async_wavedrom.py -v
```

Waveform Scenarios Generated:

1. **Write-Fill-Read-Empty Cycle**
   - Standard async FIFO operation
   - Gray code pointer progression
   - Power-of-2 depth utilization
2. **Gray Code Synchronization**
   - Efficient Gray code CDC
   - Logarithmic pointer width (vs. linear for Johnson)
   - Cross-domain pointer transitions
3. **Power-of-2 Depth Utilization**
   - Efficient addressing with power-of-2 depths
   - Full depth wraparound behavior
   - Resource-efficient pointer management

Key Differences from Johnson Counter Variant:

- **Pointer Width**: Logarithmic (`$clog2(DEPTH) + 1`) vs. linear (`DEPTH`) for Johnson
- **Depth Restriction**: Power-of-2 only vs. **any** depth for Johnson – odd depths included. The elaboration check only fires for `USE_JOHNSON == 0` (`if ((USE_JOHNSON == 0) && ((DEPTH & (DEPTH - 1)) != 0)) $error(...)`), so Johnson mode has no restriction at all. (“Even only” is stale language from the retired `fifo_async_div2`.)
- **Resource Efficiency**: Better for large depths (>32) vs. Johnson’s flexibility for small depths

Comparison Tests:

- `test_fifo_sync_wavedrom.py` - Synchronous FIFO (single clock, no CDC)

7.7 Usage Examples

Typical applications:

- **Video processing**: different pixel and memory clock domains
- **Networking**: packet buffers between different rate domains
- **Audio systems**: sample rate conversion buffers
- **Microprocessor interfaces**: CPU and peripheral clock domains

When to use what:

- **Async FIFO** when the clock domains are truly independent
- **Sync FIFO** when a single clock domain is sufficient
- **Non-power-of-2 depth**: use `fifo_async #(.USE_JOHNSON(1))` – the standalone `fifo_async_div2` module was retired

7.8 Design Notes

7.8.1 Depth sizing

```
// Calculate minimum depth for burst handling
// DEPTH ≥ burst_size + synchronizer_latency + margin
parameter int MIN_DEPTH = 16;  // Typical minimum for async FIFOs
```

7.8.2 Clock relationship

- **Asynchronous clocks**: no phase relationship assumed
- **Clock gating**: avoid gating clocks used by the FIFO
- **Reset deassertion**: ensure proper reset release sequencing

7.8.3 Almost full/empty settings

- **Almost full**: `DEPTH - ALMOST_WR_MARGIN`
- **Almost empty**: `ALMOST_RD_MARGIN`
- **Guideline**: set margins > synchronizer latency

7.8.4 Error checking

The current RTL has **no** runtime `$display` overflow/underflow checks. The `!wr_full` write guard is the only overflow protection. The one elaboration-time check that does exist is an `$error` for a non-power-of-2 `DEPTH` when Gray pointers are selected (`USE_JOHNSON=0`). Add assertions in your testbench if you need overflow telemetry.

7.9 Related Modules

- **USE_JOHNSON=1**: for non-power-of-2 depths, using Johnson counters (replaces the retired fifo_async_div2)
- **fifo_sync**: for single clock domain applications
- **counter_bingray**: binary-Gray counter implementation
- **glitch_free_n_dff_arn**: multi-stage synchronizer

7.10 Testing

- `val/cdc/test_fifo_buffer_async.py` – full functional verification
- `val/cdc/test_fifo_async_wavedrom.py` – WaveDrom timing diagrams

```
# Full functional test (basic/medium/full levels)
pytest val/cdc/test_fifo_buffer_async.py -v

# WaveDrom waveform generation
pytest val/cdc/test_fifo_async_wavedrom.py -v
```

7.11 References

- Cummings, C.E. “Simulation and Synthesis Techniques for Asynchronous FIFO Design.” *SNUG San Jose*, 2002 – the Gray-pointer async FIFO structure (`USE_JOHNSON=0`): (AW+1)-bit pointers, synchronized into the opposite domain before full/empty.
- Ginosar, R. “Metastability and Synchronizers: A Tutorial.” *IEEE Design & Test of Computers*, 28(5), Sept./Oct. 2011. DOI: 10.1109/MDT.2011.113 – peer-reviewed treatment of the two-clock FIFO synchronizer and of synchronizer depth (`N_FLOP_CROSS`) versus MTBF.
- Gray, F. “Pulse Code Communication.” U.S. Patent 2,632,058, filed Nov. 1947, issued Mar. 1953. Google Patents – the Gray code itself; `USE_JOHNSON=1` substitutes a twisted-ring code, a repo choice with no dedicated paper.
- Deschamps, J.-P., Sutter, G.D., Cantó, E. *Guide to FPGA Implementation of Arithmetic Functions*, Springer (LNEE 149), 2012, ch. 5 “Electronic Aspects of Digital Design” – sec. 5.3.5.3 “Asynchronous FIFO” and sec. 5.2.3 metastability/MTBF – textbook treatment (secondary source).
- Indexed in `rtl/cdc/references/README.md` (local PDF: `AsyncFifoGrayPointers__Cummings2002.pdf`, `MetastabilitySynchronizers__Ginosar2011.pdf`, `GrayCode__Gray1953.pdf`)

7.12 Navigation

- ← Back to CDC Index
- ← Back to Main Documentation Index

8 GAXI asynchronous FIFO (gaxi_fifo_async.sv)

8.1 Overview

gaxi_fifo_async (in rtl/cdc/, production ready) moves GAXI data safely between independent clock domains. Gray code pointers and multi-flop synchronizers keep metastability out of the datapath and get your data across intact.

What you get:

- **Clock domain crossing:** safe transfer between independent clocks
- **Gray code pointers:** no multi-bit synchronization hazards
- **Configurable CDC stages:** 2-4 flop synchronizers (3 recommended; 4 for ultra-critical systems)
- **Arbitrary clock ratios:** works with any write:read clock ratio
- **Two read modes:** mux or flop mode

```
module gaxi_fifo_async #(
    parameter fifo_mem_t MEM_STYLE        = FIFO_AUTO,  // FIFO_AUTO / FIFO_SRL / FIFO_BRAM
    parameter int        REGISTERED       = 0,          // 0 = mux mode, 1 = flop mode
    parameter int        DATA_WIDTH       = 8,
    parameter int        DEPTH            = 16,
    parameter int        USE_JOHNSON      = 0,          // 0 = Gray (power-of-2 depth), 1 = Johnson (any depth)
    parameter int        N_FLOP_CROSS     = 2,
    parameter int        ALMOST_WR_MARGIN = 1,
    parameter int        ALMOST_RD_MARGIN = 1
) (
    // Write Domain
    input  logic            axi_wr_aclk,
    input  logic            axi_wr_aresetn,
    input  logic            wr_valid,
    output logic            wr_ready,    // not full
    input  logic [DW-1:0]   wr_data,

    // Read Domain
    input  logic            axi_rd_aclk,
    input  logic            axi_rd_aresetn,
    input  logic            rd_ready,
    output logic            rd_valid,    // not empty
    output logic [DW-1:0]   rd_data
);
```

8.2 Parameters

gaxi_fifo_async parameters

Parameter	Default	Description
MEM_STYLE	FIFO_AUTO	Memory implementation: FIFO_AUTO / FIFO_SRL / FIFO_BRAM. The BRAM branch registers the read path – a registered read even when REGISTERED=0
REGISTERED	0	0=mux mode, 1=flop mode (read path)
DATA_WIDTH	8	Data bus width
DEPTH	16	FIFO depth. Power of 2 with Gray pointers; **any** depth – odd included – with USE_JOHNSON=1
USE_JOHNSON	0	Pointer CDC encoding: 0 = Gray (log2(DEPTH)+1 bits, power-of-2 depth only), 1 = Johnson (DEPTH bits, **any** depth – odd included). An illegal combination fails at elaboration with an explicit $error.
N_FLOP_CROSS	2	Synchronizer stages (3 recommended for safety)
ALMOST_WR_MARGIN	1	Almost full threshold
ALMOST_RD_MARGIN	1	Almost empty threshold

Important: Set N_FLOP_CROSS=3 for production designs to ensure metastability protection.

The RTL also declares five derived parameters after these – DW, D, AW, JCW, N. They are aliases (DW = DATA_WIDTH, AW = $clog2(DEPTH), JCW = D, N = N_FLOP_CROSS), not independent knobs. SystemVerilog lets you override them, but doing so decouples a port width from the storage that backs it: override DW alone and the ports widen while the memory stays narrow, and the FIFO silently drops the upper bits of every entry. Set DATA_WIDTH, not DW.

8.3 Ports

gaxi_fifo_async ports

Port	Direction	Width	Description
axi_wr_aclk	input	1	Write domain clock
axi_wr_aresetn	input	1	Write domain active-low reset
wr_valid	input	1	Write valid
wr_ready	output	1	Write ready (not full)
wr_data	input	DW	Write data
axi_rd_aclk	input	1	Read domain clock
axi_rd_aresetn	input	1	Read domain active-low reset
rd_ready	input	1	Read ready
rd_valid	output	1	Read valid (not empty)
rd_data	output	DW	Read data

8.4 Functional Description

8.4.1 Why Gray code?

Gray code ensures only one bit changes at a time during pointer updates:

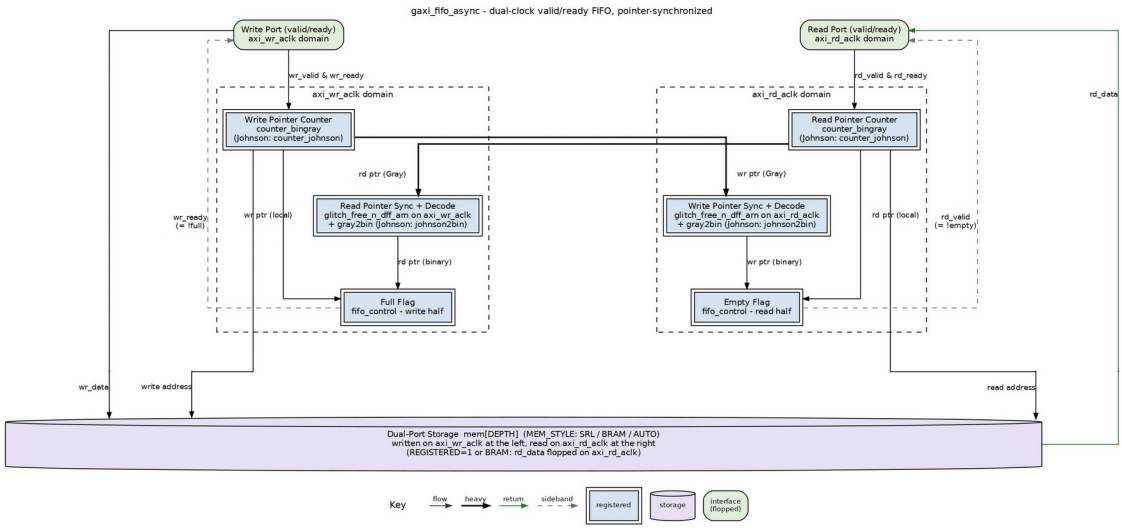
```
Binary: 011 → 100  (3 bits change - hazard!)
Gray:   010 → 110  (1 bit changes - safe!)
```

This prevents glitches during synchronization across clock domains.

8.4.2 Dependencies

- `counter_bin.sv` - Binary counters
- `counter_johnson.sv` - Johnson (twisted-ring) counters, used when `USE_JOHNSON=1`. Johnson code is NOT Gray code – it is a distinct encoding that happens to share the single-bit-change property needed for CDC. Both change exactly one bit per increment, including the wrap.
- `glitch_free_n_dff_arn.sv` - Multi-flop synchronizers
- `johnson2bin.sv` - Johnson-to-binary conversion (**combinational**; its `clk`/`rst_n` ports are declared but unused), used when `USE_JOHNSON=1`
- `counter_bingray.sv` / `gray2bin.sv` - Gray pointer path (combinational decode), used when `USE_JOHNSON=0`
- `fifo_control.sv` - Full/empty flag generation

8.4.3 Block diagram



gaxi_fifo_async block diagram

Write data drops down the left into the dual-port storage on `axi_wr_aclk`; read data comes back up the right on `axi_rd_aclk`. The only signals that cross between the domains are the two pointers, Gray coded (Johnson coded when `USE_JOHNSON=1`), each synchronized by `glitch_free_n_dff_arn` in the destination domain and decoded to binary. The write half of `fifo_control` compares the local write pointer with the synchronized read pointer to drive `wr_ready` (not full); the read half does the mirror image to drive `rd_valid` (not empty).

Each domain keeps its own binary pointer for arithmetic, converts it to Gray for the crossing, and decodes the synchronized remote pointer for flag generation.

8.5 Timing Characteristics

gaxi_fifo_async latency by clock ratio

Clock Ratio (wr:rd)	Latency	Notes
1:1 (same freq)	3-5 cycles	CDC synchronization overhead
2:1 (fast→slow)	4-6 cycles	Additional write-side delay
1:2 (slow→fast)	3-5 cycles	Read-side samples faster
Any ratio	3-7 cycles	Depends on synchronizer stages + clock relationship

Latency formula: `~(2 × N_FLOP_CROSS) + 1` in slower clock domain cycles.

8.6 Usage Examples

8.6.1 Example 1: basic CDC FIFO

```
gaxi_fifo_async #(
    .DATA_WIDTH(32),
    .DEPTH(16),
    .N_FLOP_CROSS(3),      // 3-stage synchronizer
    .REGISTERED(0)
) u_cdc_fifo (
    // Write domain @ 100 MHz
    .axi_wr_aclk    (clk_100mhz),
    .axi_wr_aresetn (rst_100_n),
    .wr_valid       (domain_a_valid),
    .wr_ready       (domain_a_ready),
    .wr_data        (domain_a_data),

    // Read domain @ 50 MHz
    .axi_rd_aclk    (clk_50mhz),
    .axi_rd_aresetn (rst_50_n),
    .rd_ready       (domain_b_ready),
    .rd_valid       (domain_b_valid),
    .rd_data        (domain_b_data)
);
```

8.6.2 Example 2: high-speed to low-speed CDC

```
// Fast writer (250 MHz) → Slow reader (62.5 MHz)
// Needs deeper FIFO to handle burst traffic
gaxi_fifo_async #(
    .DATA_WIDTH(64),
    .DEPTH(32),           // Deeper for rate mismatch
    .N_FLOP_CROSS(3),
    .REGISTERED(1)        // Flop mode for timing
) u_rate_converter (
    .axi_wr_aclk    (clk_250mhz),
    .axi_wr_aresetn (rst_fast_n),
    .wr_valid       (fast_valid),
    .wr_ready       (fast_ready),
    .wr_data        (fast_data),

    .axi_rd_aclk    (clk_62p5mhz),
    .axi_rd_aresetn (rst_slow_n),
    .rd_ready       (slow_ready),
    .rd_valid       (slow_valid),
    .rd_data        (slow_data)
);
```

8.7 Design Notes

8.7.1 Depth sizing for clock ratio

When write clock >> read clock, size the FIFO to handle burst accumulation:

```
Required Depth >= Burst Size x (1 - Read Freq / Write Freq) x Safety
Margin


Example:
- Write: 100 MHz, Read: 25 MHz
- Burst: 16 transfers
- Safety: 1.5x
→ Depth >= 16 x (1 - 25/100) x 1.5 = 18 entries
```

8.7.2 Reset synchronization

Critical: both clock domains must have properly synchronized resets!

```
// Separate reset synchronizers for each domain
reset_sync u_wr_rst_sync (
    .clk(axi_wr_aclk),
    .rst_n(global_rst_n),
    .sync_rst_n(axi_wr_aresetn)
);

reset_sync u_rd_rst_sync (
    .clk(axi_rd_aclk),
    .rst_n(global_rst_n),
    .sync_rst_n(axi_rd_aresetn)
);
```

8.7.3 Metastability protection

`N_FLOP_CROSS` sets the synchronizer depth, and MTBF rises steeply with it.

Synchronizer stage guidance

Stages	Use Case
2	Short-term prototyping only
3	**Production standard**
4	Ultra-critical systems

This page deliberately gives no MTBF figures. Real MTBF depends on the flop's metastability time constant and resolution window, the two clock frequencies and the data toggle rate – none of which a module page can know. A second table here also drifted from the one in

glitch_free_n_dff_arn, which put 2 stages at “hours” where this page said “years”. One source per fact: that page owns the discussion.

Recommendation: always use `N_FLOP_CROSS=3` in production.

8.7.4 Error checking

The RTL contains **no** assertions and no runtime `$display` checks. What is there is a single empty read-domain block:

```
always_ff @(posedge axi_rd_aclk) begin
    if (w_read && r_rd_empty) begin end   // empty -- reports nothing
end
```

There is no write-domain equivalent. Overflow is prevented structurally by the `!wr_full` write guard rather than reported; underflow is not detected at all. If you need overflow/underflow telemetry in simulation, add it in the testbench. `fifo_async.md` states the same for its module.

8.7.5 Common mistakes

1. Metastability. Symptom: random data corruption, simulation/hardware mismatch. Fix: increase `N_FLOP_CROSS` to 3 or 4.

2. Pointer synchronization failure. Symptom: FIFO full/empty signals incorrect. Debug:

1. Verify the clocks are truly independent (no PLL relationship violations)
2. Check reset synchronization – both domains must reset properly
3. Verify the Gray code conversion logic

3. Underflow/overflow. Symptom: data loss or corruption. Debug:

1. Check `DEPTH` is sufficient for the clock ratio
2. Verify flow control is respected in both domains
3. Monitor pointer values in both domains

8.8 Related Modules

- gaxi_fifo_sync - Single clock domain version
- gaxi_skid_buffer_async - Async skid buffer
- GAXI Index - Overview

8.9 Testing

```
# Async FIFO tests with various clock ratios
pytest val/cdc/test_gaxi_buffer_async.py -v

# Test specific clock ratio (wr:rd)
pytest val/cdc/test_gaxi_buffer_async.py -k "wr10_rd12" -v  # 10ns :
12ns
```

The test matrix covers:

- Same clocks (1:1)
- 1.2x ratio (10ns : 12ns)
- 2x ratio (10ns : 20ns)
- 2.5x ratio (8ns : 20ns)

8.10 References

- Cummings, C.E. “Simulation and Synthesis Techniques for Asynchronous FIFO Design.” *SNUG San Jose*, 2002 – the Gray-pointer async FIFO structure used at `USE_JOHNSON=0`: (AW+1)-bit pointers synchronized into the opposite domain before full/empty.
- Ginosar, R. “Metastability and Synchronizers: A Tutorial.” *IEEE Design & Test of Computers*, 28(5), Sept./Oct. 2011. DOI: 10.1109/MDT.2011.113 – peer-reviewed treatment of the two-clock FIFO synchronizer and of `N_FLOP_CROSS` depth versus MTBF.
- Gray, F. “Pulse Code Communication.” U.S. Patent 2,632,058, filed Nov. 1947, issued Mar. 1953. Google Patents – the Gray code; `USE_JOHNSON=1` substitutes a twisted-ring code, a repo choice with no dedicated paper.
- Deschamps, J.-P., Sutter, G.D., Cantó, E. *Guide to FPGA Implementation of Arithmetic Functions*, Springer (LNEE 149), 2012, ch. 5 “Electronic Aspects of Digital Design” – sec. 5.3.5.3 “Asynchronous FIFO” – textbook treatment (secondary source).
- Indexed in `rtl/cdc/references/README.md` (local PDF: `AsyncFifoGrayPointers__Cummings2002.pdf`, `MetastabilitySynchronizers__Ginosar2011.pdf`, `GrayCode__Gray1953.pdf`)
- **Source:** `rtl/cdc/gaxi_fifo_async.sv`
- **Tests:** `val/cdc/test_gaxi_buffer_async.py`

8.11 Navigation

- ← Back to CDC Index
- ← Back to Main Documentation Index

Version: 1.0 **Last Updated:** 2025-10-06

9 gaxi_skid_buffer_async

Module: `gaxi_skid_buffer_async.sv` **Location:** `rtl/cdc/` **Status:** Production Ready

9.1 Overview

This module is a wrapper, and it owns that: a synchronous skid buffer on the write side, an asynchronous FIFO behind it. One instantiation buys you the skid buffer's elastic buffering plus a clock domain crossing. One thing to know before you build around it — the skid buffer registers its handshake outputs, so there is no zero-latency bypass path; see Latency Components under Timing below.

9.1.1 Key Features

- **CDC Capability:** Safe clock domain crossing
- **Elastic Buffering:** Combines skid buffer + async FIFO
- **Write-Side Buffering:** Skid buffer in write domain prevents stalls
- **Configurable CDC:** Adjustable synchronizer stages
- **Dual Clock Domains:** Independent write/read clocks

9.2 Parameters

Parameter	Type	Default	Description
MEM_STYLE	fifo_mem_t	FIFO_AUTO	Async FIFO storage: FIFO_AUTO / FIFO_SRL / FIFO_BRAM
REGISTERED	int	0	Async FIFO read mode (0=mux, 1=flop)
DATA_WIDTH	int	32	Data bus width
DEPTH	int	2	Async FIFO depth (CDC stage)
USE_JOHNSON	int	0	0 = Gray pointers, power-of-2 DEPTH only; 1 = Johnson pointers, any DEPTH
N_FLOP_CROSS	int	2	Synchronizer stages (3 recommended)

USE_JOHNSON reaches the async FIFO underneath and carries the same meaning it has there: it selects the pointer encoding, and with it the legal DEPTH values. Leave it at 0 and DEPTH must be a power of 2 or elaboration fails; set it to 1 and odd depths become legal at the cost of a wider pointer. See gaxi_fifo_async for the trade-off in full.

Note: the skid half's depth is NOT DEPTH. The wrapper instantiates gaxi_skid_buffer with DATA_WIDTH only, so it takes that module's own default of 2. DEPTH sizes the async FIFO behind it.

9.3 Ports

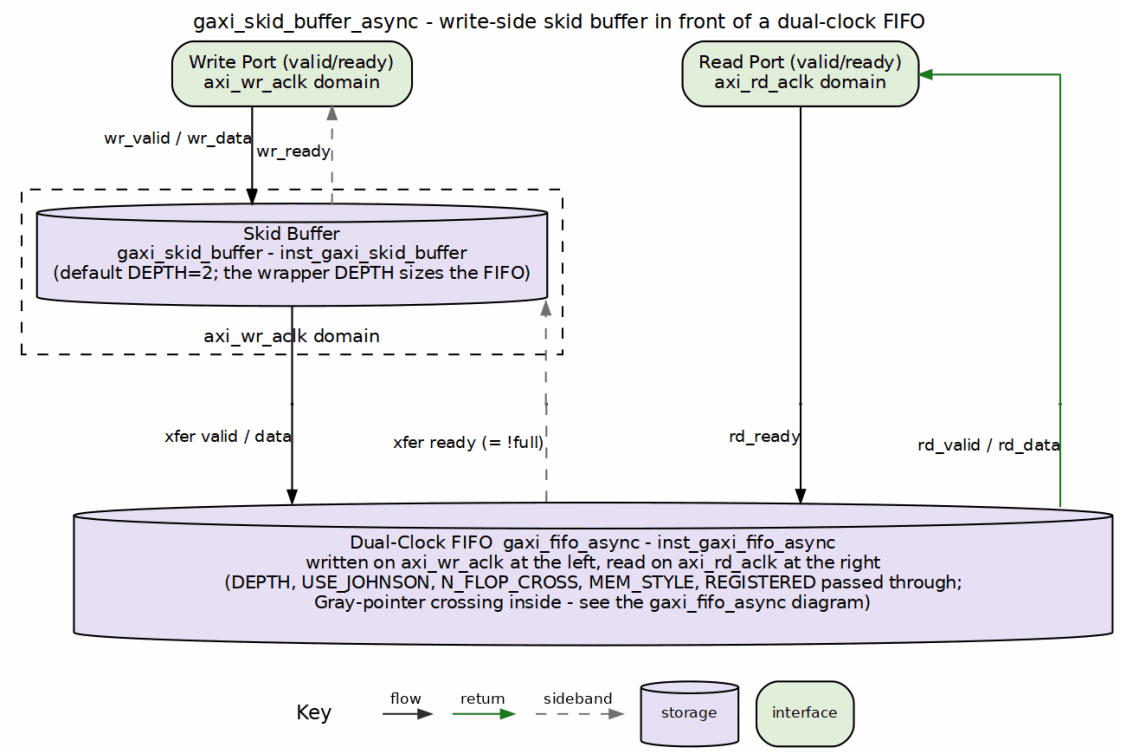
The port list splits cleanly into a write domain and a read domain:

```
module gaxi_skid_buffer_async #(
    parameter fifo_mem_t MEM_STYLE   = FIFO_AUTO,  // FIFO_AUTO /
FIFO_SRL / FIFO_BRAM
    parameter int        REGISTERED  = 0,          // 0 = mux mode, 1
= flop mode
    parameter int        DATA_WIDTH  = 32,
    parameter int        DEPTH       = 2,          // async FIFO depth
    parameter int        USE_JOHNSON = 0,          // 0 = Gray (power-
of-2 depth), 1 = Johnson (any depth)
    parameter int        N_FLOP_CROSS = 2,         // CDC synchronizer
stages
    parameter int        DW = DATA_WIDTH           // port-width alias
used below
) (
    // Write Domain
    input  logic          axi_wr_aclk,
    input  logic          axi_wr_aresetn,
    input  logic          wr_valid,
    output logic          wr_ready,
    input  logic [DW-1:0] wr_data,

    // Read Domain
    input  logic          axi_rd_aclk,
    input  logic          axi_rd_aresetn,
    output logic          rd_valid,
    input  logic          rd_ready,
    output logic [DW-1:0] rd_data
);
```

9.4 Functional Description

9.4.1 Block diagram



gaxi_skid_buffer_async block diagram

Write traffic enters a `gaxi_skid_buffer` on `axi_wr_aclk`, which registers the write handshake and returns `wr_ready` to the source. The skid buffer feeds the write side of a `gaxi_fifo_async`, whose not-full flag is the skid buffer's downstream ready. Read data leaves the FIFO on `axi_rd_aclk` and climbs back up to the read port, with `rd_ready` dropping in from that port. The only clock-domain crossing is inside `gaxi_fifo_async`; see its page for the pointer synchronization.

9.4.2 Internal Components

1. **gaxi_skid_buffer (Write Domain):**
   - Provides elastic buffering
   - Prevents write-side stalls
   - Fixed depth (typically 2-4 entries)
2. **gaxi_fifo_async (CDC):**
   - Handles clock domain crossing
   - Configurable depth (DEPTH parameter)
   - Gray code pointer synchronization

9.5 Timing Characteristics

9.5.1 Latency Components

Component	Latency	Domain
Skid buffer (empty)	1 cycle	Write
Skid buffer (buffered)	1 cycle	Write
Async FIFO CDC	3-5 cycles	Both
Total (empty path)	**4-6 cycles**	End-to-end
Total (buffered)	**4-6 cycles**	End-to-end

`gaxi_skid_buffer` registers its handshake outputs, so even a write into an empty buffer costs one write clock before `rd_valid` rises and the async FIFO can take the data. There is no zero-latency bypass path.

9.5.2 Throughput

- **Write Side:** 1 transfer/cycle (when not full)
- **Read Side:** 1 transfer/cycle (when not empty)
- **Sustained:** Limited by slower clock domain

9.6 Usage Examples

9.6.1 Example 1: Basic Async Skid Buffer

```
gaxi_skid_buffer_async #(
    .DATA_WIDTH(64),
    .DEPTH(4),            // Async FIFO depth
    .N_FLOP_CROSS(3),     // CDC stages
    .REGISTERED(0)
) u_async_skid (
    // Write domain @ 100 MHz
    .axi_wr_aclk    (clk_100),
    .axi_wr_aresetn (rst_100_n),
    .wr_valid       (src_valid),
    .wr_ready       (src_ready),
    .wr_data        (src_data),

    // Read domain @ 75 MHz
    .axi_rd_aclk    (clk_75),
    .axi_rd_aresetn (rst_75_n),
    .rd_valid       (dst_valid),
    .rd_ready       (dst_ready),
    .rd_data        (dst_data)
);
```

9.6.2 Example 2: Pipeline Stage with CDC

```
// Break long path AND cross clock domains
gaxi_skid_buffer_async #(
    .DATA_WIDTH(128),
    .DEPTH(8),            // Larger for rate buffering
    .N_FLOP_CROSS(3),
    .REGISTERED(1)        // Registered output for timing
) u_cdc_pipeline (
    .axi_wr_aclk    (source_clk),
    .axi_wr_aresetn (source_rst_n),
    .wr_valid       (stage_in_valid),
    .wr_ready       (stage_in_ready),
    .wr_data        (stage_in_data),

    .axi_rd_aclk    (dest_clk),
    .axi_rd_aresetn (dest_rst_n),
    .rd_valid       (stage_out_valid),
    .rd_ready       (stage_out_ready),
    .rd_data        (stage_out_data)
);
```

9.7 Design Notes

9.7.1 When to Use This Module

Use async skid buffer when: - Need CDC between clock domains - Want elastic buffering on write side - Write-side needs low-latency response

Use plain async FIFO instead when: - Only need CDC, no special write-side buffering - Want deeper buffering (larger DEPTH) - Don't need skid buffer complexity

9.7.2 Depth Selection

Total buffering = Skid buffer entries + DEPTH

Recommendations: - **DEPTH=4:** Minimal CDC buffering, light traffic - **DEPTH=8:** Moderate buffering, balanced - **DEPTH=16:** Heavy buffering, burst traffic

9.7.3 Reset Synchronization

Critical: Both domains need synchronized resets!

```
// Example reset synchronization
reset_sync #(.N(3)) u_wr_sync (
    .clk(axi_wr_aclk),
    .rst_n(global_rst_n),
    .sync_rst_n(axi_wr_aresetn)
);

reset_sync #(.N(3)) u_rd_sync (
    .clk(axi_rd_aclk),
    .rst_n(global_rst_n),
    .sync_rst_n(axi_rd_aresetn)
);
```

9.7.4 Resource Utilization

Combines resources of both sub-modules:

Component	Flops	LUTs
Skid buffer (fixed, DEPTH=2)	2×DW + ~4	~50
Async FIFO (DEPTH=8)	8×DW + ~40	~120
Total (DEPTH=8)	**~10×DW + 44**	**~170**

The skid row does not scale with `DEPTH`. The wrapper never overrides the skid buffer's depth, so its storage is `r_data[2]` — two words — whatever `DEPTH` you pass. Only the FIFO row moves.

9.7.5 Common Issues

Issue 1: CDC Violations

Symptom: metastability, data corruption.

Solution: - Set `N_FLOP_CROSS=3` (minimum for production) - Verify clocks are truly independent - Check reset synchronization

Issue 2: Write-Side Stalls

Symptom: `wr_ready` deasserts unexpectedly.

Debug: 1. Check async FIFO depth (DEPTH parameter) 2. Verify read-side is consuming data 3. Monitor clock ratio - may need deeper FIFO

Issue 3: Unexpected Latency

Symptom: data takes longer than expected.

Explanation: - Total latency = skid buffer (1 write clock) + async FIFO CDC (3-5 cycles) - Minimum 4-6 cycles even when empty — see the latency table above, which is where this number comes from - There is no zero-latency bypass: `gaxi_skid_buffer` registers its handshake outputs, so the skid hop always costs a write clock - This is inherent to CDC design

---

9.8 Related Modules

- gaxi_skid_buffer - Synchronous version
- gaxi_fifo_async - Async FIFO without skid buffer
- GAXI Index - Overview

9.9 Testing

```
# Async skid buffer tests
pytest val/cdc/test_gaxi_buffer_async.py -k "skid" -v

# Test specific clock ratio
pytest val/cdc/test_gaxi_buffer_async.py -k "skid" -k "wr10_rd20" -v
```

9.10 References

- Cummings, C.E. “Simulation and Synthesis Techniques for Asynchronous FIFO Design.” *SNUG San Jose*, 2002 — the Gray-pointer async FIFO inside the wrapped `gaxi_fifo_async` (`USE_JOHNSON=0`); the skid buffer in front of it is a standard register-slice technique with no dedicated paper.
- Ginosar, R. “Metastability and Synchronizers: A Tutorial.” *IEEE Design & Test of Computers*, 28(5), Sept./Oct. 2011. DOI: 10.1109/MDT.2011.113 — peer-reviewed treatment of the two-clock FIFO synchronizer.
- Deschamps, J.-P., Sutter, G.D., Cantó, E. *Guide to FPGA Implementation of Arithmetic Functions*, Springer (LNEE 149), 2012, ch. 5 “Electronic Aspects of Digital Design” — sec. 5.3.5.3 “Asynchronous FIFO” — textbook treatment (secondary source).
- Indexed in `rtl/cdc/references/README.md` (local PDF: `AsyncFifoGrayPointers__Cummings2002.pdf`, `MetastabilitySynchronizers__Ginosar2011.pdf`)
- **Source:** `rtl/cdc/gaxi_skid_buffer_async.sv`
- **Dependencies:**
  - `gaxi_skid_buffer.sv`
  - `gaxi_fifo_async.sv`
- **Tests:** `val/cdc/test_gaxi_buffer_async.py`

Version: 1.0 **Last Updated:** 2025-10-06

9.11 Navigation

- ← Back to GAXI Index
- ← Back to Main Documentation Index

10 glitch_free_n_dff_arn

10.1 Overview

This is the synchronizer the rest of the CDC toolbox is built on: a parameterized multi-stage flip-flop chain for safe clock domain crossing (CDC). You set the stage count and the width; the configurable flip-flop stages knock down metastability risk while your data makes the jump between asynchronous clock domains intact.

10.2 Parameters

Parameter	Type	Default	Description
FLOP_COUNT	int	3	Number of synchronizer flip-flop stages
WIDTH	int	4	Data width in bits

10.3 Ports

Port	Direction	Width	Description
`clk`	Input	1	Destination domain clock
`rst_n`	Input	1	Destination domain active-low reset
`d`	Input	WIDTH	Input data from source clock domain
`q`	Output	WIDTH	Synchronized output data in destination domain

10.4 Functional Description

10.4.1 The Metastability Problem

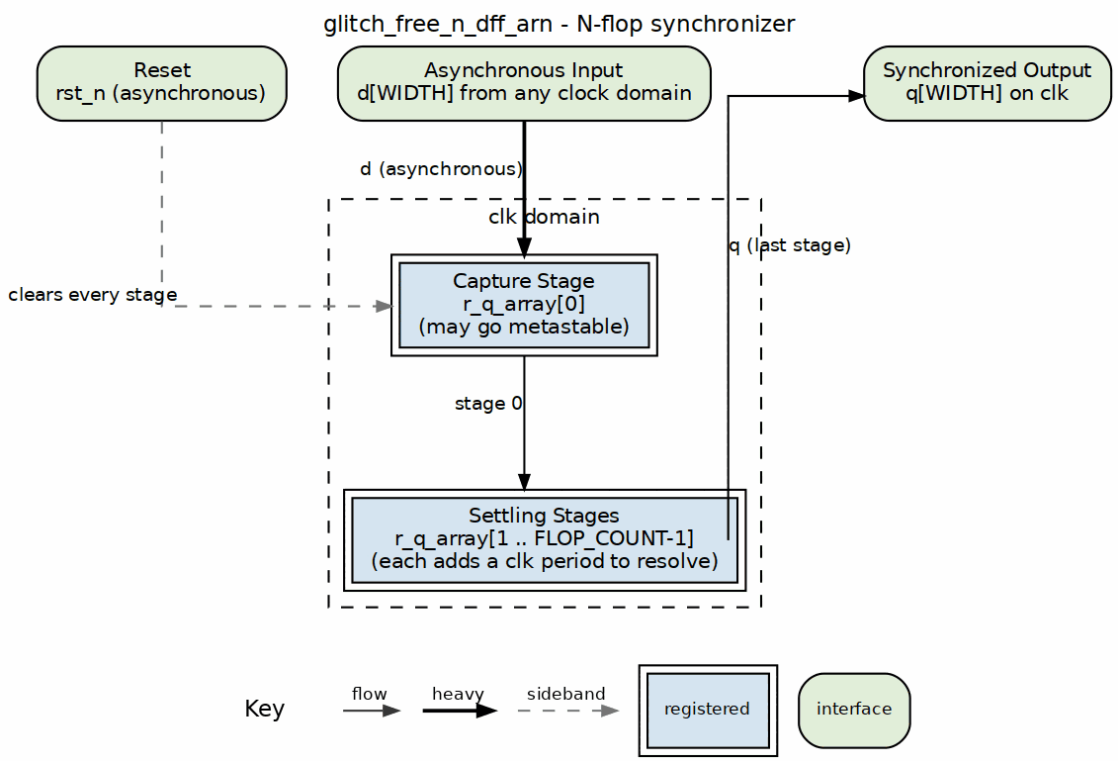
When signals cross between asynchronous clock domains:

```
Source Clock     __|‾|__|‾|__|‾|__|‾|__|‾|__
Data Change      _____|‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾
Dest Clock       ___|‾|__|‾|__|‾|__|‾|__|‾|_
Setup Violation?      ↑ Potential metastability
```

Metastability effects:

- **Undefined output**: Neither 0 nor 1, but an intermediate voltage
- **Extended resolution**: Takes longer than a normal clock period to settle
- **Propagation**: Can corrupt downstream logic
- **System failure**: Can take the entire design down

10.4.2 Multi-Stage Synchronizer Solution



glitch_free_n_dff_arn block diagram

d comes from any other clock domain and crosses into the clk domain at the capture stage r_q_array[0], the one flop that may go metastable. The settling stages r_q_array[1 .. FLOP_COUNT-1] each give it another clock period to resolve, and q is taken from the last stage. rst_n clears every stage asynchronously.

Stage by stage:

1. **Stage 1 (FF1)**: Captures the source data — this is the one that may go metastable
2. **Stage 2 (FF2)**: Gives metastability time to resolve, low probability of propagation
3. **Stage 3+ (FFN)**: Further **increases** MTBF (Mean Time Between Failures) — each added stage gives metastability more time to resolve (see the ~1000×-per- stage table below)

10.4.3 Packed Array Structure

```
logic [FC-1:0][WIDTH-1:0] r_q_array;
```

- **Efficient storage**: Single declaration for all stages
- **Clear indexing**: r_q_array[0] is first stage, r_q_array[FC-1] is last
- **Synthesis friendly**: Maps well to FPGA/ASIC resources

10.4.4 Shift Register Behavior

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        // Reset all flip-flops to zero
        r_q_array <= {FC{{DW{1'b0}}}};
    end else begin
        // Load new data into first stage
        r_q_array[0] <= d;
        // Shift existing data through remaining stages
        for (int i = 1; i < FC; i++) begin
            r_q_array[i] <= r_q_array[i-1];
        end
    end
end
```

10.4.5 Reset Behavior

- **Asynchronous reset** (active-low rst_n): the RTL uses always_ff @(posedge clk or negedge rst_n) (the “arn” suffix = async reset, active-low), driven by the destination-domain reset
- **Complete reset**: All stages reset to zero
- **Nested replication**: {FC{{DW{1'b0}}}} creates FC copies of DW zeros

10.4.6 MTBF (Mean Time Between Failures) Analysis

The MTBF calculation:

```
MTBF = (e^(t_res/τ)) / (T₀ × f_clk × f_data)

Where:
- t_res = Resolution time available (clock period - setup - routing)
- τ     = Flip-flop metastability resolution time constant (~200ps
typical)
- T₀    = Metastability aperture / window constant (device-specific,
~1e-12 s)
- f_clk = Destination clock frequency
- f_data = Data transition frequency
```

τ appears ONLY in the exponent. The denominator constant is T_0, not τ — they are different device parameters and are not interchangeable. Both are process/library specific: take them from the vendor's metastability characterization data, not from a datasheet timing table.

The stage-count table below is a **rule of thumb** for relative improvement, not a result derived from this formula. Use the formula only with real τ/T_0 values for your target device; otherwise rely on the table for sizing.

Stage count impact:

Stages	Relative MTBF
1	1x (baseline)
2	~1000x
3	~1,000,000x
4	~1,000,000,000x

Relative only, and deliberately so. This table used to carry an absolute column — Minutes / Hours / Years / Millennia — with no stated basis, and it disagreed with this module's own RTL header (which claims ~10^6 hours at FLOP_COUNT=2, about 114 years, where the table said "Hours") by roughly eight orders of magnitude. Both figures were unsourced, so neither could be trusted and the column is gone.

An absolute MTBF is a property of your device and your clock, not of this module. If you need one, put your vendor's `tau` and `T0` into the formula above with your real `f_clk` and `f_data`. If you do not have those numbers, use the relative column for sizing and do not quote an absolute figure to anyone.

Practical guidelines:

- **2 stages**: Minimum for most applications
- **3 stages**: Recommended for high-reliability systems

- **4+ stages**: Extreme reliability requirements (aerospace, medical)

10.5 Timing Characteristics

10.5.1 Synchronizer Latency

- **Minimum latency**: 2 clock cycles (2-stage synchronizer)
- **General latency**: `FLOP_COUNT` clock cycles
- **Fixed delay**: Independent of data pattern

10.5.2 Performance Impact

```
// 2-stage: 2 cycle latency, good MTBF
glitch_free_n_dff_arn #(.FLOP_COUNT(2)) fast_sync (...);

// 3-stage: 3 cycle latency, excellent MTBF
glitch_free_n_dff_arn #(.FLOP_COUNT(3)) safe_sync (...);

// 4-stage: 4 cycle latency, extreme MTBF
glitch_free_n_dff_arn #(.FLOP_COUNT(4)) ultra_safe_sync (...);
```

10.6 Usage Examples

10.6.1 Reset Synchronizer Pattern

```
// Asynchronous assert, synchronous deassert
logic reset_sync;
glitch_free_n_dff_arn #(.FLOP_COUNT(2), .WIDTH(1)) reset_sync_inst (
    .clk(dest_clk),
    .rst_n(1'b1),          // No reset feedback
    .d(~async_reset),      // Invert for active-low
    .q(reset_sync)
);

assign sync_reset_n = reset_sync & ~async_reset;
```

10.6.2 Enable Signal Synchronization

```
glitch_free_n_dff_arn #(.FLOP_COUNT(3), .WIDTH(1)) enable_sync (
    .clk(fast_clk),
    .rst_n(rst_n),
    .d(slow_enable),
    .q(fast_enable_sync)
);
```

10.6.3 Status Flag Collection

```
// Synchronize multiple independent status signals
glitch_free_n_dff_arn #(.FLOP_COUNT(2), .WIDTH(4)) status_sync (
    .clk(dest_clk),
    .rst_n(rst_n),
    .d({error_flag, done_flag, busy_flag, ready_flag}),
    .q({sync_error, sync_done, sync_busy, sync_ready})
);
```

10.7 Design Notes

10.7.1 Appropriate Use Cases

- **Control signals**: Reset, enable, mode switches
- **Slow status signals**: Error flags, configuration bits
- **Gray code pointers**: FIFO read/write pointers
- **Single-bit signals**: Generally safe for multi-bit if Gray coded

10.7.2 Inappropriate Use Cases

- **High-speed data buses**: Use proper CDC techniques (handshaking, FIFOs)
- **Binary counters**: Multiple bits can transition simultaneously
- **Address buses**: Binary addresses are not CDC-safe
- **Clock signals**: Never synchronize clocks this way

10.7.3 Multi-Bit Considerations

```
// SAFE: Gray-coded FIFO pointers
glitch_free_n_dff_arn #(.WIDTH(5)) gray_sync (
    .d(gray_pointer),      // Gray code - only 1 bit changes
    .q(sync_gray_pointer)
);

// UNSAFE: Binary counter synchronization
// Don't do this - multiple bits can change simultaneously
glitch_free_n_dff_arn #(.WIDTH(8)) bad_sync (
    .d(binary_counter),    // Binary - multiple bit transitions
    .q(sync_counter)       // May capture inconsistent values
);
```

10.7.4 Synthesis Considerations

On FPGA:

- **Dedicated resources**: Maps to flip-flop primitives
- **Placement**: Tools may place stages in different locations
- **Timing constraints**: Require proper CDC constraints

Timing constraints:

```
# Constrain the synchronizer input as asynchronous
set_false_path -to [get_pins sync_inst/r_q_array[0]/D]

# Or set maximum delay if there's some relationship
set_max_delay -to [get_pins sync_inst/r_q_array[0]/D] 2.0
```

Optimization prevention:

```
// Synthesis attributes to prevent optimization (one declaration, both
attributes --
// two separate declarations of r_q_array do not compile)
(* ASYNC_REG = "TRUE", DONT_TOUCH = "TRUE" *) logic [FC-1:0][WIDTH-
1:0] r_q_array;
```

10.7.5 Debug Features

Flatten the array for waveform viewing:

```
// Flatten array for waveform viewing
wire [(DW*FC)-1:0] flat_r_q;
genvar i;
generate
    for (i = 0; i < FC; i++) begin : gen_flatten_memory
        assign flat_r_q[i*DW+:DW] = r_q_array[i];
    end
endgenerate
```

Simulation visibility:

- **All stages visible**: Can observe data propagation through stages
- **Metastability injection**: Can inject X values for testing
- **Timing analysis**: Observe setup/hold violations

10.8 Related Modules

- **FIFO CDC modules**: Use this for pointer synchronization
- **Reset synchronizers**: Specialized CDC for reset signals
- **Handshake synchronizers**: For complex data CDC
- **Pulse synchronizers**: For single-cycle pulse CDC

10.9 Testing

10.9.1 Verification Strategies

Functional verification:

- **Data integrity**: Verify output eventually matches input
- **Reset behavior**: Check proper reset operation
- **Latency verification**: Confirm expected delay

Metastability testing:

```
// Stress test with rapid data changes
initial begin
    forever begin
        @(posedge source_clk);
        d <= $random();
        #($random() % 100);  // Random timing
    end
end
```

Formal verification:

- **Eventually properties**: Data will eventually propagate
- **Reset properties**: Reset behavior verification
- **No X propagation**: Ensure no unknown values propagate

10.9.2 Test Suite

From the test suite (`val/cdc/test_glitch_free_n_dff_arn.py`):

- **Key test scenarios**:
  - Test runner for glitch_free_n_dff_arn module
  - The glitch_free_n_dff_arn is a parameterized N-stage synchronizer for safe
  - clock domain crossing. It reduces metastability risk by passing signals
  - through multiple flip-flop stages.
  - Test Configurations:

Run levels come from the standard grid: `REG_LEVEL=GATE|FUNC|FULL` selects the parameter set, `TEST_LEVEL` the per-test depth. Run the whole area with `make -C val/cdc run-all-func-parallel`, never bare pytest for suites.

10.10 References

- Chaney, T.J., Molnar, C.E. “Anomalous Behavior of Synchronizer and Arbiter Circuits.” *IEEE Transactions on Computers*, C-22(4), Apr. 1973. DOI: 10.1109/T-C.1973.223730 — the observation of synchronizer metastability that the flop chain exists to contain.
- Veendrick, H.J.M. “The Behaviour of Flip-Flops Used as Synchronizers and Prediction of Their Failure Rate.” *IEEE Journal of Solid-State Circuits*, 15(2), Apr. 1980. DOI: 10.1109/JSSC.1980.1051359 — the exponential MTBF model behind the `FLOP_COUNT` depth trade.
- Ginosar, R. “Metastability and Synchronizers: A Tutorial.” *IEEE Design & Test of Computers*, 28(5), Sept./Oct. 2011. DOI: 10.1109/MDT.2011.113 — peer-reviewed tutorial on the N-flop synchronizer this module implements.
- Cummings, C.E. “Synthesis and Scripting Techniques for Designing Multi-Asynchronous Clock Designs.” *SNUG San Jose*, 2001 — two-flop synchronizer guidelines for RTL and one-module-per-crossing partitioning.
- Harris, S.L., Harris, D. *Digital Design and Computer Architecture, RISC-V Edition*, Morgan Kaufmann, 2022, ch. 3 “Sequential Logic Design”, sec. 3.5.4-3.5.6 (metastability, synchronizers, resolution time) — textbook treatment (secondary source).
- Deschamps, J.-P., Sutter, G.D., Cantó, E. *Guide to FPGA Implementation of Arithmetic Functions*, Springer (LNEE 149), 2012, ch. 5 “Electronic Aspects of Digital Design” — sec. 5.2.3 “Metastability”, 5.2.3.2 MTBF — textbook treatment (secondary source).
- Indexed in `rtl/cdc/references/README.md` (local PDF: `MetastabilitySynchronizers__Ginosar2011.pdf`, `MultiAsyncClockDesign__Cummings2001.pdf`)

10.11 Navigation

- ← Back to rtl-common Index
- ← Back to Main Documentation Index

11 reset_sync

11.1 Overview

The `reset_sync` module is a parameterized reset synchronizer with asynchronous assertion and synchronous deassertion. That combination is the whole point: reset needs to hit immediately when it fires, but it has to come *off* on a clock edge. Release it asynchronously and every flop downstream gets a setup/hold gamble. This is the standard circuit for proper reset distribution, and it exists to keep reset release from going metastable or violating timing.

- Asynchronous reset assertion (immediate)
- Synchronous reset deassertion (clocked)
- Parameterizable synchronization depth
- Metastability resolution
- Standard reset synchronizer topology
- Clean reset release timing

11.2 Parameters

reset_sync parameters

Parameter	Type	Default	Description
N	int	3	Number of synchronization stages. Must be >= 2; an elaboration `$error` rejects anything smaller.
KEEP_ATTRS	bit	1'b1	Attach vendor CDC attributes (`ASYNC_REG`, `SHREG_EXTRACT`, `altera_attribute`, `syn_preserve`) to the chain so tools recognize it and do not extract it into an SRL.
IN_ACTIVE_LOW	bit	1'b1	Polarity of the `rst_n` input. 1 = active-low (default), 0 = active-high.
OUT_ACTIVE_LOW	bit	1'b1	Polarity of the `sync_rst_n` output. 1 = active-low (default), 0 = active-high.
ASYNC_ASSERT	bit	1'b1	1 = asynchronous assert with synchronous deassert (FPGA best practice). 0 = fully synchronous style, where assertion is also clocked.

Polarity without renaming ports. `IN_ACTIVE_LOW` and `OUT_ACTIVE_LOW` let the module sit in an active-high reset domain while the ports stay named `rst_n` and `sync_rst_n`. The input gets normalized to an internal active-high form on the way in, and the requested polarity gets applied on the way out:

```
wire rst_in_h = IN_ACTIVE_LOW ? ~rst_n : rst_n;
...
sync_rst_n = OUT_ACTIVE_LOW ? ~sync_rst_h : sync_rst_h;
```

The names stick around for compatibility, which means a `reset_sync` with `OUT_ACTIVE_LOW = 0` drives an active-**high** reset out of a port still called `sync_rst_n`. Check the parameter, not the name.

11.3 Ports

11.3.1 Inputs

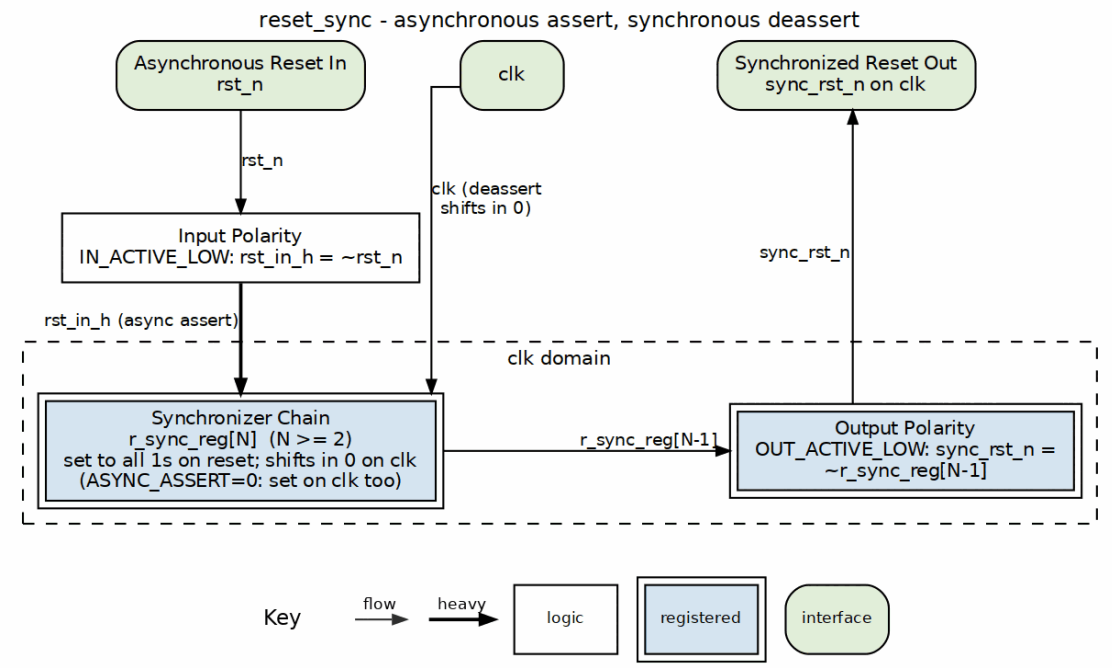
Port	Width	Description
clk	1	System clock for synchronization
rst_n	1	Asynchronous input reset (active-low)

11.3.2 Outputs

Port	Width	Description
sync_rst_n	1	Synchronized reset output (active-low)

11.4 Functional Description

11.4.1 Block diagram



reset_sync block diagram

`rst_n` arrives asynchronously on the left and is normalized to active-high `rst_in_h` by `IN_ACTIVE_LOW`. Asserting it sets the whole `r_sync_reg[N]` chain to ones at once, so reset assertion needs no clock. Release is the synchronized part: on each `clk` edge a zero shifts in, so the last stage clears `N` cycles later. The `OUT_ACTIVE_LOW` polarity stage then drives `sync_rst_n` up the right side to the output.

11.4.2 The Reset Problem

Reset release is where synchronous designs get hurt. Mishandle it and you're looking at: - **Metastability**: When reset is released asynchronously relative to clock - **Timing Violations**: Setup/hold violations during reset release - **Race Conditions**: Different flip-flops releasing from reset at different times

11.4.3 Asynchronous Assert, Synchronous Deassert

The behavior you actually want: - **Assert**: Immediate (asynchronous) when reset input becomes active - **Deassert**: Synchronized to clock edge to ensure clean timing

11.4.4 Synchronization Register Chain

```
logic [N-1:0] r_sync_reg = '0;
```

An N-stage shift register provides the synchronization delay. Here's the part that trips everyone up: the chain carries the **active-high** internal form of the reset — a 1 in the chain means "reset asserted". That's the opposite of what the port names suggest, and it's the single most common source of confusion when people read this module.

11.4.5 Reset Logic

```
// ASYNC_ASSERT = 1 (default): async assert, sync deassert
always_ff @(posedge clk or posedge rst_in_h) begin
    if (rst_in_h) r_sync_reg <= '1;                 // hold asserted
through chain
    else          r_sync_reg <= {r_sync_reg[N-2:0], 1'b0};
end
```

While reset is asserted, the chain is held at all ones. Once it releases, zeros shift in from the LSB, and after N clock edges the MSB has cleared and the reset is considered released.

With `ASYNC_ASSERT = 0` the sensitivity list drops `rst_in_h` and assertion gets sampled on the clock edge like any other synchronous load:

```
always_ff @(posedge clk) begin
    if (rst_in_h) r_sync_reg <= '1;
    else          r_sync_reg <= {r_sync_reg[N-2:0], 1'b0};
end
```

That style can't assert reset while the clock is stopped — which is exactly why the asynchronous-assert variant is the default.

11.4.6 Output Assignment

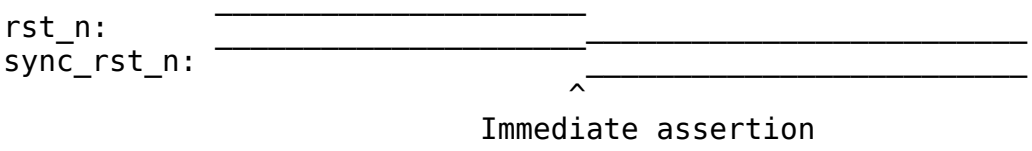
```
wire sync_rst_h = r_sync_reg[N-1];
always_comb begin
    sync_rst_n = OUT_ACTIVE_LOW ? ~sync_rst_h : sync_rst_h;
end
```

The MSB of the chain is the active-high synchronized reset; the output stage applies the requested polarity.

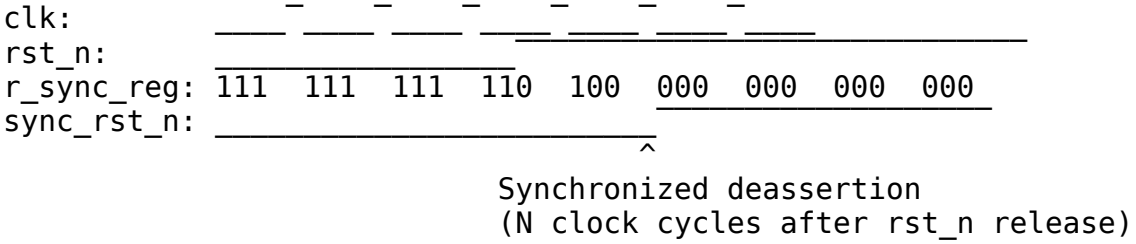
The RTL body is written out four times inside a `generate`, once per combination of `ASYNC_ASSERT` and `KEEP_ATTRS`, because the vendor attributes have to be attached at the declaration. All four variants are logically identical apart from the attributes and the sensitivity list. Yes, it's repetitive. That's the attribute syntax's fault, not the author's.

11.5 Timing Characteristics

11.5.1 Reset Assertion (Asynchronous)



11.5.2 Reset Deassertion (Synchronous, N=3)



11.5.3 State Sequence for N=3

Default polarities (`IN_ACTIVE_LOW = 1`, `OUT_ACTIVE_LOW = 1`), so `rst_n = 0` means reset asserted and `sync_rst_n = 0` means reset asserted downstream. Remember the chain holds the active-high form, so 111 is “asserted”.

reset_sync state sequence for N=3 at default polarities

Clock	rst_n	rst_in_h	r_sync_reg	sync_rst_n	Description
-	0	1	111	0	Reset asserted, chain forced to all ones
1	1	0	110	0	Reset released; first 0 shifted in
2	1	0	100	0	Second 0 shifted in
3	1	0	000	1	MSB clears; reset released

Clock	rst_n	rst_in_h	r_sync_reg	sync_rst_n	Description
					downstream
4+	1	0	000	1	Stable, out of reset

The output releases on the third clock edge after `rst_n` goes high — those N cycles are the metastability margin the chain exists to buy you.

11.6 Usage Examples

11.6.1 Repository Convention (read first)

House rule, established 2026-10-14: **component top wrappers synchronize; library leaf cells never do.** Every `*_top.sv` under `projects/components/*/rtl/top*/` treats its external reset input as an async-assert signal of unknown deassert quality and runs it through one `reset_sync` (N=3, `ASYNC_ASSERT=1`, polarities matching the port) per clock domain before any internal logic sees it. Blocks below the top — the `rtl/` library, `fub/`, `macro/` levels — take the already-clean reset as a plain input. Rationale:

- One synchronizer per domain at the boundary gives the metastability margin exactly once; nested chains below would only add deassert latency.
- Every top is self-protecting, whether it is instantiated under a system top or driven directly by a testbench.
- Multi-clock tops (mem-ctrl `aclk`/`dfi_clk` pairs) synchronize **each domain separately**, with CDC-FIFO side resets synchronized in their own clock.

The reference idiom lives in `cache-ip/amber-mesi-l1/rtl/top/amber_top.sv` (`u_reset_sync`, net `w_aresetn`). Swept 2026-10-14: rapids, bch, reed-solomon, andesite, scoria, kestrel, and the user-facing wrappers of utility-ip (all protocol converters, `error_injector`, `axi4_slave_rom`; internal `*_rd`/`*_wr`/`*_core`/upsize-dnsize/UART leaves excluded, as are passive observers/monitors — a monitor must track the observed domain's reset, so synchronizing it would blind it to post-reset traffic). Follow-on areas pending their active lanes: stream, granite, pumice, `l1_core`, `amber_pair_fabric`, `scoria_top_geared`; the rtl/amba subsystem-level candidates are classified in vault `tracking/rtl/cdc` TASK-004. `rtl/cdc/fifo_async.sv` and `rtl/common/fifo_sync.sv` header comments state the leaf-side contract ("reset synchronization not built-in; use externally").

Validation standard for this change (owner-set): gate-level tests are the accepted minimum — the change confines itself to reset-deassert timing, and gate suites caught zero regressions across the sweep. Func/full runs on the touched areas are recorded as follow-on work (TASK-004), not a merge blocker.

11.6.2 Choosing N Value

11.6.2.1 N=2 (Minimum)

```
reset_sync #(.N(2)) rst_sync_inst (
    .clk(sys_clk),
    .rst_n(external_rst_n),
    .sync_rst_n(internal_rst_n)
);
```

- **Use when**: Fast reset release required
- **Risk**: Minimal metastability margin

- **Applications**: High-speed designs with good clock quality

11.6.2.2 N=3 (Recommended)

```
reset_sync #(.N(3)) rst_sync_inst (
    .clk(sys_clk),
    .rst_n(external_rst_n),
    .sync_rst_n(internal_rst_n)
);
```

- **Use when**: Standard applications
- **Benefit**: Good balance of speed vs. metastability margin
- **Applications**: Most digital designs

11.6.2.3 N≥4 (Conservative)

```
reset_sync #(.N(4)) rst_sync_inst (
    .clk(sys_clk),
    .rst_n(external_rst_n),
    .sync_rst_n(internal_rst_n)
);
```

- **Use when**: Critical applications, poor clock quality
- **Benefit**: Maximum metastability protection
- **Trade-off**: Slower reset release

11.6.3 Global Reset Distribution

```
// Synchronize external reset for internal use
reset_sync #(.N(3)) global_rst_sync (
    .clk(system_clock),
    .rst_n(external_reset_n),
    .sync_rst_n(global_reset_n)
);

// Use synchronized reset throughout design
always_ff @(posedge system_clock or negedge global_reset_n) begin
    if (!global_reset_n) begin
        // Reset state
    end else begin
        // Normal operation
    end
end
```

11.6.4 Clock Domain Crossing

```
// Separate reset synchronizers for each clock domain
reset_sync #(.N(3)) clk_a_rst_sync (
    .clk(clk_a),
```

```
    .rst_n(system_rst_n),
    .sync_rst_n(clk_a_rst_n)
);

reset_sync #(.N(3)) clk_b_rst_sync (
    .clk(clk_b),
    .rst_n(system_rst_n),
    .sync_rst_n(clk_b_rst_n)
);
```

11.6.5 Power-On Reset

```
// Synchronize power-on reset
reset_sync #(.N(3)) por_sync (
    .clk(main_clk),
    .rst_n(power_on_rst_n),
    .sync_rst_n(system_rst_n)
);
```

11.6.6 Reset Controller Integration

```
module reset_controller (
    input  logic sys_clk,
    input  logic external_rst_n,
    input  logic watchdog_rst_n,
    input  logic sw_rst,
    output logic sync_rst_n
);

    logic combined_rst_n;

    // Combine all reset sources
    assign combined_rst_n = external_rst_n & watchdog_rst_n & ~sw_rst;

    // Synchronize the combined reset
    reset_sync #(.N(3)) rst_sync_inst (
        .clk(sys_clk),
        .rst_n(combined_rst_n),
        .sync_rst_n(sync_rst_n)
    );

endmodule
```

11.7 Design Notes

11.7.1 1. Why the chain looks inverted

The chain stores the **active-high** form of the reset, while the ports are named for the active-low one. Read the dataflow in order and the confusion evaporates:

- `rst_in_h` is the input normalized to active-high
- while `rst_in_h` is 1, the chain is forced to all ones (reset asserted)
- when `rst_in_h` falls, zeros shift in from the LSB
- after N edges the MSB is 0, and the output stage inverts it back to the requested polarity

So during reset the register holds 1s and after release it drains to 0s — the opposite of what the port names alone would tell you.

11.7.2 2. Metastability Resolution

The N stages buy you multiple clock cycles for metastability to resolve: - Each flip-flop stage gets a chance to resolve potential metastability - Probability of metastability propagation: (MTBF)^(-N) - N=2 typically sufficient, N=3 provides extra margin

11.7.3 3. Initialization

```
logic [N-1:0] r_sync_reg = '0;
```

The declaration initializer gives FPGA configuration a defined starting value. Note which value: 0, the *released* state. So on an FPGA the design comes out of configuration with reset already deasserted unless `rst_n` is asserted. On an ASIC the initializer is ignored, and the first asynchronous assertion establishes the state.

11.7.4 4. Vendor attributes

With `KEEP_ATTRS = 1` (the default) the chain carries `ASYNC_REG = "TRUE"` and `SHREG_EXTRACT = "NO"` for Xilinx, `altera_attribute` forcing synchronizer identification for Quartus, and `syn_preserve` for Synplify. These do two jobs: they stop the tool from packing the chain into an SRL primitive — which would quietly destroy your metastability margin — and they let CDC reports recognize it as a synchronizer. Set `KEEP_ATTRS = 0` only if your flow objects to the attributes.

11.7.5 5. Parameterization Benefits

Configurable N lets you tune for different requirements: - **N=2**: Minimum permitted, fast reset release - **N=3**: Standard choice, good metastability margin - **N>=4**: Conservative, slower release in exchange for maximum metastability protection

11.7.6 Reset Tree Planning

```
// Plan reset distribution hierarchy
// Level 1: Global reset synchronizer
// Level 2: Domain-specific reset synchronizers
// Level 3: Module-level reset distribution
```

11.7.7 Clock Quality Requirements

- **Clean Clock**: Stable, low-jitter clock required
- **Clock Frequency**: Must be running during reset release
- **Clock Gating**: Avoid gating clock used for reset synchronization

11.7.8 Reset Source Management

```
// Multiple reset sources need careful combination
wire combined_reset = por_n & external_rst_n & watchdog_rst_n &
~soft_reset;
```

11.7.9 Incorrect Sensitivity List

```
// WRONG: Missing async reset in sensitivity list
always_ff @(posedge clk) begin
    if (!rst_n) r_sync_reg <= '0;
    else r_sync_reg <= {r_sync_reg[N-2:0], 1'b1};
end

// CORRECT: Include async reset
always_ff @(posedge clk or negedge rst_n) begin
    // ... correct implementation
end
```

11.7.10 Wrong Reset Polarity

```
// Ensure consistent reset polarity throughout design
// This module uses active-low reset (rst_n)
```

11.7.11 Clock Domain Issues

```
// Each clock domain needs its own reset synchronizer
// Don't share synchronized reset across clock domains
```

11.8 Related Modules

- **Positive-edge reset**: For active-high reset systems
- **Multi-domain reset**: For complex clock domain systems
- **Reset pulse generator**: For generating reset pulses
- **Reset debouncer**: For mechanical switch inputs

Integration points worth knowing about:

- Clock domain crossing modules
- PLL/clock management units
- Power management controllers
- System controllers and reset managers

11.9 Testing

11.9.1 Test Coverage

Test coverage for the reset_sync module lives in: - **Testbench Class**: `bin/TBClasses/cdc/reset_sync_tb.py` - **Test Runner**: `val/cdc/test_reset_sync.py`

11.9.2 Test Scenarios

- Reset assertion during various clock phases
- Reset deassertion timing verification
- Multiple reset assertion/deassertion cycles
- Clock jitter during reset release
- Power-on behavior

11.9.3 Testbench Scenarios

Basic Reset Synchronization - Verifies N-cycle synchronization delay **Immediate Reset Assertion** - Verifies asynchronous assertion behavior **Multiple Reset Cycles** - Tests repeated reset/release sequences **Reset Glitch Filtering** - Validates recovery from short reset pulses

11.9.4 Assertions

```
// Verify synchronous deassertion
property sync_deassert;
    @(posedge clk)
    $rose(rst_n) |-> ##N $rose(sync_rst_n);
endproperty

// Verify asynchronous assertion
property async_assert;
    $fell(rst_n) |-> $fell(sync_rst_n);
endproperty
```

11.9.5 Coverage Points

- All reset assertion scenarios
- Reset release with different clock phases
- Multiple consecutive reset cycles
- Parameter variation coverage (different N values)

11.9.6 Running Tests

```
# Run all reset_sync tests (4 configurations: N=2,3,4,5)
pytest val/cdc/test_reset_sync.py -v
```

```
# Run specific configuration
pytest val/cdc/test_reset_sync.py::test_reset_sync[2-min] -v

# Run with waveform generation
pytest val/cdc/test_reset_sync.py -v -s
```

11.9.7 Test Results

All 4 parameter configurations passing (100% success rate): - N=2 (min) - PASSED - N=3 (typical) - PASSED - N=4 (safe) - PASSED - N=5 (max) - PASSED

Bug Discovery: the test suite caught a critical RTL bug on the initial run — an inverted reset polarity. Exactly the kind of bug that sails through a smoke test and ruins someone's bring-up. Thorough verification earns its keep.

11.10 References

- Cummings, C.E., Mills, D., Golson, S. “Asynchronous & Synchronous Reset Design Techniques - Part Deux.” *SNUG Boston*, 2003 — sec. 7 “Reset synchronizer”: asynchronous assertion, synchronous de-assertion (`ASYNC_ASSERT=1`).
- Ginosar, R. “Fourteen Ways to Fool Your Synchronizer.” *Proc. 9th IEEE Int. Symp. on Asynchronous Circuits and Systems (ASYNC)*, pp. 89-96, May 2003. DOI: 10.1109/ASYNC.2003.1199169 — “Global Reset Synchronizer”: peer-reviewed treatment of synchronizing reset removal per clock domain.
- Harris, S.L., Harris, D. *Digital Design and Computer Architecture, RISC-V Edition*, Morgan Kaufmann, 2022, ch. 3 “Sequential Logic Design”, sec. 3.5.4-3.5.6 (metastability, synchronizers, resolution time) — the synchronizer chain and its resolution-time budget — textbook treatment (secondary source).
- Indexed in `rtl/cdc/references/README.md` (local PDF: `ResetSynchronizer__Cummings2003.pdf`, `SynchronizerPitfalls__Ginosar2003.pdf`)

11.11 Navigation

- ← Back to rtl-cdc Index
- ← Back to Main Documentation Index

12 clock_pulse

12.1 Overview

The `clock_pulse` module is a periodic pulse generator with a configurable period: it emits a single-cycle pulse every WIDTH clock cycles. It's the part you reach for whenever something in the system needs to happen on a schedule — timing generation, heartbeat signals, sampling triggers, periodic events of any kind.

12.2 Parameters

Parameter	Type	Default	Description
WIDTH	int	10	Period of the pulse generation in clock cycles

WIDTH can range from 2 to 2^31-1. The parameter is declared int, which is 32-bit SIGNED, so 2^32-1 is not representable. It sets the pulse frequency (f_clk / WIDTH). The pulse itself is always exactly 1 clock cycle wide, which makes the duty cycle 1/WIDTH — 10% for WIDTH=10.

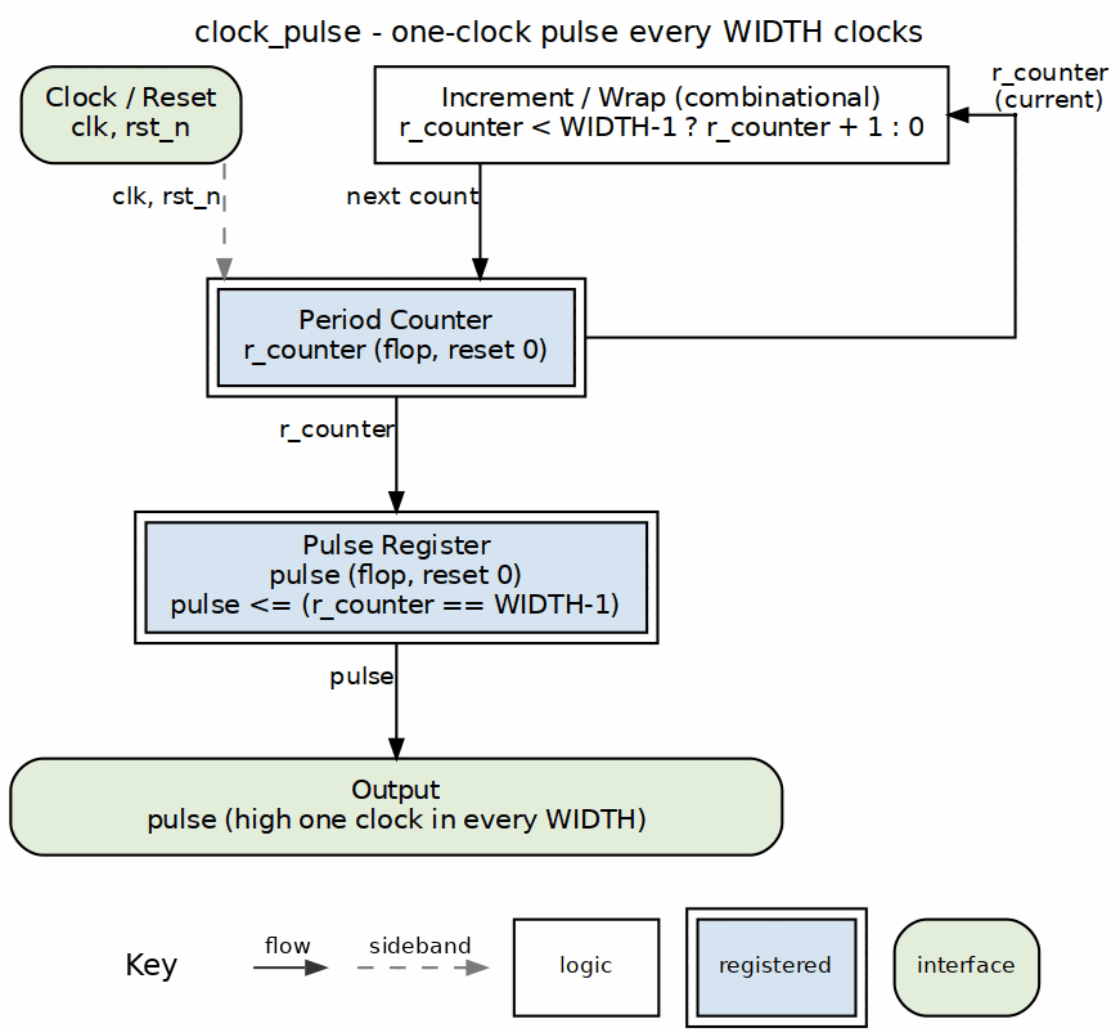
12.3 Ports

```
module clock_pulse #(
    parameter int WIDTH = 10  // PERIOD in clock cycles (the pulse
itself is
                              // always exactly 1 cycle wide)
) (
    input  logic clk,    // Input clock signal
    input  logic rst_n,  // Input reset signal
    output logic pulse   // Output pulse signal
);
```

Port	Direction	Width	Description
clk	Input	1	System clock input
rst_n	Input	1	Active-low asynchronous reset
pulse	Output	1	Periodic pulse output

12.4 Functional Description

12.4.1 Block diagram



clock_pulse block diagram

A period counter runs 0 to `WIDTH-1` and wraps, feeding back up the right edge into its increment logic. The pulse register samples the compare `r_counter == WIDTH-1`, so `pulse` is high for one clock in every `WIDTH`.

12.4.2 Internal Counter

```
// WIDTH is the PERIOD; the counter only needs to hold 0..WIDTH-1,
i.e.
// $clog2(WIDTH) bits (NOT WIDTH bits).
localparam int CW = (WIDTH < 2) ? 1 : $clog2(WIDTH);
```

```
logic [CW-1:0] r_counter;
logic [CW-1:0] w_width_minus_one;

// Properly sized period-1 constant
assign w_width_minus_one = CW'(WIDTH - 1);
```

12.4.3 Core Logic

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_counter <= 'b0;
        pulse     <= 'b0;
    end else begin
        if (r_counter < w_width_minus_one)
            r_counter <= r_counter + 1'b1;
        else
            r_counter <= 'b0;

        pulse <= (r_counter == w_width_minus_one);
    end
end
```

12.4.4 Operation Principles

1. **Counter**: Free-running counter from 0 to WIDTH-1
2. **Pulse Generation**: `pulse` is **registered** — `pulse <= (r_counter == WIDTH-1)` — so it asserts on the cycle **after** the counter reaches WIDTH-1, i.e. during the cycle when the counter has wrapped back to 0.
3. **Auto-Reset**: Counter wraps to 0 after reaching maximum
4. **Synchronous**: All operations synchronized to input clock
5. **Single Cycle**: Pulse duration is exactly one clock cycle

That second point is the one people get wrong, so let's be blunt about it: the registered comparison means `pulse` never coincides with the terminal count. It lands one cycle later, on the wrapped-to-zero count. Every timing diagram below follows from that.

12.5 Related Modules

Nothing in the tree instantiates this module and it instantiates nothing: it is a leaf, used directly by whatever design needs it. Its nearest neighbours in `rtl/common/` are:

- `clock_divider`
- `clock_gate_ctrl`

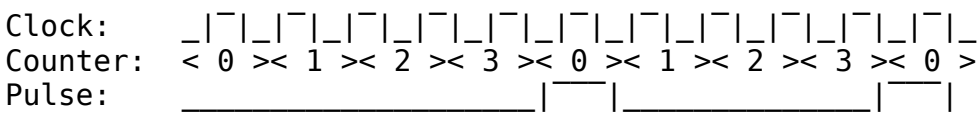
12.6 Timing Characteristics

12.6.1 Timing Characteristics

- **Period**: WIDTH clock cycles
- **Frequency**: f_clk / WIDTH
- **Pulse Width**: 1 clock cycle
- **Duty Cycle**: 1/WIDTH
- **Phase**: Because the comparison is registered, `pulse` is high one cycle **after** `r_counter == WIDTH-1` — that is, during the `r_counter == 0` cycle of each period, not on the WIDTH-1 count itself.

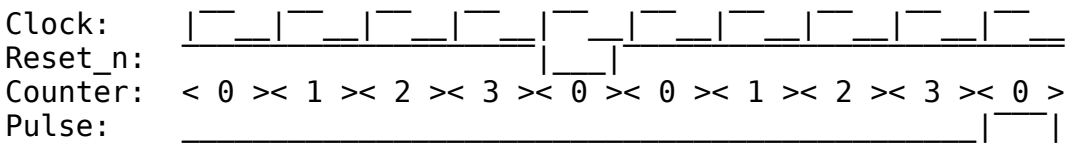
12.6.2 Basic Operation (WIDTH=4)

Watch where `pulse` actually lands — high during the `< 0 >` cell that immediately follows `< 3 >`, one cycle after the counter hit WIDTH-1. That's the registered comparison at work:



(The first `< 0 >` is the post-reset state, where `pulse` is still low.)

12.6.3 Reset Behavior



Two things worth seeing here, both consequences of the pulse being registered (`pulse <= (r_counter == w_width_minus_one)`):

- **The pulse that reset swallows.** The counter reaches 3 in the fourth cell, so a pulse would normally appear in the fifth — but reset is asserted exactly there, and reset forces `pulse <= 0`. That pulse never happens.
- **Recovery costs a full period.** After reset releases, the counter restarts from 0 and the first pulse appears in the cell *after* it next reaches 3 — the final cell above, not the one where the counter reads 3.

12.6.4 Different WIDTH Values

WIDTH	Period	Duty Cycle	Use Case
2	2 cycles	50%	Clock divider
4	4 cycles	25%	Sampling

WIDTH	Period	Duty Cycle	Use Case
10	10 cycles	10%	Timing reference
100	100 cycles	1%	Heartbeat
1000	1000 cycles	0.1%	Slow events

12.7 Usage Examples

Every parameter and port below is read from the module declaration.

```
clock_pulse #(
    .WIDTH                  (10)
) u_clock_pulse (
    .clk                    (clk),
    .rst_n                  (rst_n),
    .pulse                  (pulse)
);
```

12.8 Design Notes

12.8.1 Resource Utilization

WIDTH	Counter Bits	LUTs	FFs	Max Freq
8	3	8	4	500MHz
16	4	12	5	450MHz
32	5	18	6	400MHz
1024	10	28	11	350MHz

12.8.2 Synthesis Considerations

For high-frequency applications, pipeline the comparison:

```
// For high-frequency applications, pipeline the comparison (WIDTH >=
2).
// IMPORTANT: because the registered compare_result ALSO gates the
counter
// wrap, it must be produced from the WIDTH-2 threshold, not WIDTH-1.
Comparing
// against WIDTH-1 here makes the counter sequence 0..WIDTH,0 and
yields a
// period of WIDTH+1, not WIDTH. With WIDTH-2 the period is exactly
WIDTH and
// the pulse still lands on the r_counter==0 cycle, matching the base
module.
module clock_pulse_pipelined #(
    parameter int WIDTH = 1000
) (
    input  logic clk,
    input  logic rst_n,
    output logic pulse
);

    localparam int CW = (WIDTH < 2) ? 1 : $clog2(WIDTH);
    logic [CW-1:0] r_counter;
    logic          compare_result;

    // Register the comparison one count early so the pipelined wrap
keeps a
    // WIDTH-cycle period.
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) compare_result <= 1'b0;
        else        compare_result <= (r_counter == CW'(WIDTH - 2));
    end
```

```
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            r_counter <= 'b0;
            pulse     <= 1'b0;
        end else begin
            r_counter <= compare_result ? '0 : r_counter + 1'b1;
            pulse     <= compare_result;
        end
    end

endmodule
```

12.8.3 Design Considerations

1. **Choose Appropriate WIDTH**: Balance between resolution and resource usage
2. **Consider Reset Timing**: Ensure clean startup behavior
3. **Validate Timing**: Verify pulse frequency matches requirements
4. **Plan for Variation**: Consider process, voltage, temperature effects
5. **Monitor Resource Usage**: Large counters can impact timing and area
6. **Use Enables**: Add enable signals for conditional operation
7. **Document Timing**: Clearly specify pulse rates and relationships

12.8.4 Common Applications

1. **Timing References**: System heartbeats, watchdog timers
2. **Sampling Systems**: ADC triggers, data acquisition timing
3. **Communication**: Baud rate generation, protocol timing
4. **Memory Systems**: Refresh timing, access scheduling
5. **Test Equipment**: Pattern generation, stimulus timing
6. **Power Management**: Activity monitoring, timeout generation
7. **Display Systems**: Refresh rates, sync generation

That's the whole module. It's a small thing, but precise periodic timing shows up in nearly every design you'll ever ship — get this one right once, parameterize it well, and reuse it forever.

12.9 Testing

12.9.1 Comprehensive Test Bench

```
module tb_clock_pulse;

    parameter WIDTH = 10;

    logic clk, rst_n, pulse;

    // Clock generation
    initial clk = 0;
    always #5ns clk = ~clk; // 100MHz

    // DUT instantiation
    clock_pulse #(.WIDTH(WIDTH)) dut (.*);

    // Test sequence
    initial begin
        rst_n = 0;
        #100ns rst_n = 1;

        // Test basic pulse generation
        test_basic_pulse_generation();

        // Test reset behavior
        test_reset_behavior();

        // Test timing accuracy
        test_timing_accuracy();

        $display("All tests completed!");
        $finish;
    end

    // Test basic pulse generation
    task test_basic_pulse_generation();
        int pulse_count = 0;
        int cycle_count = 0;
        begin
            $display("Testing basic pulse generation (WIDTH=%0d)...",
WIDTH);

            // Monitor for several pulse periods. WIDTH*3 + 1 edges:
`@(posedge clk)`
```

```
            // resumes BEFORE the NBA update, so pulses are first
visible at edges
            // WIDTH+1, 2*WIDTH+1, 3*WIDTH+1 (see test_reset_behavior
for the trace).
            repeat (WIDTH * 3 + 1) begin
                @(posedge clk);
                cycle_count++;
                if (pulse) pulse_count++;
            end

            // Verify pulse count
            if (pulse_count == 3) begin
                $display("PASS: Correct number of pulses generated");
            end else begin
                $error("FAIL: Expected 3 pulses, got %0d",
pulse_count);
            end
        end
    endtask

    // Test reset behavior -- the two consequences of the registered
pulse
    task test_reset_behavior();
        int pulses_during_reset = 0;
        int cycles_to_first_pulse = 0;
        begin
            $display("Testing reset behavior...");

            // Line the counter up so a pulse is due, then assert
reset in
            // exactly that cycle. Reset forces pulse <= 0, so that
pulse is
            // swallowed and never appears.
            repeat (WIDTH - 1) @(posedge clk);
            rst_n = 0;
            repeat (WIDTH * 2) begin
                @(posedge clk);
                if (pulse) pulses_during_reset++;
            end
            if (pulses_during_reset == 0) begin
                $display("PASS: no pulse while reset is asserted");
            end else begin
                $error("FAIL: %0d pulse(s) during reset",
pulses_during_reset);
            end

            // Recovery costs a full period: the counter restarts from
```

```
0, so the
            // first pulse lands one cycle AFTER it next reaches
WIDTH-1.
            @(negedge clk);
            rst_n = 1;
            // Expect the first pulse on the (WIDTH+1)th sampled edge,
not the
            // WIDTHth. Three separate cycles are involved and it is
easy to
            // lose one: the counter reaches WIDTH-1 at edge WIDTH-1,
`pulse`
            // is NBA-assigned at edge WIDTH, and `@(posedge clk)`
resumes in
            // the Active region -- BEFORE that edge's NBA update --
so the
            // high value is first visible at edge WIDTH+1.
            forever begin
                @(posedge clk);
                cycles_to_first_pulse++;
                if (pulse) break;
                if (cycles_to_first_pulse > WIDTH * 2) begin
                    $error("FAIL: no pulse within two periods of reset
release");
                    break;
                end
            end
            if (cycles_to_first_pulse == WIDTH + 1) begin
                $display("PASS: first pulse %0d sampled edges after
release",
                         WIDTH + 1);
            end else begin
                $error("FAIL: expected first pulse at cycle %0d, got
%0d",
                       WIDTH + 1, cycles_to_first_pulse);
            end
        end
    endtask

    // Test timing accuracy
    task test_timing_accuracy();
        time pulse_times[10];
        time pulse_periods[9];
        int i;
        real avg_period, expected_period;
        begin
            $display("Testing timing accuracy...");

            expected_period = WIDTH * 10.0; // 10ns clock period
```

```
            // Capture pulse timing
            for (i = 0; i < 10; i++) begin
                @(posedge pulse);
                pulse_times[i] = $time;
            end

            // Calculate periods
            for (i = 0; i < 9; i++) begin
                pulse_periods[i] = pulse_times[i+1] - pulse_times[i];
            end

            // Calculate average period
            avg_period = 0;
            for (i = 0; i < 9; i++) begin
                avg_period += pulse_periods[i];
            end
            avg_period = avg_period / 9.0;

            // Check accuracy (within 1% tolerance)
            if (abs(avg_period - expected_period) < expected_period *
0.01) begin
                $display("PASS: Average period = %.1f ns (expected
%.1f ns)",
                         avg_period, expected_period);
            end else begin
                $error("FAIL: Average period = %.1f ns (expected %.1f
ns)",
                       avg_period, expected_period);
            end
        end
    endtask

    function real abs(real value);
        abs = (value >= 0) ? value : -value;
    endfunction

endmodule
```

12.9.2 Coverage Model

```
covergroup clock_pulse_cg @(posedge clk);

    cp_counter_values: coverpoint dut.r_counter {
        bins zero = {0};
        bins low[] = {[1:WIDTH/4]};
        bins mid[] = {[WIDTH/4+1:3*WIDTH/4]};
```

```
        bins high[] = {[3*WIDTH/4+1:WIDTH-2]};
        bins max = {WIDTH-1};
    }

    cp_pulse: coverpoint pulse {
        bins asserted = {1};
        bins deasserted = {0};
    }

    cp_reset: coverpoint rst_n {
        bins reset = {0};
        bins normal = {1};
    }

    // Transition coverage
    cp_pulse_edges: coverpoint pulse {
        bins rising = (0 => 1);
        bins falling = (1 => 0);
        bins stable_low = (0 => 0);
        bins stable_high = (1 => 1);
    }

    // Cross coverage
    cross_counter_pulse: cross cp_counter_values, cp_pulse;

endcovergroup
```

12.9.3 Formal Properties

```
module clock_pulse_properties;

    // Bind to DUT
    bind clock_pulse clock_pulse_properties props_inst (.*);

    // Property: reaching max count produces a pulse on the NEXT
cycle.
    // The comparison is registered (pulse <= r_counter == WIDTH-1),
so the
    // implication is |=> (next cycle), NOT |-> (same cycle).
    property pulse_after_max_count;
        @(posedge clk) disable iff (!rst_n)
        (dut.r_counter == WIDTH-1) |=> pulse;
    endproperty

    // Property: a pulse implies the PREVIOUS count was max
    property pulse_implies_prev_max;
        @(posedge clk) disable iff (!rst_n)
```

```
        pulse |-> $past(dut.r_counter == WIDTH-1);
    endproperty

    // Property: Counter wraps correctly
    property counter_wrap;
        @(posedge clk) disable iff (!rst_n)
        (dut.r_counter == WIDTH-1) |=> (dut.r_counter == 0);
    endproperty

    // Property: Counter increments
    property counter_increment;
        @(posedge clk) disable iff (!rst_n)
        (dut.r_counter < WIDTH-1) |=>
        (dut.r_counter == $past(dut.r_counter) + 1);
    endproperty

    // Property: Reset behavior
    property reset_behavior;
        @(posedge clk)
        !rst_n |=> (dut.r_counter == 0) && !pulse;
    endproperty

    // Assertions
    assert property (pulse_after_max_count);
    assert property (pulse_implies_prev_max);
    assert property (counter_wrap);
    assert property (counter_increment);
    assert property (reset_behavior);

endmodule
```

12.10 References

- No peer paper — a modulo-WIDTH counter emitting a one-cycle periodic pulse is a standard technique with no defensible single origin (marked “standard technique” in the references index).
- Roy, S. *Advanced Digital System Design*, Springer, ch. 4 “Basic Sequential Circuits”, §4.6 “Synchronous Counter Design” — textbook treatment (secondary source).
- Indexed in `rtl/common/references/README.md`

12.11 Navigation

- ← Back to rtl-common Index
- ← Back to Main Documentation Index

13 apb4_slave_cdc

Module: `apb4_slave_cdc.sv` **Location:** `rtl/amba/apb4/` **Status:** Production Ready

13.1 Overview

The APB Slave CDC (Clock Domain Crossing) module is a complete APB slave interface with the crossing built in: full APB on the `pclk` side, an AXI/GAXI-style backend on the `aclk` side. It's what lets you integrate APB peripherals running at different clock frequencies without metastability becoming your problem.

13.1.1 Key Features

- **Safe CDC:** Gray-pointer asynchronous FIFOs, one per direction
- **Dual Clock Domains:** APB (pclk) and AXI (aclk) operate independently
- Caveat: **reset both domains together** — a one-sided reset is NOT safe; see Reset Behavior
- **Full APB4 Support:** Complete AMBA 4 APB protocol compliance
- **Command/Response Interface:** Clean GAXI-style backend interface
- **Buffered Operation:** Integrated skid buffers for elastic storage

Protocol scope: APB4 only. For APB5 signalling use `apb5_slave_cdc` from `rtl/amba/apb5/` — see the APB5 book.

13.2 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
PROT_WIDTH	int	3	APB protection signal width
DEPTH	int	2	Skid-buffer depth **in entries** inside the wrapped apb4_slave; also the floor for the CDC FIFO depth
USE_JOHNSON	int	0 (Gray)	CDC-FIFO pointer encoding: 0 Gray (power-of-2 derived depth only), 1 Johnson (any FIFO depth, max(DEPTH,4)-bit pointers; module-level DEPTH stays limited to 2..8 inclusive by the skid buffers). Gray by default — Johnson is opt-in because its pointers cost DEPTH bits in both domains and every synchronizer stage.
USE_2_PHASE_CDC	bit	1	**Deprecated and ignored.** Has no effect on the generated logic

13.3 Ports

```
module apb4_slave_cdc #(
    parameter int ADDR_WIDTH  = 32,
    parameter int DATA_WIDTH  = 32,
    parameter int STRB_WIDTH  = DATA_WIDTH / 8,
    parameter int PROT_WIDTH  = 3,
    parameter int DEPTH       = 2,
    // Async-FIFO pointer encoding: 0 = Gray (power-of-2 derived depth
only),
    // 1 = Johnson (any depth, DEPTH-bit pointers). Forwarded to both
CDC FIFOs.
    parameter int USE_JOHNSON = 0,
    // DEPRECATED / NO EFFECT -- retained only so existing
instantiations
    // still elaborate. See "CDC Implementation" below.
    parameter bit USE_2_PHASE_CDC = 1'b1,
    // Derived width aliases the port list references -- not
independent knobs
    parameter int DW  = DATA_WIDTH,
    parameter int AW  = ADDR_WIDTH,
    parameter int SW  = STRB_WIDTH,
    parameter int PW  = PROT_WIDTH
) (
    // Clock and Reset
    input  logic              aclk,
    input  logic              aresetn,
    input  logic              pclk,
    input  logic              presetn,

    // APB Slave Interface (pclk domain)
    input  logic              s_apb_PSEL,
    input  logic              s_apb_PENABLE,
    output logic              s_apb_PREADY,
    input  logic [AW-1:0]     s_apb_PADDR,
    input  logic              s_apb_PWRITE,
    input  logic [DW-1:0]     s_apb_PWDATA,
    input  logic [SW-1:0]     s_apb_PSTRB,
    input  logic [PW-1:0]     s_apb_PPROT,
    output logic [DW-1:0]     s_apb_PRDATA,
    output logic              s_apb_PSLVERR,

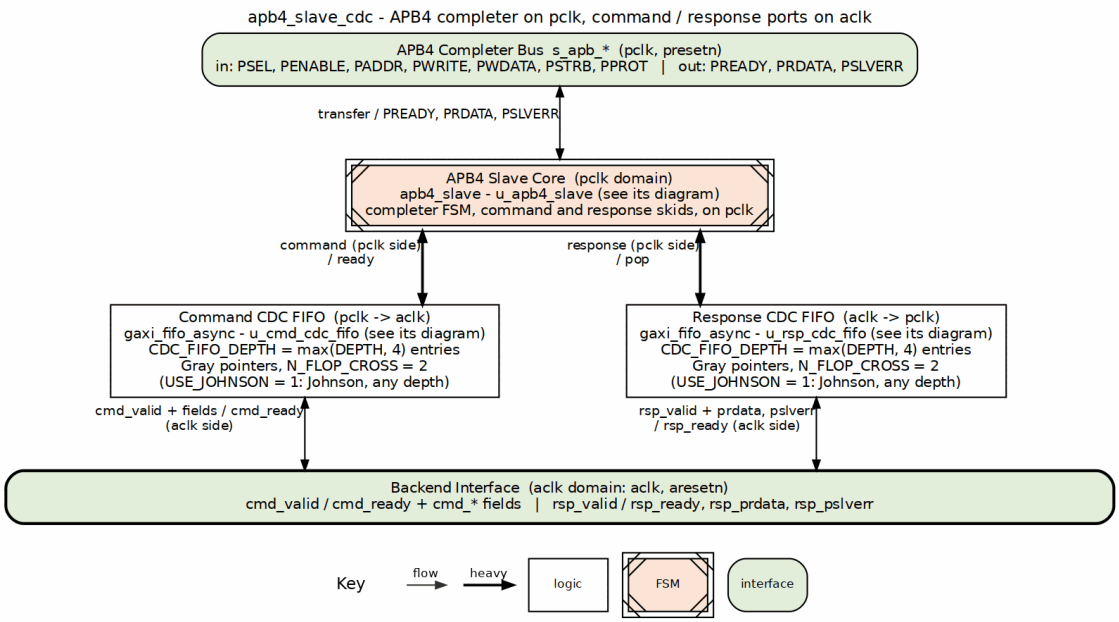
    // Command Interface (aclk domain)
    output logic              cmd_valid,
    input  logic              cmd_ready,
    output logic              cmd_pwrite,
    output logic [AW-1:0]     cmd_paddr,
    output logic [DW-1:0]     cmd_pwdata,
```

```
    output logic [SW-1:0]     cmd_pstrb,
    output logic [PW-1:0]     cmd_pprot,

    // Response Interface (aclk domain)
    input  logic              rsp_valid,
    output logic              rsp_ready,
    input  logic [DW-1:0]     rsp_prdata,
    input  logic              rsp_pslverr
);
```

13.4 Functional Description

13.4.1 Block diagram



apb4_slave_cdc block diagram

The APB side runs on `pclk`: the bus feeds the wrapped `apb4_slave` (one box; see its diagram). Each direction crosses to the `aclk` backend through its own `gaxi_fifo_async`: the command drops down the left lane (pclk to aclk), the response climbs the right lane (aclk to pclk). The crossings themselves – Gray pointers through two-flop synchronizers – are inside the async FIFOs and drawn in the `gaxi_fifo_async` diagram. Both FIFOs are `max(DEPTH, 4)` entries deep.

13.4.2 Clock Domains

APB domain (pclk):

- APB slave interface signals
- Typical frequency: 50-200 MHz
- Used by APB master/interconnect

AXI domain (aclk):

- Command and response interfaces
- Can be faster or slower than pclk
- Used by backend processing logic

13.4.3 CDC Structure

The module is an `apb4_slave` in the `pclk` domain plus two independent asynchronous FIFOs:

FIFO	Direction	Write domain	Read domain	Payload width
`u_cmd_cdc_fifo`	Command	`pclk` / `presetn`	`aclk` / `aresetn`	`CPW = AW + DW + SW + PW + 1`
`u_rsp_cdc_fifo`	Response	`aclk` / `aresetn`	`pclk` / `presetn`	`RPW = DW + 1`

Both are `gaxi_fifo_async` instances with:

- **Pointer encoding:** absolute read/write pointers, Gray by default (`USE_JOHNSON = 0`); pass `USE_JOHNSON = 1` for Johnson
- **Synchronizer depth:** `N_FLOP_CROSS = 2` (two-flop synchronizer per crossed pointer)
- **FIFO depth:** `CDC_FIFO_DEPTH = (DEPTH < 4) ? 4 : DEPTH` — a floor of 4 entries regardless of the `DEPTH` used for the internal skid buffers. **Under the default `USE_JOHNSON = 0` a power of two is required**, because Gray carries a generate-scope elaboration check (`(USE_JOHNSON == 0) && ((DEPTH & (DEPTH-1)) != 0)` -> `$error`). The check sees the DERIVED depth `max(DEPTH, 4)`, while the same `DEPTH` also reaches the wrapped `apb4_slave`'s two `gaxi_skid_buffer` instances UNFLOORED, whose guard accepts 2..8 inclusive (any integer). Under the default Gray encoding the floored depth must be a power of two, so the legal `DEPTH` set is **{2, 3, 4, 8}** (2 and 3 floor to 4); 5, 6 and 7 floor to themselves, fail the power-of-2 check, and elaborate only with `USE_JOHNSON = 1` – Johnson buys exactly those three depths. It pays for that with pointers as wide as the derived FIFO depth `max(DEPTH, 4)` against Gray's `$clog2(max(DEPTH, 4))+1` – at the default `DEPTH = 2` that is 4 bits vs 3, not 2 vs 2 – in both domains and every synchronizer stage; Gray is the default so that cost is never paid by accident.

There is no separate metastability-hardening option — two-flop synchronization is fixed at instantiation.

13.4.4 Reset Behavior

`presetn` and `aresetn` are separate reset domains, but they must be asserted together. **A one-sided reset is not safe. Quiesce the bus first.**

The local reset clears that domain's own pointer, but the crossed copy of the *remote* pointer is a live synchronizer (`glitch_free_n_dff_arn`, N=2) that keeps sampling the non-reset domain the moment reset deasserts — it does not hold at zero. The reset side comes back with its own pointer at zero and the remote pointer at whatever the other side had reached: not an empty FIFO, a mismatched one.

Neither consequence is a clean discard:

- **Consumed commands are re-presented.** Pulsing `aresetn` with an **already-consumed** command in the cmd FIFO rewinds the read pointer behind the write pointer, so the backend sees that command again after reset. It does not time out — it re-executes. An *unread* command is not at risk: its read pointer is already behind the write pointer, so resetting it to 0 rewinds nothing and the entry is delivered exactly once.
- **The response FIFO can fabricate responses.** The same rewind on the response path presents entries the APB side never queued, so the APB master can complete a transfer the backend never answered.

Pointers being absolute positions rather than toggle parity is what makes the *steady-state* crossing reliable; it does not make a one-sided reset safe.

13.4.5 Why Not the Previous 2-Phase Handshake

`USE_2_PHASE_CDC` selected a toggle-based handshake in an earlier revision. It is now deprecated and ignored; the parameter survives only for source-compatibility with existing instantiations.

The failure mode is worth understanding, because it's the kind that ships. A 2-phase handshake encodes each transfer as a **toggle**. Reset the two domains independently and the toggle parity desynchronizes — the link fabricates or drops exactly one transfer. Permanently, because nothing ever re-syncs it. Pair that with the `apb4_slave` FSM, which pairs commands and responses by position rather than by tag, and a single phantom transfer offsets the response stream by one forever: every read returns the previous read's data.

This was observed on the Nexys A7 `ddr2-char` board on 2026-07-19. Reading a single CSR eight times returned the previous register's value about three times before settling, while the non-CDC harness window was stable. The two mitigations now in place are the gray-pointer FIFOs described above and the orphan-response guard in `apb4_slave` — see apb4_slave.md.

13.5 Design Notes

This is a clock-domain crossing; the usual CDC rules bind. Pointers cross Gray-coded through `glitch_free_n_dff_arn`, and the reset must be asserted in both domains – releasing one before the other leaves the pointer comparison meaningless until the second releases.

Depth buys latency tolerance, not bandwidth. A CDC FIFO's throughput is set by the slower domain; a deeper buffer absorbs a longer burst without stalling but does not raise the sustained rate. See `vault/handbook/design/cdc.md`.

13.6 Timing Characteristics

13.6.1 Maximum Clock Ratio

There is no maximum ratio between `pclk` and `aclk`. Gray-pointer FIFOs impose no relationship between the two clocks — either may be arbitrarily faster, slower, or phase-unrelated, and either may be stopped indefinitely. Stopping `aclk` simply stalls the command FIFO's read side; the APB side backpressures via `PREADY` held low and no data is lost.

What the ratio affects is throughput, not correctness. Each transfer pays the usual two-flop synchronizer latency in each direction (roughly 2-3 destination clock edges per crossing), so a very slow `aclk` directly lengthens APB wait states.

13.6.2 Timing Constraints

The crossed signals are the gray-coded pointers and the FIFO memory read path. Standard practice applies:

- Declare `pclk` and `aclk` asynchronous to each other (`set_clock_groups -asynchronous`).
- Do not over-constrain the pointer synchronizer paths; the gray encoding tolerates a one-bit-at-a-time skew by construction.

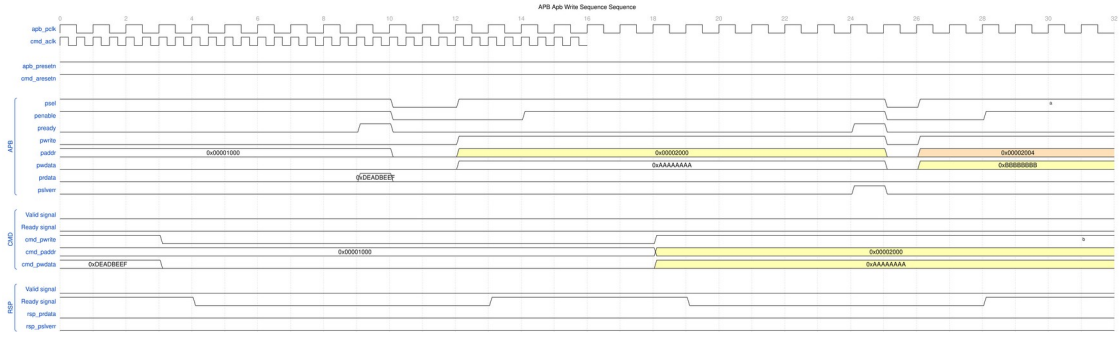
13.7 Waveforms

The following timing diagrams show CDC behavior with **both clock domains visible**:

Clock Configuration: - `apb_pclk`: 100MHz (10ns period) - `cmd_aclk`: 500MHz (2ns period), displayed with `period=0.4` for visual compactness

13.7.1 Scenario 1: Write Transaction with CDC

Shows APB write crossing from pclk domain to aclk domain:



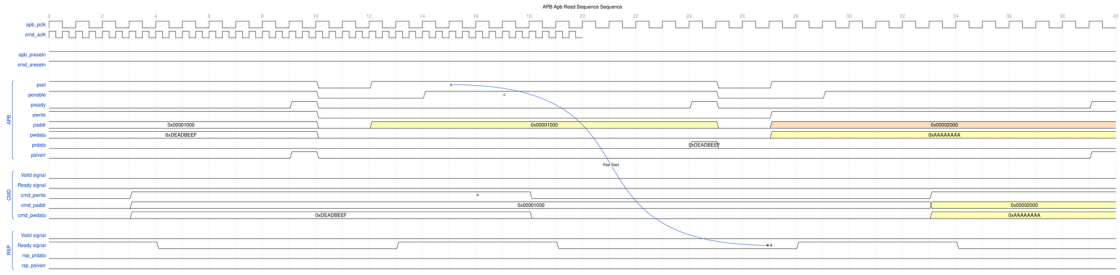
Write CDC

WaveJSON: apb_write_sequence_001.json

Key Observations: - APB transaction in pclk domain - CMD transaction crosses to aclk domain - Note the CDC latency between domains

13.7.2 Scenario 2: Read Transaction with CDC

Shows APB read with response crossing back from aclk to pclk domain:

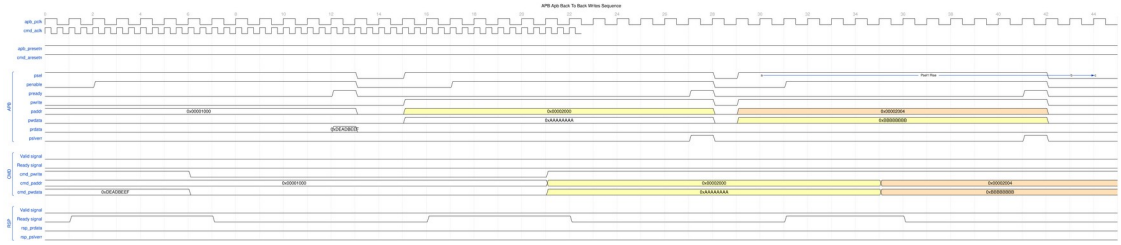


Read CDC

WaveJSON: apb_read_sequence_001.json

Key Observations: - APB read request in pclk domain - CMD crosses to aclk domain - RSP crosses back to pclk domain - Complete round-trip CDC visible

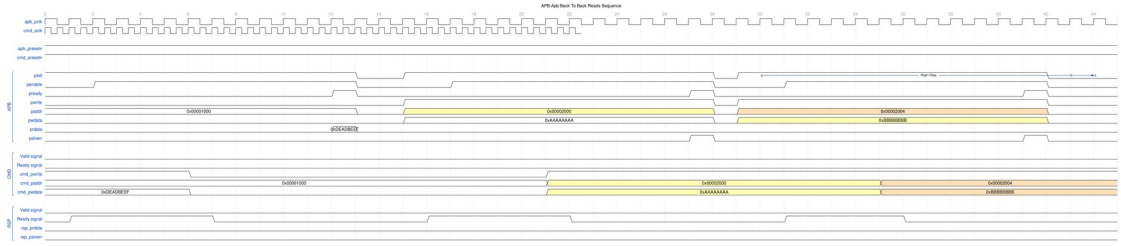
13.7.3 Scenario 3: Back-to-Back Writes with CDC



B2B Writes CDC

WaveJSON: apb_back_to_back_writes_001.json

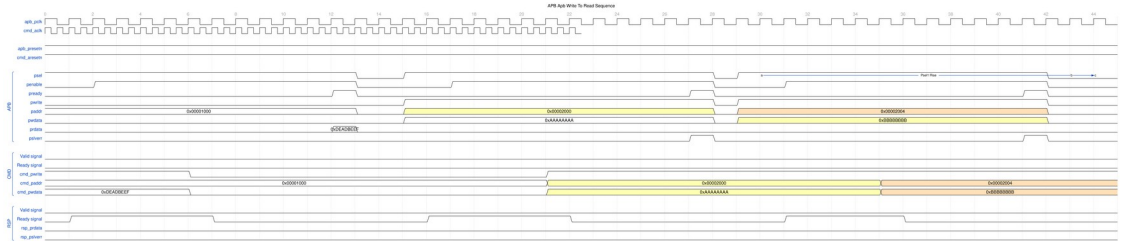
13.7.4 Scenario 4: Back-to-Back Reads with CDC



B2B Reads CDC

WaveJSON: apb_back_to_back_reads_001.json

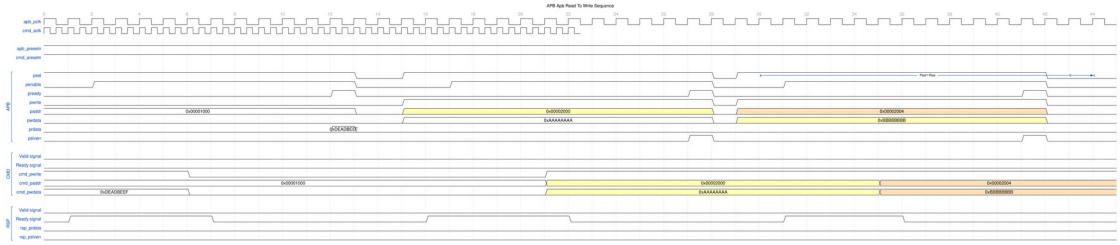
13.7.5 Scenario 5: Write-to-Read Transition with CDC



Write-to-Read CDC

WaveJSON: apb_write_to_read_001.json

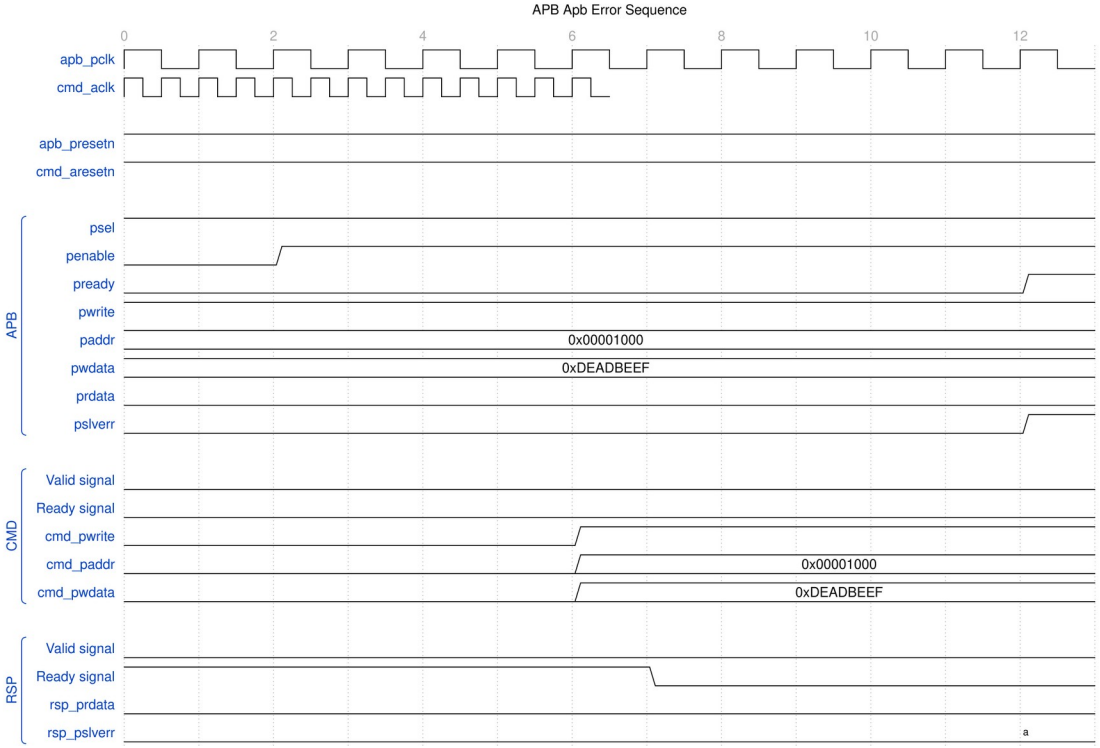
13.7.6 Scenario 6: Read-to-Write Transition with CDC



Read-to-Write CDC

WaveJSON: apb_read_to_write_001.json

13.7.7 Scenario 7: Error Response with CDC



Error CDC

WaveJSON: apb_error_001.json

13.8 Usage Examples

```
apb4_slave_cdc #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .DEPTH(2)
) u_apb_cdc (
    // APB clock domain
    .pclk         (apb_clk),
    .presetn      (apb_resetn),

    // AXI clock domain
    .aclk         (axi_clk),
    .aresetn      (axi_resetn),

    // APB slave interface (pclk domain)
    .s_apb_PSEL     (apb_psel),
    .s_apb_PENABLE  (apb_penable),
    .s_apb_PREADY   (apb_pready),
    .s_apb_PADDR    (apb_paddr),
    .s_apb_PWRITE   (apb_pwrite),
    .s_apb_PWDATA   (apb_pwdata),
    .s_apb_PSTRB    (apb_pstrb),
    .s_apb_PPROT    (apb_pprot),
    .s_apb_PRDATA   (apb_prdata),
    .s_apb_PSLVERR  (apb_pslverr),

    // Command interface (aclk domain)
    .cmd_valid      (cmd_valid),
    .cmd_ready      (cmd_ready),
    .cmd_pwrite     (cmd_pwrite),
    .cmd_paddr      (cmd_paddr),
    .cmd_pwdata     (cmd_pwdata),
    .cmd_pstrb      (cmd_pstrb),
    .cmd_pprot      (cmd_pprot),

    // Response interface (aclk domain)
    .rsp_valid      (rsp_valid),
    .rsp_ready      (rsp_ready),
    .rsp_prdata     (rsp_prdata),
    .rsp_pslverr    (rsp_pslverr)
);
```

13.9 Related Modules

- **APB Slave:** apb4_slave.md
- **Clock-Gated Variant:** apb4_slave_cdc_cg.md
- **APB5 Equivalent:** apb5_slave_cdc.md
- **CDC FIFO:** `rtl/cdc/gaxi_fifo_async.sv`

13.10 Testing

`val/amba/test_apb4_slave_cdc.py` exercises this module. It collects 2 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_apb4_slave_cdc.py -v
```

13.11 References

- **Source:** `rtl/amba/apb4/apb4_slave_cdc.sv`
- **Tests:** `val/amba/test_apb4_slave_cdc.py`
- **WaveDrom Test:** `val/amba/test_apb4_slave_cdc.py::test_apb4_slave_cdc_wavedrom`
- Arm. *AMBA APB Protocol Specification*, Version 2.0, ARM IHI 0024C, Apr. 2010. developer.arm.com/documentation/ihi0024/c – ch. 3 “Transfers”: the Completer side of §3.1 write and §3.3 read transfers (extending ACCESS with `PREADY` low), §3.2 `PSTRB`, §3.4 error response (`PSLVERR`), §3.5 `PPROT`; ch. 4 “Operating States”. Citation only.
- Cummings, C.E. “Simulation and Synthesis Techniques for Asynchronous FIFO Design.” *SNUG San Jose*, 2002 – the Gray-pointer async FIFO (`gaxi_fifo_async`) that carries the command and response across the two clocks.
- Ginosar, R. “Metastability and Synchronizers: A Tutorial.” *IEEE Design & Test of Computers*, 28(5), Sept./Oct. 2011. DOI: 10.1109/MDT.2011.113 – synchronizer depth versus MTBF for the pointer crossings.
- Deschamps, J.-P., Sutter, G.D., Cantó, E. *Guide to FPGA Implementation of Arithmetic Functions*, Springer (LNEE 149), 2012, ch. 5 “Electronic Aspects of Digital Design”, §5.3.5.3 “Asynchronous FIFO” – textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `rtl/cdc/references/AsyncFifoGrayPointers__Cummings2002.pdf`, `rtl/cdc/references/MetastabilitySynchronizers__Ginosar2011.pdf`)

Last Updated: 2026-07-19

13.12 Navigation

- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

14 apb4_slave_cdc_cg

Module: `apb4_slave_cdc_cg.sv` **Base Module:** apb4_slave_cdc **Location:** `rtl/amba/apb4/`
Status: Production Ready

14.1 Overview

This is the **clock-gated variant** of apb4_slave_cdc.

For the clock-gating architecture and the underlying gate cell, see:

→ Clock-Gated Variants Guide and **→ amba_clock_gate_ctrl**

Both live in the Shared book, which is published as a separate PDF. Take the parameter and port names for this module from the tables below, not from the generic guide.

The `apb4_slave_cdc_cg` module adds power optimization to `apb4_slave_cdc` through activity-based clock gating. **Both clock domains are gated independently** — the module instantiates two `amba_clock_gate_ctrl` blocks, one on `pclk` and one on `aclk`, sharing the same configuration inputs.

Structurally it's a sibling of `apb4_slave_cdc`, not a wrapper around it: it re-instantiates the same `apb4_slave` plus the same pair of `gaxi_fifo_async` CDC FIFOs, but drives them from the gated clocks. The CDC behaviour described in apb4_slave_cdc.md — gray pointers, `N_FLOP_CROSS=2`, no maximum clock ratio — applies unchanged. That includes the reset rule: a one-sided reset is NOT safe here either, for the same reason (the crossed pointer copy is a live synchronizer, not a snapshot).

While a domain is gated, that domain's `ready` outputs are forced low, so no handshake can complete against a stopped clock.

- **Same Functionality:** 100% equivalent to base module
- **Dual-Domain Gating:** Separate gate cell per clock domain
- **Runtime Control:** Gating is enabled and tuned by input signals, not parameters
- **Bypassable:** Tie `cfg_cg_enable = 0` for behaviour identical to the base module

---

14.2 Parameters

In addition to all apb4_slave_cdc parameters:

Parameter	Type	Default	Description
CG_IDLE_COUNT_WIDTH	int	4	Width of the idle countdown counter, shared by both domains

USE_2_PHASE_CDC is inherited from the base module and is likewise **deprecated and ignored**. There is no ENABLE_CLOCK_GATING parameter and no per-domain CG_GATE_* parameters.

14.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
STRB_WIDTH	DATA_WIDTH / 8
DW	DATA_WIDTH
AW	ADDR_WIDTH
SW	STRB_WIDTH
PW	PROT_WIDTH
CPW	AW + DW + SW + PW + 1
RPW	DW + 1

14.3 Ports

In addition to all apb4_slave_cdc ports:

Port	Width	Direction	Description
`cfg_cg_enable`	1	Input	Global clock-gate enable for both domains. 0 = never gate
`cfg_cg_idle_count`	CG_IDLE_COUNT_WIDTH	Input	Idle cycles to count down before gating
`cg_gating`	1	Output	Asserted while the `pclk` domain is gated
`cg_idle`	1	Output	Asserted one cycle after the `pclk`-domain activity terms go low. **Not an occupancy flag** — see below
`cg_gating`	1	Output	Asserted while the `aclk` domain is gated
`cg_idle`	1	Output	Asserted one cycle after the `aclk`-domain activity terms go low. **Not an occupancy flag** — see below

`*_cg_idle` does not know whether the FIFOs are empty.
`amba_clock_gate_ctrl` has no occupancy input at all — its whole idle logic is `r_wakeup <= user_valid || axi_valid;` and `assign idle = ~r_wakeup`. So idle asserts one cycle after the activity terms drop, whatever the CDC FIFOs hold. On this wrapper the activity terms mostly hide occupancy anyway: APB holds `s_apb_PSEL` until `PREADY`, and this slave asserts `PREADY` only when the response returns, so a stalled backend keeps `pclk_user_valid` high and `cg_idle` low for the whole stall; an unconsumed response likewise holds `w_rsp_valid`/`rsp_valid` high. Do not read that as "idle implies empty" — it does not, here or anywhere: idle is "nothing has been handed to me recently", the input to the gating countdown.

Do not use it as a safe-to-reset or safe-to-power-down qualifier; that requires an emptiness check this signal does not perform.

14.3.1 Clock-gating status outputs

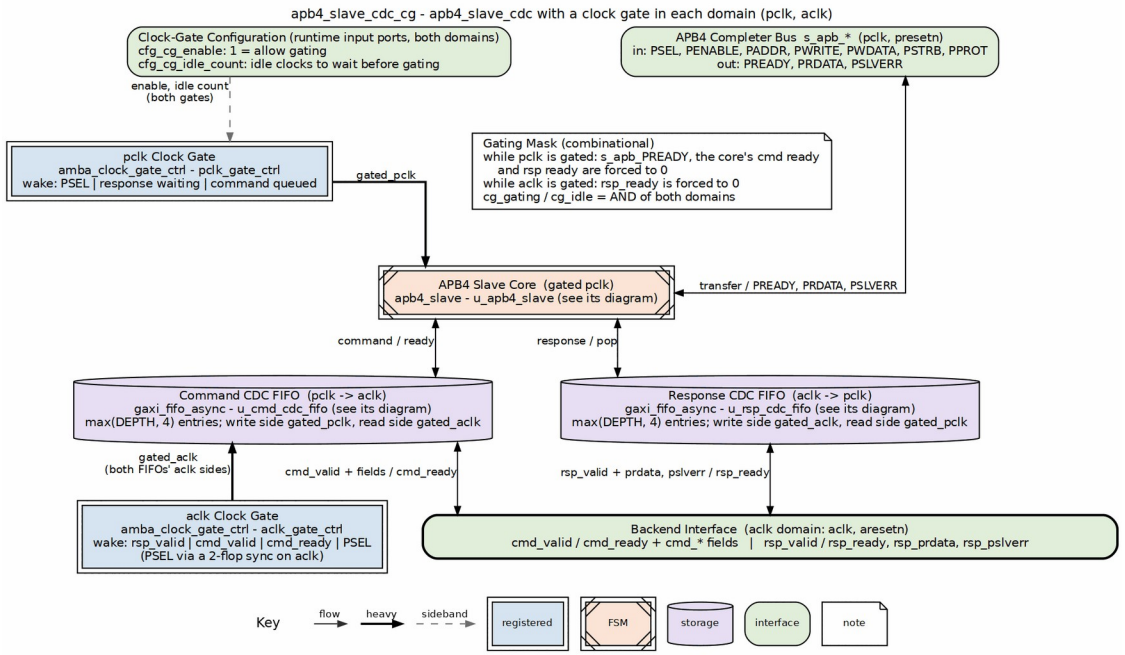
This wrapper spans TWO clock domains and gates each independently, so it carries three pairs rather than one. They are not interchangeable.

Port	Dir	Width	Description
`pclk_cg_gating`	Out	1	APB-domain clock gated. Masks `s_apb_PREADY` and the command-side ready.
`pclk_cg_idle`	Out	1	APB-domain activity terms quiet.
`aclk_cg_gating`	Out	1	AXI-domain clock gated. Masks the response-side ready.
`aclk_cg_idle`	Out	1	AXI-domain activity terms quiet.
`cg_gating`	Out	1	Whole-module aggregate: `pclk_cg_gating && aclk_cg_gating`. High only when BOTH domains are gated.
`cg_idle`	Out	1	Whole-module aggregate: `pclk_cg_idle && aclk_cg_idle`.

Use the per-domain signals to reason about one side's readys; use `cg_gating` only as the "is this block asleep" summary. Treating `cg_gating` as equivalent to `pclk_cg_gating` is wrong: with one domain busy the aggregate stays low while that domain's readys are still masked.

14.4 Functional Description

14.4.1 Block diagram



apb4_slave_cdc_cg block diagram

`apb4_slave_cdc` with a clock gate in each domain. The pclk gate's gated clock (heavy) runs the `apb4_slave` core and the pclk sides of both CDC FIFOs; the aclk gate's gated clock runs their aclk sides. Each gate wakes on its own domain's activity (the aclk gate sees PSEL through a two-flop synchronizer). While a domain is gated, the ready signals it drives are forced to 0 (the Gating Mask legend), and `cg_gating` / `cg_idle` are the AND of both domains.

This wrapper is the base module plus one `amba_clock_gate_ctrl` instance. The datapath is untouched – every channel signal is forwarded verbatim – so functional behaviour is identical to the base module and the wrapper adds no latency of its own.

What it adds is a gated clock. `amba_clock_gate_ctrl` watches two activity terms, `user_valid` (this side's valids plus the base module's `busy`) and `axi_valid` (the far side's valids), registers their OR into `r_wakeup`, and stops the inner module's clock once both have been quiet for `cfg_cg_idle_count` cycles. The clock restarts on the next activity, one cycle later.

While the clock is stopped the wrapper masks its outward-facing READY signals with `! cg_gating`, so a peer sees no acceptance until the clock runs again and no handshake is lost across the wake boundary.

`cfg_cg_enable` arms this behaviour; with it low the clock free-runs and the module is indistinguishable from its base.

14.5 Timing Characteristics

This module is **sequential**: it contains 1 `always_ff` block(s), clocked on `aclk` with active-low asynchronous reset `aresetn`. Outputs derived in those blocks are registered and therefore appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

14.6 Usage Examples

```
apb4_slave_cdc_cg #(
    // Base module parameters (see apb4_slave_cdc.md)
    .ADDR_WIDTH          (32),
    .DATA_WIDTH          (32),
    .DEPTH               (2),
    .USE_JOHNSON         (0),   // 0 = Gray: DEPTH in {2,3,4,8}
(floored max(DEPTH,4) must be pow2); 1 = Johnson: any DEPTH 2..8

    // Clock gating
    .CG_IDLE_COUNT_WIDTH (4)
) u_cg (
    .pclk              (apb_clk),
    .presetn           (apb_resetn),
    .aclk              (core_clk),
    .aresetn           (core_resetn),

    // Clock gating control (applies to both domains)
    .cfg_cg_enable     (1'b1),
    .cfg_cg_idle_count (4'd8),

    // Per-domain gating status
    .cg_gating    (pclk_gated),
    .cg_idle      (pclk_idle),
    .cg_gating    (aclk_gated),
    .cg_idle      (aclk_idle),

    // ... all other ports same as apb4_slave_cdc
);
```

Simulation note: set `cfg_cg_enable = 0` when debugging CDC waveforms. Two independently gated clocks make cross-domain timing very hard to read.

14.7 Design Notes

A peer's READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that signal into `user_valid` pins this block permanently awake and defeats gating entirely – silently, because function is unaffected. Ten wrappers in this repository shipped that way and nothing failed until someone measured. `val/amba/test_cg_peer_ready.py` parks the peer READY high, holds every VALID low, and requires `cg_gating`. Canonical rule: `vault/handbook/design/clock-gating-activity-terms.md`.

`cfg_cg_enable` is not a kill switch. It arms gating and reaches `amba_clock_gate_ctrl` only; the datapath and any monitor enables are forwarded untouched. With it low the clock free-runs and this module behaves exactly like its base.

Gating latency. The clock stops `cfg_cg_idle_count` + 2 cycles after the last bus activity – the idle counter, plus one for the `r_wakeup` flop. Size the idle count against your traffic's inter-burst gap: too small and the block wakes constantly, too large and it never gates.

Cost. Five flops: `r_wakeup` plus `r_idle_counter` at `IDLE_CNTR_WIDTH`, scaling as 1 + `CG_IDLE_COUNT_WIDTH`. The ICG itself is a latch or BUFGCE, not fabric flops.

14.8 Related Modules

- **Base Module Functionality:** apb4_slave_cdc.md
- **Clock Gating Guide:** clock_gated_variants.md
- **Detailed CG Examples:**
  - axi4_master_rd_mon_cg.md (AXI4 monitor)
  - axil4_master_rd_mon_cg.md (AXIL4 monitor)
  - apb4_slave_cg.md (APB interface)

14.9 Testing

`val/amba/test_apb4_slave_cdc_cg.py` exercises this module. It collects 2 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_apb4_slave_cdc_cg.py -v
```

14.10 References

- Arm. *AMBA APB Protocol Specification*, Version 2.0, ARM IHI 0024C, Apr. 2010. developer.arm.com/documentation/ihi0024/c — ch. 3 “Transfers”: the Completer side of §3.1 write and §3.3 read transfers (extending ACCESS with `PREADY` low), §3.2 `PSTRB`, §3.4 error response (`PSLVERR`), §3.5 `PPROT`; ch. 4 “Operating States”. Citation only.
- Cummings, C.E. “Simulation and Synthesis Techniques for Asynchronous FIFO Design.” *SNUG San Jose*, 2002 — the Gray-pointer async FIFO (`gaxi_fifo_async`) that carries the command and response across the two clocks.
- Ginosar, R. “Metastability and Synchronizers: A Tutorial.” *IEEE Design & Test of Computers*, 28(5), Sept./Oct. 2011. DOI: 10.1109/MDT.2011.113 — synchronizer depth versus MTBF for the pointer crossings.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 — the fixed-timeout policy `amba_clock_gate_ctrl` implements: gate after `cfg_cg_idle_count` idle cycles, wake on activity.
- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. CAD*, 15(6), June 1996. DOI: 10.1109/43.503933 — the latch-based gated clock (`icg`) at the bottom of the gating chain.
- Deschamps, J.-P., Sutter, G.D., Cantó, E. *Guide to FPGA Implementation of Arithmetic Functions*, Springer (LNEE 149), 2012, ch. 5 “Electronic Aspects of Digital Design”, §5.3.5.3 “Asynchronous FIFO” — textbook treatment (secondary source).
- Roy, S. *Advanced Digital System Design*, Springer, ch. 15 “Low-Power Digital System Design”, §15.5.5 “Logic Level Power Down” — textbook treatment of clock gating (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `rtl/cdc/references/AsyncFifoGrayPointers__Cummings2002.pdf`, `rtl/cdc/references/MetastabilitySynchronizers__Ginosar2011.pdf`, `rtl/common/references/DynamicPowerManagementTimeout__Benini2000.pdf`, `rtl/common/references/LatchBasedClockGating__Benini1996.pdf`)

14.11 Navigation

- ← Back to APB Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

15 APB5 Slave CDC

Module: `apb5_slave_cdc.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

15.1 Overview

The APB5 Slave CDC module provides clock domain crossing between an APB5 bus clock domain and a backend clock domain: full APB5 slave on the `pclk` side, a command/response backend interface on the `aclk` side, and transactions carried safely across the asynchronous boundary in between.

15.1.1 Key Features

- Full APB5 protocol support with CDC
- Asynchronous FIFO-based clock domain crossing (Gray pointers by default; Johnson available via `USE_JOHNSON`, and opt-in by design)
- All APB5 user signals (PAUSER, PWUSER, PRUSER, PBUSER)
- PWAKEUP is a `pclk`-domain input from the Requester (IHI 0024E §3.7.2)
- Single `DEPTH` parameter sizes both directions
- Metastability protection with 2-flop pointer synchronizers

15.2 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
PROT_WIDTH	int	3	Protection signal width
AUSER_WIDTH	int	4	Address user signal width
WUSER_WIDTH	int	4	Write user signal width
RUSER_WIDTH	int	4	Read user signal width
BUSER_WIDTH	int	4	Response user signal width
DEPTH	int	2	Skid-buffer depth of the wrapped apb5_slave; 2..8 inclusive (any integer)
USE_JOHNSON	int	0 (Gray)	CDC-FIFO pointer encoding: 0 Gray, 1 Johnson, -1 auto-select. See below.
ENABLE_PARITY	bit	0	Enable parity generation and checking
USE_2_PHASE_CDC	bit	1	Deprecated and ignored — retained for source compatibility

There is no CMD_DEPTH, RSP_DEPTH or SYNC_STAGES parameter on this module. The two asynchronous FIFOs are sized internally from DEPTH:

```
localparam int CDC_FIFO_DEPTH = (DEPTH < 4) ? 4 : DEPTH;
```

so the CDC FIFOs are never shallower than 4 entries regardless of DEPTH. The synchronizer depth is fixed at 2 flops (N_FLOP_CROSS(2) on both gaxi_fifo_async instances) and is not exposed as a parameter. If your design needs 3-flop synchronization for an extreme clock ratio, you're editing the instantiation in the RTL.

15.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
DW	DATA_WIDTH
AW	ADDR_WIDTH
SW	STRB_WIDTH
PW	PROT_WIDTH
AUW	AUSER_WIDTH
WUW	WUSER_WIDTH
RUW	RUSER_WIDTH
BUW	BUSER_WIDTH
CPW	AW + DW + SW + PW + AUW + WUW + 1
RPW	DW + RUW + BUW + 1

15.3 Ports

15.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB bus clock
presetn	1	Input	APB reset (active low)
aclk	1	Input	Backend/user clock
aresetn	1	Input	Backend/user reset (active low)

The backend clock and reset are named `aclk` and `aresetn`, matching the rest of the AMBA library. There are no `bclk`/`bresetn` ports.

15.3.2 APB5 Slave Interface

Same as apb5_slave - operates in `pclk` domain, including the optional parity signals.

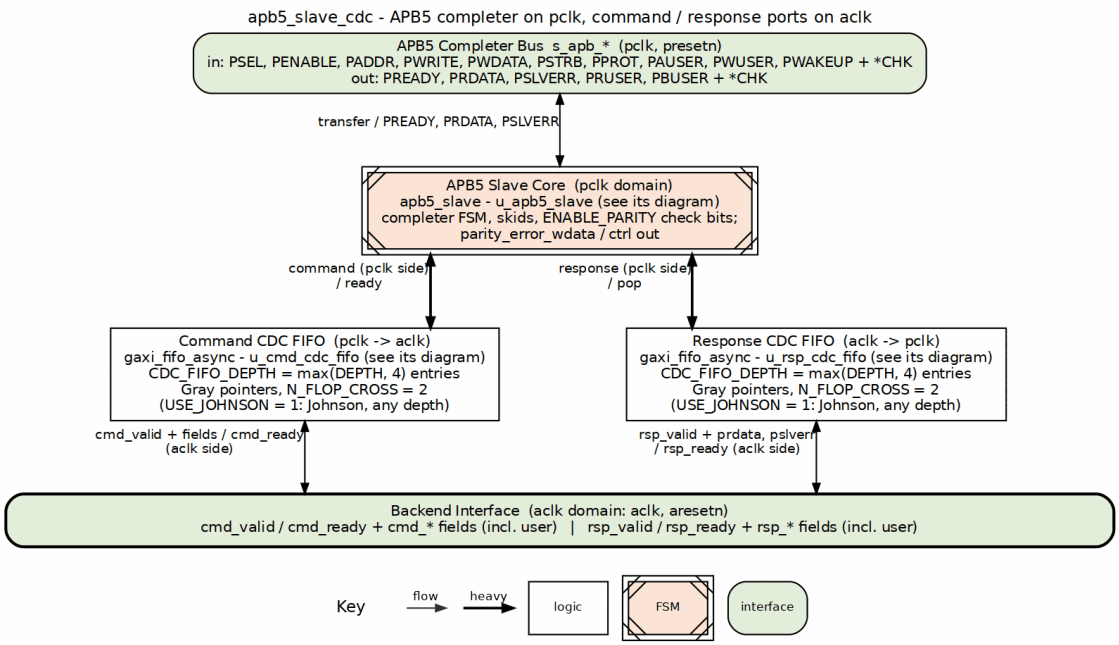
15.3.3 Backend Interface

Same command/response interface as apb5_slave - operates in the `aclk` domain. There is no backend `wakeup_request` input: PWAKEUP is a Requester signal (IHI 0024E §3.7.2) and arrives on the `pclk`-side APB port.

`parity_error_wdata` and `parity_error_ctrl` are **not** `aclk`-domain signals, despite sitting next to these ports. `apb5_slave` drives them combinationally from the APB inputs, qualified by each check's IHI 0024E Table 5-1 Check Enable term (`PSEL`, `PSEL & PWRITE`, or reset deasserted for `PSELCHK`; see apb5_slave), so they are `pclk`-domain (gated-`pclk` in the CG variant) and they cross no synchronizer. Sample them in `pclk`, or synchronize them yourself before using them in `aclk`.

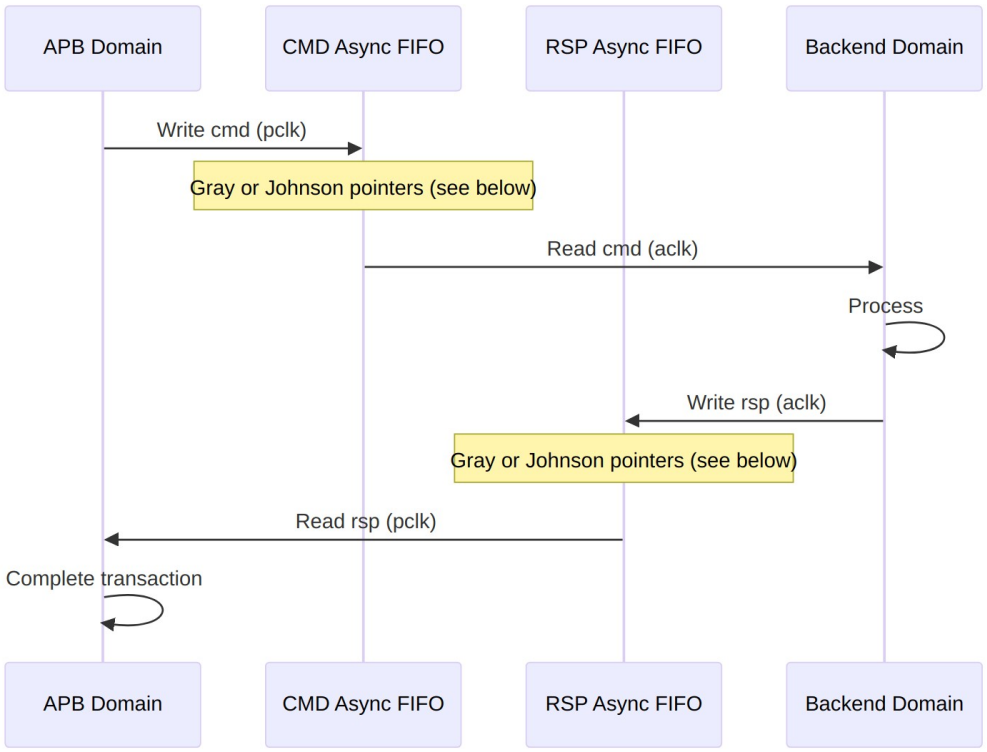
15.4 Functional Description

15.4.1 Architecture



apb5_slave_cdc block diagram

The APB side runs on `pclk`: the bus feeds the wrapped `apb5_slave` (one box; see its diagram). Each direction crosses to the `aclk` backend through its own `gaxi_fifo_async` (`max(DEPTH, 4)` entries): the command drops down the left lane (pclk to aclk), the response climbs the right lane (aclk to pclk). The Gray-pointer crossings are inside the async FIFOs.



CDC Mechanism

Both directions cross through `gaxi_fifo_async` instances with a fixed 2-flop synchronizer (`N_FLOP_CROSS(2)`) on each pointer crossing. The command FIFO is written in `pclk` and read in `aclk`; the response FIFO is written in `aclk` and read in `pclk`.

Every crossing goes through one of the two FIFOs. (The old `wakeup_request` level synchronizer is gone with that port: PWAKEUP is a Requester output and enters on the `pclk` side, so it needs no crossing here.)

15.4.2 CDC FIFO Pointer Encoding

The wrapper derives `CDC_FIFO_DEPTH = (DEPTH < 4) ? 4 : DEPTH` and hands that to two `gaxi_fifo_async` instances (command and response). Gray pointers only close on a power-of-2 depth, so a FIFO depth of 6 cannot use Gray — `gaxi_fifo_async` carries an elaboration-time `$error` for exactly that case.

`USE_JOHNSON` selects the encoding:

USE_JOHNSON	Encoding	Pointer width	Depth constraint
`0` (default)	Gray	`$clog2(max(DEPTH, 4))+1`	floored `max(DEPTH, 4)` must be a power of 2: DEPTH in {2,

USE_JOHNSON	Encoding	Pointer width	Depth constraint
			3, 4, 8}
1	Johnson	`max(DEPTH, 4)` bits	any depth
-1	auto	per depth	none — Gray when the derived FIFO depth is a power of 2, Johnson otherwise

The default is Gray, not auto. With defaults, DEPTH 2, 3, 4 and 8 elaborate (2, 3 and 4 all derive a depth-4 FIFO) and **DEPTH 5, 6 and 7 fail the build**. That is intentional. Johnson costs `max(DEPTH, 4)`-bit pointers against Gray's `$clog2(max(DEPTH, 4))+1` (widths follow the floored FIFO depth, so the default `DEPTH = 2` pays 4 bits vs 3) — at depth 6, 6 bits against 4 — duplicated in both domains and again in every synchronizer stage. Nobody should pay that because a default quietly decided for them.

If you want a non-power-of-2 depth, say so: pass `USE_JOHNSON=1` for Johnson, or `USE_JOHNSON=-1` to restore per-depth auto-selection. The capability is intact; only the default changed, so that the choice is visible in the instantiation.

`USE_JOHNSON=0` with `DEPTH=6` is an elaboration error, deliberately. Asking for Gray at a non-power-of-2 depth is a real configuration mistake and should fail the build.

The pointer encoding is selectable, and defaults to Gray. `USE_JOHNSON` feeds a localparam that resolves against the DERIVED depth:

```
localparam int CDC_FIFO_DEPTH  = (DEPTH < 4) ? 4 : DEPTH;
localparam bit CDC_DEPTH_POW2  = ((CDC_FIFO_DEPTH & (CDC_FIFO_DEPTH -
1)) == 0);
localparam int CDC_USE_JOHNSON = (USE_JOHNSON >= 0) ? USE_JOHNSON
                                                    :
(CDC_DEPTH_POW2 ? 0 : 1);
```

With the default `USE_JOHNSON = 0`, `CDC_USE_JOHNSON` is 0 and the crossing is Gray — the default `DEPTH = 2` derives 4, a power of 2, so it elaborates. `DEPTH = 6` would not; pass `USE_JOHNSON = 1` for 6-bit Johnson pointers, or `-1` to let the derived depth decide. See the CDC FIFO pointer encoding section above for the full table.

15.4.3 Reset Synchronization

`presetn` and `aresetn` are independent reset domains and either may be asserted alone. **A one-sided reset is not safe. Quiesce the bus first.**

The local reset clears that domain's own pointer, but the crossed copy of the *remote* pointer is a live synchronizer (`glitch_free_n_dff_arn`, N=2) that keeps sampling the non-reset domain the moment reset deasserts — it does not hold at zero. The reset side therefore comes back with its own pointer at zero and the remote pointer at whatever the other side had reached, which is not an empty FIFO. It is a mismatched one.

Two concrete consequences, neither of which is a clean discard:

- **Consumed commands are re-presented.** Pulsing `aresetn` with an **already-consumed** command in the cmd FIFO rewinds the read pointer behind the write pointer, so the backend sees that command again after reset. It does not time out — it re-executes. An *unread* command is not at risk: its read pointer is already behind the write pointer, so resetting it to 0 rewinds nothing and the entry is delivered exactly once.
- **The response FIFO can fabricate responses.** The same rewind on the response path presents entries the APB side never queued, so the APB master can complete a transfer that the backend did not answer.

Pointers being absolute positions rather than toggle parity is what makes the *steady-state* crossing reliable; it does not make a one-sided reset safe.

Neither reset is internally synchronized to the other domain's clock; each is expected to be already synchronized (or asynchronously asserted and synchronously deasserted) in its own domain by the integrator.

15.5 Timing Characteristics

15.5.1 Latency

- Crossing latency is the async-FIFO write-to-read pointer synchronization: on the order of 2-3 destination-clock cycles per direction, plus the source-clock cycle that performs the write
- A full APB transfer therefore costs the command crossing plus the response crossing before PREADY can assert
- Additional latency if the FIFOs are full (backpressure) or the backend is slow

15.5.2 Timing Considerations

Timing diagram pending. The signals and sequence this scenario exercises:

- pclk
- aclk (different frequency)
- APB transaction signals
- FIFO write/read pointers
- Backend transaction signals
- CDC latency

15.6 Usage Examples

```
apb5_slave_cdc #(
    .ADDR_WIDTH     (32),
    .DATA_WIDTH     (32),
    .AUSER_WIDTH    (4),
    .WUSER_WIDTH    (4),
    .RUSER_WIDTH    (4),
    .BUSER_WIDTH    (4),
    .DEPTH          (2),
    .ENABLE_PARITY  (0)
) u_apb5_slave_cdc (
    // APB clock domain
    .pclk           (apb_clk),
    .presetn        (apb_rst_n),

    // Backend clock domain
    .aclk           (backend_clk),
    .aresetn        (backend_rst_n),

    // APB5 slave interface (pclk domain)
    .s_apb_PSEL     (s_apb_psel),
    .s_apb_PENABLE  (s_apb_penable),
    // ... other APB signals

    // Backend interface (aclk domain)
    .cmd_valid      (backend_cmd_valid),
    .cmd_ready      (backend_cmd_ready)
    // ... other backend signals
);
```

15.7 Design Notes

15.7.1 FIFO Depth Sizing

- `DEPTH` sizes the `apb5_slave` skid buffers; the CDC FIFOs are sized separately as `max(DEPTH, 4)`, so the crossing itself is never shallower than 4 entries even at the default `DEPTH=2`
- Deeper FIFOs buy tolerance for a slow backend or a bursty APB master; they do not reduce the per-transfer crossing latency
- Because APB is single-outstanding, depth beyond a handful of entries has little effect on throughput for this module

15.8 Related Modules

- **APB5 Slave** - Base slave without CDC
- **APB5 Slave CDC CG** - CDC with clock gating
- **GAXI Async FIFO** - The async FIFO used for both crossings
- **Clock Domain Crossing** - CDC design patterns and reset behavior

15.9 Testing

`val/amba/test_apb5_slave_cdc.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_apb5_slave_cdc.py -v
```

15.10 References

- Arm. *AMBA APB Protocol Specification*, ARM IHI 0024E, Feb. 2023 (APB5 first published in Issue D, Apr. 2021). developer.arm.com/documentation/ihi0024/e — ch. 3 “Transfers”, ch. 4 “Operating States”, §3.7 “Wake-up signaling” and §3.8 “User signaling”: the protocol of the wrapped APB5 block. Citation only.
- Cummings, C.E. “Simulation and Synthesis Techniques for Asynchronous FIFO Design.” *SNUG San Jose*, 2002 — the Gray-pointer async FIFO (`gaxi_fifo_async`) that carries the command and response across the two clocks.
- Ginosar, R. “Metastability and Synchronizers: A Tutorial.” *IEEE Design & Test of Computers*, 28(5), Sept./Oct. 2011. DOI: 10.1109/MDT.2011.113 — synchronizer depth versus MTBF for the pointer crossings.
- Deschamps, J.-P., Sutter, G.D., Cantó, E. *Guide to FPGA Implementation of Arithmetic Functions*, Springer (LNEE 149), 2012, ch. 5 “Electronic Aspects of Digital Design”, §5.3.5.3 “Asynchronous FIFO” — textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `rtl/cdc/references/AsyncFifoGrayPointers__Cummings2002.pdf`, `rtl/cdc/references/MetastabilitySynchronizers__Ginosar2011.pdf`)

15.11 Navigation

- ← Back to APB5 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

16 APB5 Slave CDC (Clock-Gated)

Module: `apb5_slave_cdc_cg.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

16.1 Overview

The APB5 Slave CDC + Clock-Gated module combines clock domain crossing with clock gating for maximum power efficiency. It wraps `apb5_slave_cdc` with clock gating control to cut power during idle periods while keeping operation safe across asynchronous clock boundaries.

16.1.1 Key Features

- Full APB5 protocol support with all extensions
- Asynchronous clock domain crossing (APB to backend)
- Clock gating for power reduction during idle
- All APB5 user signals (PAUSER, PWUSER, PRUSER, PBUSER)
- Wakes the `pclk` clock gate on the Requester's PWAKEUP (IHI 0024E §3.7.2)
- Optional IHI 0024E ch. 5 interface parity (`*CHK` signals, odd parity)
- Automatic wake-up on transaction activity

16.2 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
PROT_WIDTH	int	3	Protection signal width
AUSER_WIDTH	int	4	Address user signal width
WUSER_WIDTH	int	4	Write user signal width
RUSER_WIDTH	int	4	Read user signal width
BUSER_WIDTH	int	4	Response user signal width
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
DEPTH	int	2	Skid-buffer depth of the wrapped slave; CDC FIFOs use `max(DEPTH, 4)`
USE_JOHNSON	int	0 (Gray)	Forwarded to `apb5_slave_cdc`: `0` Gray, `1` Johnson, `-1` auto. Newly reachable from this level — it used to be unreachable, so a CG instance could not select the encoding at all.
ENABLE_PARITY	bit	0	Enable parity generation and checking
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter (max idle = 2^N-1 cycles)
USE_2_PHASE_CDC	bit	1	Deprecated and ignored — retained for source compatibility

As with apb5_slave_cdc, there is no `SYNC_STAGES` parameter: pointer synchronization is fixed at 2 flops inside the async FIFOs.

16.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
DW	DATA_WIDTH
AW	ADDR_WIDTH
SW	STRB_WIDTH
PW	PROT_WIDTH
AUW	AUSER_WIDTH
WUW	WUSER_WIDTH
RUW	RUSER_WIDTH
BUW	BUSER_WIDTH
ICW	CG_IDLE_COUNT_WIDTH
CPW	AW + DW + SW + PW + AUW + WUW + 1
RPW	DW + RUW + BUW + 1

16.3 Ports

16.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB bus clock
presetn	1	Input	APB reset (active low)
aclk	1	Input	User/backend clock
aresetn	1	Input	User reset (active low)

16.3.2 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Enable clock gating
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating

16.3.3 APB5 Slave Interface

Same as apb5_slave - operates in pclk domain.

16.3.4 Backend Interface

Same command/response interface as apb5_slave_cdc - operates in aclk domain.

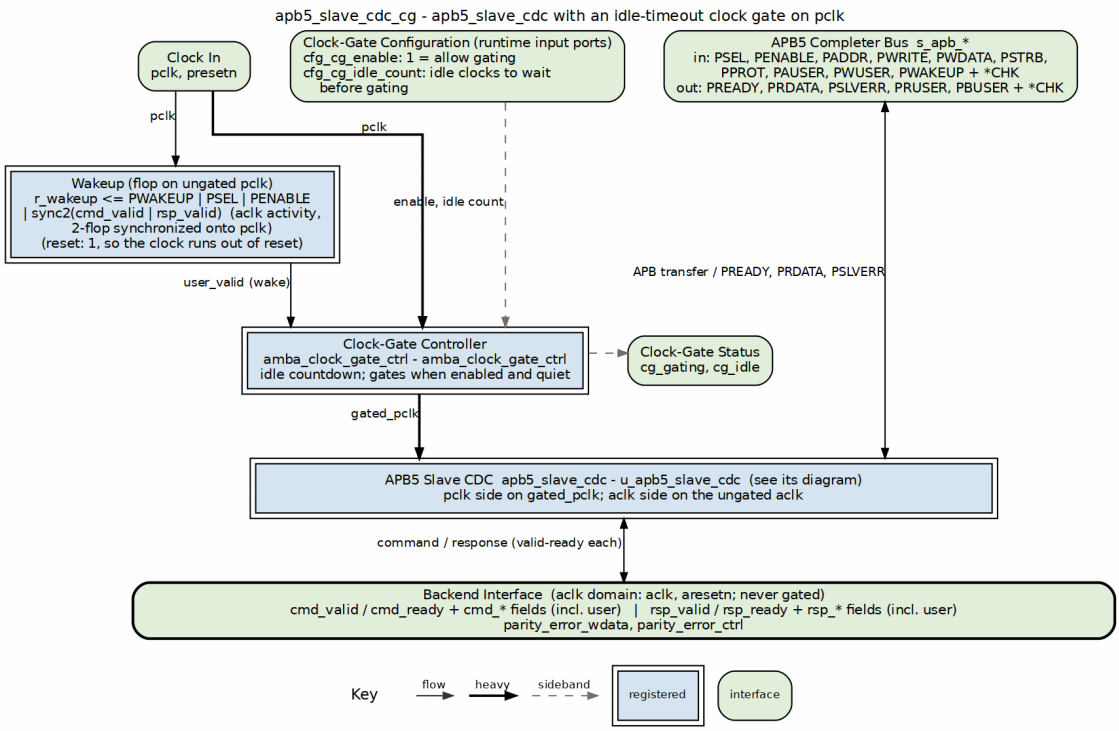
16.3.5 Status Outputs

Port	Width	Direction	Description
cg_gating	1	Output	Indicates clock is currently gated
cg_idle	1	Output	Activity terms quiet (registered ~wakeup).
parity_error_wdata	1	Output	PWDATACHK, PSTRBCHK or PWUSERCHK mismatch (gated-pclk domain)
parity_error_ctrl	1	Output	PADDRCHK, PCTRLCHK, PSELCHK, PENABLECHK

Port	Width	Direction	Description
			or PAUSERCHK mismatch (gated-`pclk` domain)

16.4 Functional Description

16.4.1 Architecture



apb5_slave_cdc_cg block diagram

The whole `apb5_slave_cdc` is one box (see its diagram). One clock gate, on `pclk` only: its wakeup flop ORs PWAKEUP, PSEL and PENABLE with the `aclk`-side activity (`cmd_valid | rsp_valid`), which is brought onto `pclk` through a two-flop synchronizer first. The gated clock (heavy) runs the pclk side; the aclk side is never gated.

16.4.2 Wake-up Logic

The wakeup terms are in the block diagram's Wakeup box. The `aclk`-domain activity terms are ORed together and passed through a two-flop synchronizer before being combined with the `pclk`-domain terms. Without that synchronizer, these signals would cross domains unsynchronized.

16.5 Timing Characteristics

Timing diagram pending. The signals and sequence this scenario exercises:

- pclk (ungated)
- gated_pclk
- aclk (different frequency)
- s_apb_PSEL (wake trigger)
- cg_gating indicator
- Transaction flow across CDC with gating
- Wake-up latency from PSEL to clock active

16.6 Usage Examples

```
apb5_slave_cdc_cg #(
    .ADDR_WIDTH         (32),
    .DATA_WIDTH         (32),
    .AUSER_WIDTH        (4),
    .WUSER_WIDTH        (4),
    .RUSER_WIDTH        (4),
    .BUSER_WIDTH        (4),
    .DEPTH              (2),
    .ENABLE_PARITY      (0),
    .CG_IDLE_COUNT_WIDTH(4)
) u_apb5_slave_cdc_cg (
    // APB clock domain
    .pclk               (apb_clk),
    .presetn            (apb_rst_n),

    // User clock domain
    .aclk               (user_clk),
    .aresetn            (user_rst_n),

    // Clock gating
    .cfg_cg_enable      (1'b1),
    .cfg_cg_idle_count  (4'd8),
    .cg_gating   (slave_clk_gated),

    // APB5 slave interface (pclk domain)
    .s_apb_PSEL         (s_apb_psel),
    .s_apb_PENABLE      (s_apb_penable),
    .s_apb_PREADY       (s_apb_pready),
    // ... other APB signals

    // Backend interface (aclk domain)
    .cmd_valid          (backend_cmd_valid),
    .cmd_ready          (backend_cmd_ready),
    // ... other command signals

    .rsp_valid          (backend_rsp_valid),
    .rsp_ready          (backend_rsp_ready)
    // ... other response signals
);
```

16.7 Design Notes

16.7.1 Power and Latency Trade-offs

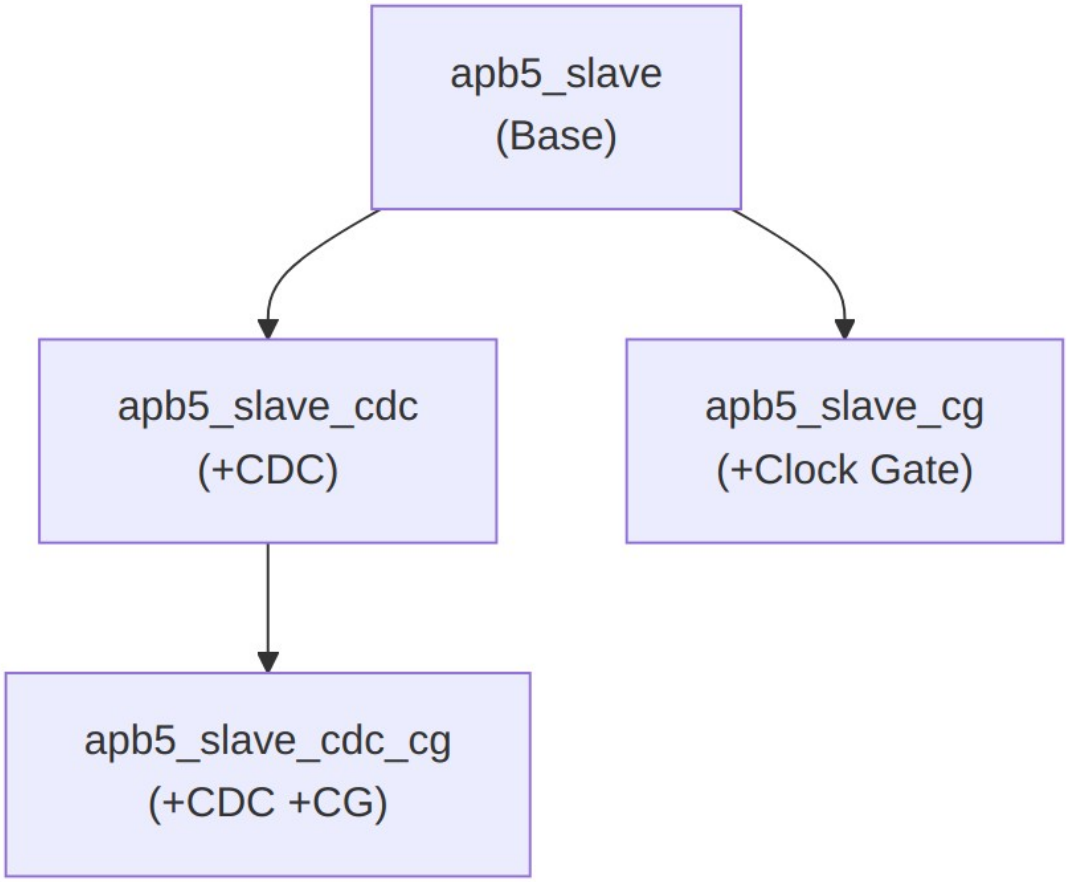
Wake-up is **registered, not combinational**. Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream, APB) or twice (APB5, AXI5-Stream) before reaching the ICG enable, which is combinational. A pclk-domain trigger (PWAKEUP, PSEL or PENABLE) passes through this wrapper's `r_wakeup` and then through the flop inside `amba_clock_gate_ctrl`, so APB5 is a two-stage family and the first usable gated `pclk` rising edge arrives **3 ungated `pclk` cycles** after the trigger. An aclk-domain trigger (`cmd_valid`, `rsp_valid`) first crosses a 2-flop synchronizer into `pclk`, adding about two more `pclk` cycles.

Configuration	Power Savings	Wake-up Latency from PSEL
CG disabled	None	0 cycles (clock always running)
CG idle=4	Moderate	2 register stages; first usable edge 3 ungated pclk cycles
CG idle=16 [1]	Good	2 register stages; first usable edge 3 ungated pclk cycles

The wake-up latency does not depend on `cfg_cg_idle_count`; the idle count only controls how long the block waits before gating. Latency is absorbed as APB wait states, since the slave holds PREADY low until it is running again.

Deliberately keeping the wake-up path registered avoids driving a clock-gate enable from a combinational function of a bus input. The gating element itself is an ICG cell instantiated by `clock_gate_ctrl`, which is glitch-free by construction. ICG cells are an ASIC library primitive; FPGA targets should use a clock-enable approach instead.

[1] `idle=16` requires `CG_IDLE_COUNT_WIDTH >= 5`. At the default of 4 the counter holds 0-15, so writing 16 truncates to 0 and gates after the first idle cycle – the opposite of the intent. Raise the parameter for this row.



Combined Feature Hierarchy

16.7.2 Reset Considerations

- APB domain reset (`presetn`) controls the APB interface, the wake-up synchronizer and the clock gate
- User domain reset (`aresetn`) controls the backend side of the CDC FIFOs
- Each reset must already be synchronized (or asynchronously asserted and synchronously deasserted) in its own domain by the integrator; this module does not synchronize either reset into the other domain
- A one-sided reset is NOT safe: the crossed pointer copy is a live synchronizer, so the reset side returns with its own pointer at zero against a remote pointer that kept advancing. Quiesce the bus first — see apb5_slave_cdc for the mechanism and for the in-flight transaction caveat

16.8 Related Modules

- **APB5 Slave** - Base slave module
- **APB5 Slave CDC** - CDC variant without clock gating
- **APB5 Slave CG** - Clock gating without CDC

16.9 Testing

`val/amba/test_apb5_slave_cdc_cg.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_apb5_slave_cdc_cg.py -v
```

16.10 References

- Arm. *AMBA APB Protocol Specification*, ARM IHI 0024E, Feb. 2023 (APB5 first published in Issue D, Apr. 2021). developer.arm.com/documentation/ihi0024/e — ch. 3 “Transfers”, ch. 4 “Operating States”, §3.7 “Wake-up signaling” and §3.8 “User signaling”: the protocol of the wrapped APB5 block. Citation only.
- Cummings, C.E. “Simulation and Synthesis Techniques for Asynchronous FIFO Design.” *SNUG San Jose*, 2002 — the Gray-pointer async FIFO (`gaxi_fifo_async`) that carries the command and response across the two clocks.
- Ginosar, R. “Metastability and Synchronizers: A Tutorial.” *IEEE Design & Test of Computers*, 28(5), Sept./Oct. 2011. DOI: 10.1109/MDT.2011.113 — synchronizer depth versus MTBF for the pointer crossings.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 — the fixed-timeout policy `amba_clock_gate_ctrl` implements: gate after `cfg_cg_idle_count` idle cycles, wake on activity.
- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. CAD*, 15(6), June 1996. DOI: 10.1109/43.503933 — the latch-based gated clock (`icg`) at the bottom of the gating chain.
- Deschamps, J.-P., Sutter, G.D., Cantó, E. *Guide to FPGA Implementation of Arithmetic Functions*, Springer (LNEE 149), 2012, ch. 5 “Electronic Aspects of Digital Design”, §5.3.5.3 “Asynchronous FIFO” — textbook treatment (secondary source).
- Roy, S. *Advanced Digital System Design*, Springer, ch. 15 “Low-Power Digital System Design”, §15.5.5 “Logic Level Power Down” — textbook treatment of clock gating (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `rtl/cdc/references/AsyncFifoGrayPointers__Cummings2002.pdf`, `rtl/cdc/references/MetastabilitySynchronizers__Ginosar2011.pdf`, `rtl/common/references/DynamicPowerManagementTimeout__Benini2000.pdf`, `rtl/common/references/LatchBasedClockGating__Benini1996.pdf`)

16.11 Navigation

- ← Back to APB5 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index